

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
DEPARTAMENTO ACADÊMICO DE ELETRÔNICA
ENGENHARIA INDUSTRIAL ELÉTRICA COM ÊNFASE EM ELETRÔNICA E
TELECOMUNICAÇÕES

MARCO AURÉLIO BARBOSA FAGNANI GOMES LOTZ

PROCESSAMENTO DISTRIBUÍDO EXTENSÍVEL UTILIZANDO UM CLUSTER E
MAPREDUCE COM APLICAÇÃO NA DETECÇÃO DE NÓDULOS PULMONARES
PARA GRANDES VOLUMES DE TOMOGRAFIAS

TRABALHO DE CONCLUSÃO DE CURSO

CURITIBA

2014

MARCO AURÉLIO BARBOSA FAGNANI GOMES LOTZ

PROCESSAMENTO DISTRIBUÍDO EXTENSÍVEL UTILIZANDO UM CLUSTER E
MAPREDUCE COM APLICAÇÃO NA DETECÇÃO DE NÓDULOS PULMONARES
PARA GRANDES VOLUMES DE TOMOGRAFIAS

Trabalho de Conclusão de Curso apresentado
à disciplina de Trabalho de Conclusão de
Curso, como requisito parcial para obtenção
de grau de Engenheiro do curso de
Engenharia Industrial Elétrica com ênfase em
Eletrônica e Telecomunicações, da
Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Dr. Gustavo Benvenutti
Borba

Co-orientador: Prof. Dr. Charlie Antoni
Miquelin

CURITIBA

2014

AGRADECIMENTOS

Com toda a certeza uma página não é o suficiente para agradecer a todas as pessoas que me ofereceram porto seguro durante o período em que estava trilhando meu caminho para me tornar engenheiro de telecomunicações e eletrônico. Tentando ainda assim (e provavelmente esquecendo algumas), gostaria de mencionar as que tiveram maior relevância nesta jornada.

Primeiramente agradeço ao meu pai, Jaime Luiz Gomes, e à minha mãe Cíntia Denise Barbosa Lotz, que sempre me apoiaram – seja emocionalmente ou financeiramente – em diversos momentos. Quando chegou minha hora de partir para conhecer outra realidade cultural, eles foram os primeiros a me garantir apoio em caso de necessidade e me confortar quando me deparei com o saudosismo da minha terra natal.

Agradeço ao meu orientador, Gustavo Benvenutti Borba, pela sua bravura em me orientar em uma área ainda não explorada dentro desta instituição e por sempre se fazer disponível, inclusive durante suas férias, para elucidar inúmeras dúvidas minhas na área de processamento de imagens.

Em seguida, gostaria de agradecer aos professores do departamento acadêmico de física – DAFIS. Em especial os professores Charlie Antoni Miquelin, Rafael de Carvalho Barreto, Rosangela Requi Jakubiak, Pedro Zambianchi Júnior e a aluna de radiologia Yasmim Rocha de Paula, por sempre depositarem em mim sua confiança, me auxiliando sempre que necessitei de seus recursos e me provendo de um ótimo ambiente para desenvolver este trabalho de conclusão de curso.

Não menos importante é o apoio de meus amigos: Luiz Gustavo Dal Pai, Felipe Thomaz Grob, Rodrigo Bozza, Romulo de Lazzari, Fátima Botella Segura e Telmo Machado Júnior. Nestes anos de jornadas, eles sempre me compreenderam e buscaram me ensinar não somente conhecimentos acadêmicos, mas sim valores de vida.

Por último, gostaria de agradecer ao professor Félix Cuadrado, que durante meu período de pesquisa na Queen Mary – University of London, me introduziu a área de Big Data e me proveu de todos os recursos que ele possuía no momento, inclusive muito de seu tempo, em prol de meu aprendizado na área.

*"Se consegui enxergar mais
longe é porque procurei ver acima dos
ombros dos gigantes."*

Sir Isaac Newton

RESUMO

LOTZ, Marco A. B. F. G.. PROCESSAMENTO DISTRIBUÍDO EXTENSÍVEL UTILIZANDO UM CLUSTER E MAPREDUCE COM APLICAÇÃO NA DETECÇÃO DE NÓDULOS PULMONARES PARA GRANDES VOLUMES DE TOMOGRAFIAS. 2014, 134 f. Trabalho de Conclusão de Curso – Engenharia Industrial Elétrica, ênfase em Eletrônica e Telecomunicações. Universidade Tecnológica Federal do Paraná, Curitiba, 2014.

Este trabalho de conclusão de curso apresenta um grupo de aplicações para a detecção de nódulos pulmonares em tomografias computadorizadas, utilizando conceitos de computação distribuída, *Big Data* e *MapReduce*. Foi utilizado o *cluster* do Departamento Acadêmico de Física da Universidade Tecnológica Federal do Paraná. Este *cluster* possui 96 processadores (48 reais e 48 virtuais), distribuídos em quatro *blades*, sendo que cada *blade* possui 32 GBytes de memória e 12 TBytes de disco rígido. O *framework* adotado para implementar o modelo computacional *MapReduce* foi o *Hadoop 2.3*. Foram utilizadas as tomografias computadorizadas de cinquenta e três pacientes, com o pulmão sendo a região de interesse. As imagens de cada um destes exames foram processadas utilizando versões distribuídas em *MapReduce*. A primeira etapa envolve a segmentação do pulmão através da seleção de uma faixa de valores na escala Hounsfield. Em seguida, realiza-se a detecção de candidatos a nódulo também utilizando a seleção de uma faixa de valores. Finalmente, uma etapa de pós-processamento baseada em características morfológicas e posicionamento tridimensional dos candidatos é efetuada para o refinamento dos resultados da etapa anterior. Com relação à arquitetura, uma solução modular foi adotada, para viabilizar o posterior aprimoramento e facilitar a depuração. A solução distribuída desenvolvida é 146 vezes mais rápida que uma aplicação de máquina única, levando menos de 5 minutos para processar 41.305 imagens. Através da análise de mínimos locais encontraram-se parâmetros que acarretam em até 91% de verdadeiros positivos.

Palavras-Chave: Hadoop, detecção de nódulos pulmonares, Big Data, computação distribuída, processamento de grandes volumes de dados, processamento de imagens médicas, computação de alta performance, MapReduce.

ABSTRACT

LOTZ, Marco A. B. F. G.. EXTENSIBLE DISTRIBUTED PROCESSING USING A CLUSTER AND MAPREDUCE WITH APPLICATIONS ON LUNG NODULES DETECTION FOR LARGE SETS OF TOMOGRAPHIES. 2014, 134 f. Trabalho de Conclusão de Curso – Engenharia Industrial Elétrica, ênfase em Eletrônica e Telecomunicações. Universidade Tecnológica Federal do Paraná, Curitiba, 2014.

This graduation project presents a set of applications for lung nodule detection on computed tomographies, using concepts of distributed computing, Big Data and MapReduce. The Departamento Acadêmico de Física cluster from the Universidade Tecnológica Federal do Paraná was used. This cluster has 96 processors (48 real and 48 virtuals), distributed on four blades. The MapReduce implementation employed was Hadoop 2.3. Computed tomographies of fifty three patients, with the lungs as the region of interest, were used. The images in each of these exams were processed using MapReduce distributed versions. For the first stage, the lung segmentation occurs through the detection of values using the Hounsfield scale. Following this, the nodule candidates detection is performed, also employing a selection over a range of values. Finally, a post-processing stage based on morphological and tridimensional positioning of the candidates is performed, in order to improve the results from the previous step. On the matter of architectural design the solution uses a modular approach, in order to allow further development and aid debugging. The distributed solution is 146 times faster than a single machine application, taking less than 5 minutes to process 41.305 images. Through the local minima analysis, parameters that produce up to 91% true positive rate were found.

Keywords: Hadoop, lung nodules detection, Big Data, distributed computing, large datasets computing, medical images processing, High Performance Computing, MapReduce.

SUMÁRIO

1. INTRODUÇÃO	9
1.1. JUSTIFICATIVA.....	9
1.2. OBJETIVOS.....	10
1.2.1. Objetivo Geral.....	10
1.2.2. Objetivos Específicos.....	11
1.3. DIAGRAMA.....	11
1.4. METODOLOGIA	12
1.5. APRESENTAÇÃO DO DOCUMENTO	13
2. FUNDAMENTAÇÃO TEÓRICA.....	14
2.1. BIG DATA.....	14
2.2. MAPREDUCE.....	15
2.3. HADOOP / MAPREDUCE 1.0.....	23
2.3.1. Hadoop Distributed File System.....	24
2.3.2. JobTracker e TaskTrackers.....	25
2.4. YARN / MAPREDUCE 2.0	26
2.5. CLOUDERA MANAGER	30
2.6. PROCESSAMENTO DIGITAL DE IMAGENS	31
2.6.1. Profundidade de Bits.....	32
2.6.2. Aplicação de Máscaras binárias.....	32
2.6.3. Segmentação por Threshold	33
2.6.4. Operações Morfológicas	33
2.6.5. Rotulagem de Componentes Conectados.....	35
2.7. DICOM.....	36
2.7.1. Escala Hounsfield	37
2.7.2. DICOM Tags.....	38
2.8. PROTOCOLO PXE.....	39
3. DESENVOLVIMENTO	40
3.1. O PROBLEMA	40
3.2. O MÉTODO	40
3.2.1. As imagens pulmonares.....	41
3.2.2. A sequência de processamento completa.....	42
3.2.3. Processamento de Imagens.....	43
3.2.4. Os Metadados da Imagem	51
3.2.5. O Pós-processamento	52
3.3. PROCESSANDO IMAGENS COM MAPREDUCE	57
3.3.1. Mappers.....	57
3.3.2. Sorting	59
3.3.3. Reducer	60
3.4. HARDWARE.....	61

3.4.1. Especificações	61
3.4.2. Configuração	62
3.5. SOFTWARE	66
3.5.1. Script de MATLAB	66
3.5.2. Lung Processor	66
3.5.3. HadoopLung	72
3.5.4. Lung Post-Processor.....	75
3.5.5. Scripts.....	80
4. RESULTADOS E DISCUSSÕES.....	84
4.1. OTIMIZAÇÕES DE PROCESSAMENTO	84
4.1.1. Configurações de <i>Container</i>	84
4.2. COMPARAÇÕES DE PÓS-PROCESSAMENTO	94
4.2.1. Candidatos entre 111 e 129	96
4.2.2. Candidatos entre 111 e 128.....	97
4.2.3. Candidatos entre 111 e 123	97
4.2.4. Candidatos entre 111 e 126	98
4.2.5. Candidatos entre 110 e 125	99
4.3. CONSIDERAÇÕES FINAIS	100
4.4. TRABALHOS FUTUROS	102
4.4.1. No âmbito de processamento de imagem:	102
4.4.2. No âmbito de pós-processamento.....	103
5. GESTÃO	105
5.1. ESCOPO	105
5.2. CRONOGRAMA	105
5.3. CUSTOS.....	107
5.4. RISCOS.....	108
5.5. CONSIDERAÇÕES	109
6. REFERÊNCIAS	111
7. ANEXOS	115
7.1. SCRIPT DE MATLAB	115
7.2. UML LUNG PROCESSOR.....	119
7.3. UML HADOOPLUNG	120
7.4. UML LUNG POST-PROCESSOR.....	121
7.5. ARQUIVO DE CONFIGURAÇÃO DO HADOOP	122
7.6. ROCPROCESS.SH	124
7.7. POSTPROCESS.SH.....	125
7.8. FINDSOLUTIONS.SH	126
7.9. SOLUTIONS.AWK	127
7.10. ITERATESOLUTION.SH.....	129
7.11. SAÍDA DO HADOOP APÓS OTIMIZAÇÃO FINAL.....	131

1. INTRODUÇÃO

No ano de 2002, o câncer pulmonar era a maior fonte de morte entre homens e mulheres nos Estados Unidos da América (ZHAO *et al.*, 2003). Somente naquele ano, estimou-se que haveria em média 169.400 novos diagnósticos deste tipo de câncer e 157.900 mortes associadas a ele dentro do país. Os dados para o Brasil, para o ano de 2003, indicaram ao menos 3682 mortes por câncer de pulmão (INCA, 2014).

Usualmente, este tipo de câncer é utilizado como indicador do impacto do tabagismo relacionado a problemas de saúde uma vez que, em média, 90% dos casos ocorrem com fumantes ou ex-fumantes. Dentro do Brasil, é o tipo de câncer que mais mata homens e o segundo que mais tira a vida de mulheres.

O clímax de mortalidade por este tipo de câncer no Brasil foi em 1994, onde ele se relacionava à morte de 17,54 homens a cada 100 mil. Embora no ano de 2007 a taxa de mortalidade tenha se reduzido para 15,94, este tipo de patologia ainda ocupa o primeiro lugar como a principal causa de câncer entre os homens.

Uma das maneiras de se identificar esta patologia é através de tomografias computadorizadas (CT), onde médicos analisam o espaço interno dos pulmões em busca de nódulos. Os nódulos encontrados são, posteriormente, classificados entre benignos e malignos (ROSENOW, 2014).

1.1. JUSTIFICATIVA

Devido à dificuldade de visualização de nódulos pulmonares, muitos deles passam despercebidos por olhos não altamente treinados. A dificuldade advém do fato de que as estruturas dos nódulos são muito similares às estruturas dos vasos sanguíneos e bronquíolos no interior do pulmão.

Por tal razão, o autor deste documento propõe um método de pré-diagnóstico, utilizando princípios de Big Data para poder realizar o processamento. Com isto, médicos, antes de fornecerem seu diagnóstico final, podem ter uma ferramenta extra de auxílio e referência.

Como a quantidade de imagens e seu tamanho tendem a ser consideravelmente grande (podendo alcançar até alguns gigabytes para somente um exame de tomografia), o autor sugere aplicar conceitos de *Big Data* para permitir o processamento paralelo de diversas imagens ao mesmo tempo.

Este tipo de abordagem no momento do pré-diagnóstico pode agilizar o trabalho dos médicos especialistas da área, reduzindo assim o lapso de tempo entre o exame e a detecção de uma enfermidade, fator que pode ser crucial para a sobrevivência do paciente. Além disso, devido às características do *Big Data*, pode-se processar bancos de dados inteiros com imagens pulmonares em intervalos de tempo consideravelmente reduzidos, independente de quando os exames foram realizados.

Mais do que fornecer um auxílio ao diagnóstico, a solução do autor deve ter um formato modular. Ou seja, posteriormente pesquisadores e alunos podem testar *plugins* para comparar os resultados deste tipo de processamento. Finalmente, deve-se apresentar um simulador de máquina única que seja integralmente compatível com o código paralelo, o que irá facilitar a depuração de erros.

1.2. OBJETIVOS

Os itens que este projeto – e o documento que o acompanha – visam atender podem ser descritos em dois subitens: Objetivos gerais e objetivos específicos. Segue abaixo a descrição de cada um destes.

1.2.1. Objetivo Geral

Este trabalho tem como objetivo desenvolver e avaliar tanto um *framework* quanto algoritmos para o processamento distribuído utilizando um cluster e MapReduce com aplicação na detecção de nódulos pulmonares para grandes volumes de tomografias. É importante que a solução seja extensível (*extensibility*).

1.2.2. Objetivos Específicos

Os objetivos específicos deste projeto são:

- Desenvolver um sistema modular de processamento de imagens médicas distribuído utilizando o modelo computacional MapReduce da Google, mais especificamente implementado sobre a plataforma Apache Hadoop.
- Desenvolver um *framework* modular de máquina única que possa simular o tratamento de uma imagem pela solução distribuída, permitindo que, posteriormente, novos métodos sejam implantados no código e estes sejam somente conectados como *plugins* ao *framework* distribuído.
- Viabilizar a atualização do cluster institucional, tendo como meta seu alinhamento com as últimas tendência na área de computação de alta performance (HPC). Para tal, deve-se configurar e garantir a operabilidade posterior de aplicações como Apache Hadoop, Apache Mahout, Apache Pig e Apache Zookeeper.
- Criar scripts e métodos de rápida instalação e configuração de todo o conteúdo do cluster institucional, diminuindo a necessidade de interação humana com as máquinas e almejando a garantia de que estes processos serão o mais automatizados possíveis.
- Utilizar conceitos de processamento de imagens sobre o banco de dados médicos em formato DICOM, visando extrair as informações de interesse a partir de cada uma das imagens presentes.

1.3. DIAGRAMA

A figura 1 apresenta um diagrama de como é esperado o funcionamento da solução distribuída. Originalmente tem-se um banco de imagens médicas dentro do sistema de arquivos *Hadoop Distributed File System*. Estes dados serão processados pelo Hadoop e gerarão meta-informações sobre as imagens de uma mesma *series* (exame). Entre essas informações estarão dados sobre os candidatos a nódulos e metadados da imagem e da *series*. Uma vez que todas as imagens forem computadas, ter-se-á uma fase de pós-processamento que irá ler as

informações de candidatos a nódulos e julgar se aquele candidatos é possivelmente um nódulo ou não.

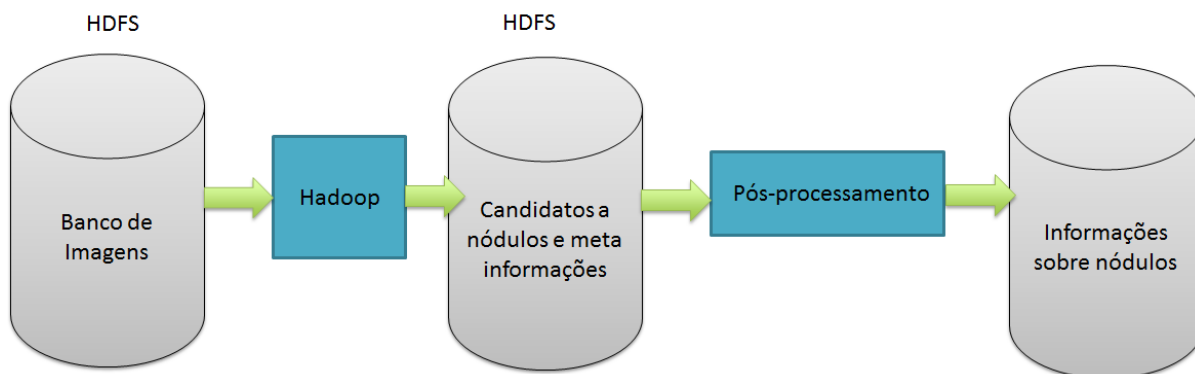


Figura 1 - Diagrama de fluxo de processamento. Fonte: Autoria própria

1.4. METODOLOGIA

O projeto foi separado em seis etapas, descritas abaixo:

- *Etapa 1:* Estudo do problema e de conceitos relacionados ao projeto, bem como configuração do cluster para o início da implementação.
- *Etapa 2:* Implementação de um script em MATLAB para servir de referência e verificar o desempenho do método de processamento de imagem utilizado.
- *Etapa 3:* Desenvolvimento de um *framework* modular para permitir o desenvolvimento simplificado de novos métodos de processamento e compará-los com os resultados do MATLAB.
- *Etapa 4:* Implementação da solução distribuída utilizando os mesmos módulos do *framework*.
- *Etapa 5:* Desenvolvimento de uma aplicação de pós-processamento.
- *Etapa 6:* Redação do trabalho de conclusão de curso, solução de pequenos erros e discussões sobre resultados.

1.5. APRESENTAÇÃO DO DOCUMENTO

Este documento é, em realidade, a descrição pormenorizada do trabalho de conclusão de curso do autor. Para podê-lo compreender por completo, primeiro é necessário adentrar na fundamentação teórica das soluções desenvolvidas. Com isso, o leitor agregará conhecimento sobre conceitos como *Big Data*, MapReduce, Hadoop, Processamento de Imagens Digitais, o padrão DICOM e o protocolo PXE.

Uma vez familiarizado a estes conceitos, o leitor encontra a seção de desenvolvimento. Neste ponto, o programa é detalhado, bem como os métodos para desenvolvê-lo. As soluções do autor são apresentadas, tanto para âmbito de Hardware quanto para o âmbito de software. Finalmente, existem discussões sobre os resultados e algumas considerações.

Em uma nova seção, alguns aspectos de gestão são abordados. Entre estes, pode-se citar: o escopo, o cronograma, os custos, os riscos e considerações sobre gestão. Finalmente, as considerações finais sobre o documento e seus resultados são discutidos e comentados.

2. FUNDAMENTAÇÃO TEÓRICA

Neste capítulo apresentar-se-á ao leitor, de forma introdutória, a fundamentação teórica de conceitos importantes para a correta compreensão de métodos e ferramentas utilizadas na seção seguinte, a de desenvolvimento. Caso faça-se necessário, discussões mais aprofundadas sobre cada um dos itens tratados nesta seção, aplicados ao contexto deste projeto, serão feitas no capítulo seguinte.

2.1. BIG DATA

Big Data não é uma tecnologia única, mas sim uma combinação de tecnologias recentes e mais antigas que permitem realizar previsões em tempo hábil. Assim, pode-se definir *Big Data* como a capacidade de gerenciar uma grande quantidade de dados não necessariamente relacionados diretamente entre si, em uma velocidade tangível e dentro de um espaço de tempo factível de maneira a permitir análise em tempo real (ou *quasi-real*) sobre estes dados, bem como reações aos resultados das análises (HURWITZ *et al.*, 2013).

Por tal razão, pode-se afirmar que *Big Data* apresenta três características principais, também chamadas de os três Vs:

- **Volume:** A quantidade de dados a serem processados.
- **Velocidade:** O quão rápido estes dados serão processados.
- **Variedade:** Os vários tipos de dados que podem estar presente em um conjunto de entrada de informações.

Vale ressaltar que nem sempre as três características estarão presentes. Por exemplo, em alguns casos pode-se ter um conjunto pequeno de entrada de dados, mas, devido a sua complexidade estrutural, a computação destes pode exigir muito processamento.

2.2. MAPREDUCE

O MapReduce foi originalmente descrito em um artigo publicado pela Google (DEAN; GHEMAWAT, 2004) como uma solução para fazer computações em grandes volumes de dados usando uma abordagem paralela e clusters de computadores *commodities*¹. O código da Google era orientado objetos e foi escrito principalmente em C++. Dean e Ghemawat (2004) descrevem qual seria o desempenho de um *framework* paralelo utilizando as funções *Map* e *Reduce* da programação funcional em grandes conjuntos de dados (YAHOO!, 2014).

No documento, existem duas fases principais (chamadas de *Map* e *Reduce*) com uma fase opcional entre elas (chamada de *Combine*). A fase de *Map* é a primeira a ser executada, realizando processamentos em um par chave-valor (a partir daqui referenciado como *key-value*) de entrada e gerando um novo par *key-value* de saída. Vale ressaltar que as estrutura de dados dos pares de entrada e saída não necessariamente são as mesmas. Por exemplo, é possível a entrada ser um par do tipo $\langle \text{integer}, \text{string} \rangle$ e a saída ser do tipo $\langle \text{string}, \text{string} \rangle$. Tudo depende do tipo de processamento que é feito dentro de cada uma das etapas.

Na fase de *Reduce* todos os *values* de uma mesma *key* são agrupados². Então, realiza-se processamento sobre esse conjunto. Como um resultado final, a etapa de *Reduce* também gera pares *key-value*.

Com isso, pode-se dar um exemplo de como funciona um programa composto somente das fases *Map* e *Reduce* com expressão formal abaixo:

$$\begin{aligned} \text{map: } (K1, V1) &\rightarrow \text{list}(K2, V2) \\ \text{reduce: } (K2, \text{list}(V2)) &\rightarrow \text{list}(K3, V3) \end{aligned}$$

¹ Por computador *commodity* entenda-se computadores cujas peças possam ser compradas no varejo, sem necessidade de desenvolvimentos especializados para aquela máquina específica.

² Usualmente, esta união ocorre de maneira transparente ao usuário, em uma etapa chamada *sort*.

Por essa razão, uma das aplicações mais triviais de MapReduce é a implementação de *word counters*³. Para ilustrar como isso seria implementado, tem-se os pseudo-códigos dos Quadros 1 e 2:

Assuma uma função *Map* com a seguinte assinatura:

Map (key, value)

O pseudo-código desta função será mostrado no Quadro 1 – Pseudo código de *Mapper* para *wordcounter*:

```
// key: o nome do arquivo
// value: uma linha do documento

Para cada palavra w em value:
    Emita(w,"1");
```

Quadro 1 – Pseudo código de *Mapper* para *wordcounter*

Assuma agora um *Reducer*, posterior a etapa de *Mapping*, com a seguinte assinatura:

Reduce (key, list<values>)

O seu pseudo-código será mostrado no Quadro 2:

```
// key: uma palavra encontrada na etapa anterior
// list<values>: uma lista de inteiros

Int resultado = 0;
Para cada v em list<values>:
    Resultado += v;
Emita(key, comoString(resultado));
```

Quadro 2 – Pseudo-código de *Reducer* para *wordcounter*

³ Ou seja, contadores de palavras. É um algoritmo básico que conta quantas vezes uma mesma palavra aparece no texto de entrada.

No exemplo acima a função de *Map* lê todas as palavras em um *record*⁴, que neste exemplo é uma linha de texto. Ela emite, então, a palavra como uma *key* e o inteiro 1 como um *value*.

Anterior à aplicação do *Reduce*, todos os *values* de uma mesma *key* serão agrupados. De maneira a exemplificar isto, assuma que a palavra “casa” apareça três vezes em um *record*. A saída da função de *Map* seria o par *key-value* [casa, 1] para cada vez que a palavra “casa” fosse encontrada, totalizando portanto três vezes⁵. O *framework* agrupa os valores de uma mesma chave de maneira transparente ao usuário, antes de apresentar esse valor como a entrada do *Reducer*⁶. Assim, a entrada do *Reducer* seria: [casa, [1, 1, 1]]. Como pode-se ver, o *key* é a palavra casa e o *value* é uma lista de inteiros.

A função de *Reduce* retorna a soma de todos os *values* da *key* “casa”, fornecendo como saída o seguinte par *key-value*: [casa, 3]. A Figura 2 mostra um exemplo de um contador de palavras com vários *Mappers* e *Reducers* rodando simultaneamente:

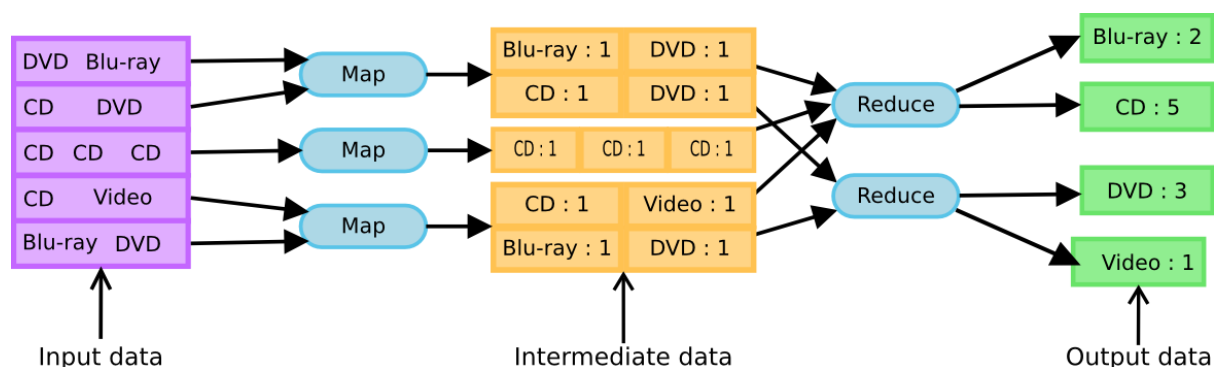


Figura 2 - Fluxograma de Processamento do modelo computacional MapReduce. Fonte: (LENGAUER; APEL; DÖRRE, 2013).

⁴ Gravação, tradução livre. Um *record* é a entrada de um *mapper*. Assim sendo, ele é a unidade mínima que representa um dado que passará pela etapa de *mapping*.

⁵ A saída de um *mapper*, *reducer* ou *combiner* é chamada de uma emissão (*emit*, tradução livre).

⁶ Considere *Mappers* e *Reducers* como objetos nos quais ocorrem os métodos de *Map* e *Reduce*, respectivamente.

Vale ressaltar que todos os *values* de uma mesma *key* são entregues para somente um *Reducer*. Assim, somente um *Reducer* irá processar os *values* da *key* “casa”. Na Figura 2, pode-se verificar que somente um dos *Reducers* trabalha com os *values* da *key* “Blu-ray”. Desta maneira, somente este objeto irá apresentar esta *key* como dado de saída.

Como algumas aplicações clássicas de MapReduce, pode-se listar:

- Contagem de frequência de acessos a um determinado website.
- Grafos de Web-links reversos.
- Grep⁷ distribuído.
- Algoritmos de *Term Vector* em cada host⁸.

Visando evitar tráfego de rede desnecessário – uma vez que isto pode acarretar em gargalo para aplicações em larga escala – o artigo também descreve como o *framework* deve visar manter a localidade dos dados. Isto significa que ele deve sempre tentar⁹ garantir que a máquina rodando os trabalhos de *Mapping* tenha os dados a serem processados em sua memória ou disco local, evitando ao máximo buscá-los na rede. Além disso, como todas as saídas das fases de *Mapping* vão para a rede, um passo opcional mencionado anteriormente, chamado *Combining*, pode ser utilizado. O *Combiner* deve realizar o processamento dos valores da saída dos *Mappers* daquela máquina antes que seus resultados sejam enviados para os *Reducers* através da rede. Como os *Reducers* podem estar em outra máquina, uma redução no tráfego de rede pode reduzir o tempo total processamento.

⁷ Grep é um utilitário de linha de comando para procurar conjuntos de dados em formato de texto pleno que sejam compatíveis com a expressão regular (regex) utilizada.

⁸ É um tipo de algoritmo muito utilizado para detectar palavras-chave muito utilizadas. Cada palavra compõe uma dimensão espacial. Maiores detalhes fogem do escopo deste documento.

⁹ Vale ressaltar que isto nem sempre é possível. Quando isto não é possível, os *mappers* realizam leituras remotas para acessar os dados não locais.

Tendo em vista que o *Combiner* deve ser um passo opcional, o resultado de um processamento utilizando MapReduce não deve ser alterado pela aplicação ou não desta função. Ele deve ser somente acelerado¹⁰.

O *framework* de MapReduce proposto no artigo é constituído de dois *daemons*¹¹ distintos, chamados de *Master* e *Worker*. Os *Workers* são os nós (máquinas) que irão realizar o processamento intensivo dos dados. O *Master*, por sua vez, somente coordena o funcionamento dos *Workers*, garantindo que eles se comportem de maneira adequada. O diagrama da Figura 3 mostra como os *daemons* do *framework* foram originalmente projetados para se comportar.

Originalmente o programa de usuário é enviado (1) para todos os nós que irão participar da computação. Um dos nós torna-se o *Master*. Este então decide, entre os nós que foram disponibilizados como *Workers*, quais serão utilizados como *mappers* e quais como *reducers* (2).

Um grupo de dados de entrada – chamados *input files* – é fornecido aos *Workers* que irão trabalhar na fase de *Map*. As *Splits* locais são lidas (3) por cada um dos *Workers* designados para realizar as funções de *Map*. Uma vez terminado o processo, as saídas intermediárias dos *Mappers* são armazenadas no disco local da máquina hospedeira (4). Há aqui uma barreira de sincronização: quando todos os *Mappers* tiverem encerrado seu processamento, iniciar-se-á a fase de *Reduce*.

Os *Workers* designados para fase de *Reduce* leem (5) estes arquivos intermediários e os processam, escrevendo o resultado nos arquivos de saída – *output files* (6).

¹⁰ Em realidade, os *combiners* possuem alguns parâmetros teóricos para demonstrar se eles podem ou não ser aplicados em cada caso. Estes parâmetros estão fora do escopo introdutório desta seção.

¹¹ Por *daemon*, leia-se um programa de computador que roda como um processo de *background*.

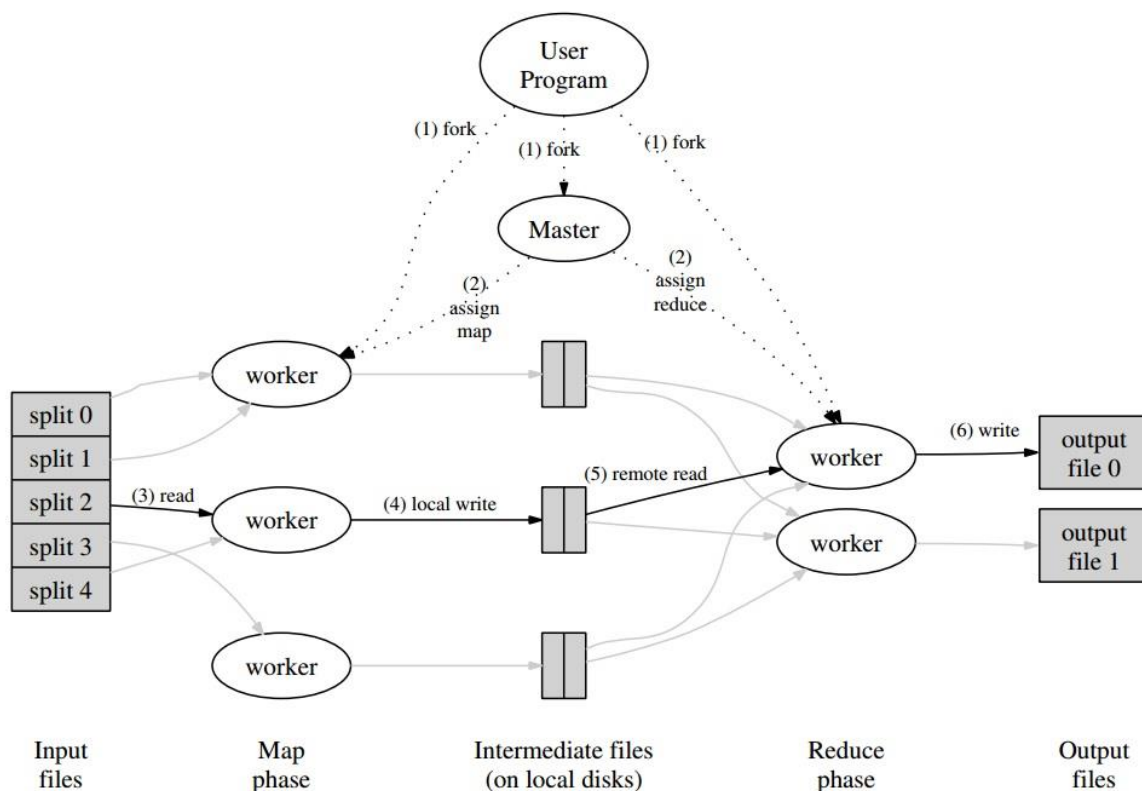


Figura 3 - Demonstração de comportamento dos *daemons Master/Worker* e fluxograma de processamento. Fonte: (DEAN; GHEMAWAT, 2004).

Como a Google somente descreve o modelo computacional no artigo, mantendo todo o código por trás do *framework* de maneira proprietária, vários *frameworks open-source* foram criados para implementar o modelo. Como exemplos pode-se citar Hadoop ou o recurso limitado de MapReduce do MongoDB.

Segundo Dean e Ghemawat (2004), a aplicação, em tempo de execução, deve tomar conta de detalhes de baixo nível de maneira transparente ao usuário comum. Entre estes detalhes, estão: como os dados de entrada serão particionados, qual o escalonamento para a execução dos programas em um grande conjunto de máquinas, tratamento de falhas de máquinas e a comunicação inter-nós. Um usuário experiente pode ajustar esses parâmetros – a exemplo definir como os dados de entrada serão particionados entre os *Workers*.

As grandes inovações do MapReduce estão em alguns conceitos chave associados ao *framework* descrito. Estas são:

- **Tolerância a falhas:** O *framework* deve suportar a eventual falha de computadores graciosamente (*gracefully*). De maneira a implementar isso, o *Master* envia *pings* aos nós periodicamente. Se o *Master* não receber uma resposta de um dado *Worker* em um intervalo definido de tempo, ele considerará que aquele *Worker* falhou. Neste caso, todas as tarefas de *Map* completadas pelo *Worker* falho são descartadas e dadas novamente a um *Worker* disponível. De maneira similar acontece se um *Worker* falho estava no meio do processamento de uma tarefa de *Mapping* ou de *Reducing*. Atente-se que caso o *Worker* já tenha completado as tarefas de *Reduce*, todas as computações nele realizadas terão terminado antes do momento da falha. Assim, os dados processados poderão ser mantidos. Há um ponto principal de falha, que é o *Master*. Caso ele falhe, todo o *job*¹² falhará. Por esta razão, é interessante definir *checkpoints* para o *Master*, visando salvar sua estrutura de dados. Todas as computações que ocorrerem entre o último *checkpoint* e o momento de falha do *Master* são inevitavelmente perdidas.
- **Localidade:** De maneira a evitar o tráfego de rede, o *framework* tenta garantir que todos os dados de entrada estejam disponíveis localmente para as máquinas que vão processá-los na fase de *Map*. Dean e Ghemawat (2004) utilizam o GFS (*Google File System*) com um fator de replicação três¹³ e um tamanho de bloco¹⁴ de 64 MB. O *Master* sabe onde estão os blocos e irá tentar agendar que tarefas de *Map* rodem em uma máquina que contenha estes blocos. Caso não seja possível, o *Master* tenta alocar uma máquina próxima a uma réplica dos dados de entrada – tentando garantir a menor distância entre os dados e o nó, para aumentar a velocidade de transferência da rede entre os dois. Por exemplo, ele

¹² Job é como é chamado um código que foi submetido para ser processado no *framework* do MapReduce.

¹³ Fator de replicação é como é chamado a redundância de dados em um sistema de arquivos distribuído. Um fator de replicação dois indica que o bloco de dados está disponível em ao menos dois nós de armazenamento diferentes.

¹⁴ Arquivos maiores que o tamanho de bloco são separados em vários blocos e estes são replicados pelo sistema de arquivos.

pode tentar agendar um nó de processamento (*Mapper*) no mesmo *rack* que um nó que somente armazena os dados.

- **Granularidade das Tarefas:** Assumindo que cada fase de *Map* é dividida em M pedaços e que cada fase de *Reduce* é dividida em R pedaços, o ideal seria que M e R fossem muito maiores que o número de máquinas trabalhando como *Workers*. A razão disto é que um *Worker* realizando diversos papéis melhora o balanceamento dinâmico de carga. Além disso, ele aumenta a velocidade de recuperação caso um *Worker* pare de funcionar.
- **Tarefas de Backup:** Algumas vezes, um *Worker* desempenhando *Mapper* ou *Reducer* pode ser muito mais lento que os outros *Workers* de um cluster. Isto pode atrasar o tempo de processamento total e fazer o tempo tender ao do tempo de processamento daquele *Worker* mais lento¹⁵. Dean e Ghemawat (2004) descrevem o que é chamado de *backup tasks*, que são agendadas pelo *Master* quando um MapReduce *job* está próximo de ser completado. Elas permitem que o MapReduce *job* seja encerrado quando a tarefa de backup ou a original terminem – o evento que ocorrer antes.
- **Counters¹⁶:** Os *Counters* foram criados para quando o usuário deseja contar o número de vezes que um evento ocorre. Os valores dos *Counters* em cada um dos *Workers* são periodicamente propagados para o *Master* durante o tempo de execução da tarefa. O *Master* então agrega os valores de uma tarefa de *Map* ou *Reduce* bem sucedida e retorna eles ao usuário quando a operação de MapReduce é totalmente terminada.

¹⁵ Um exemplo prático de aplicação da lei de Amdahl, que será comentada em seções futuras deste documento.

¹⁶ Contadores, tradução livre.

2.3. HADOOP / MAPREDUCE 1.0

Como mencionado anteriormente, uma das implementações mais famosas do MapReduce é a Hadoop. Este é um *framework open-source* desenvolvido em Java e mantido pela Apache Foundation. Originalmente, ele foi desenvolvido por Doug Cutting e Mike Cafarella como uma ferramenta para Apache Nutch¹⁷ (LOTZ, 2014). Rapidamente, os autores perceberam que eles não iriam conseguir expandir Nutch para pesquisar em um bilhão de páginas como havia sido previsto. Neste período, a Google havia acabado de lançar seu artigo sobre o GFS, que descrevia uma maneira de tornar o gerenciamento de nós de armazenamento menos oneroso para o usuário final.

Com os recursos descritos no artigo do GFS (GHEMAWAT; GOBIOFF; LEUNG, 2003), os criadores de Hadoop perceberam que poderiam aplicar aqueles conhecimentos para gerar um *file system* capaz de gerenciar os grandes arquivos gerados durante o processo de *web crawling* e indexamento. Por esta razão, um ano depois do lançamento do artigo pela Google, o *Nutch Distributed File System* foi criado.

Em 2004 foi publicado o artigo original do MapReduce (DEAN; GHEMAWAT, 2004) e, em apenas um ano, os dois programadores possuíam uma implementação do *framework* rodando sobre Nutch. Em fevereiro de 2006, eles perceberam que as aplicações de MapReduce iam muito além da busca de sites. Por essa razão eles removeram o componente do projeto Nutch e criaram um totalmente novo, chamado Hadoop. Em seguida, Doug Cutting foi contratado pela Yahoo! para desenvolver ainda mais o Hadoop.

Oficialmente o nome Hadoop¹⁸ significa: High-Availability Distributed Object-Oriented Platform (Plataforma Distribuída de Alta Disponibilidade Orientada Objetos, tradução livre) (LOTZ, 2014). A logomarca do projeto é mostrada na Figura 4.

¹⁷ Nutch era uma *engine open-source* para buscas na web.

¹⁸ Em realidade, o termo Hadoop vem do nome do elefante de pelúcia do filho de Doug Cutting. Isso também explica a logomarca do projeto (MORRIS, 2013).

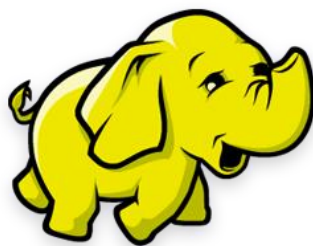


Figura 4 – Logomarca do Hadoop. Fonte: (APACHE FOUNDATION, 2014).

Hadoop segue as premissas de Dean e Ghemawat (2004) e também foca em *hardwares commodities* de cluster. Entre as companhias que oficialmente utilizam Hadoop atualmente, pode-se listar (APACHE FOUNDATION, 2014):

- **Adobe:** Utiliza Hadoop e HBase para o armazenamento de dados estruturas e para realizar processamento interno.
- **Facebook:** Aplica-o para armazenamento de cópias do log interno, fonte de relatórios/análises e aprendizado de máquina.
- **Spotify:** Aproveita-o para agregar dados, gerar conteúdo, análises e relatórios.

Quando o *framework* foi transformado em um projeto independente de Nutch, tornou-se necessário implementar um sistema de arquivos para a plataforma. Assim criou-se o *Hadoop Distributed File System* (HDFS).

2.3.1. Hadoop Distributed File System

O HDFS é composto de três tipos de *daemons*: o *NameNode*, o *SecondaryNameNode* e os *DataNodes*. Eles implementam os elementos descritos no artigo do GFS (GHEMAWAT; GOBIOFF; LEUNG, 2003).

Os *DataNodes* contêm os blocos de dados. Por padrão todos os dados são divididos em blocos de 64 Mbytes e replicados. O fator de replicação padrão é três. Alguns *DataNodes* irão, possivelmente, vir a realizar MapReduce *jobs*.

Como uma tentativa de manter o controle de todos os blocos de dados replicados, o *daemon* chamado *NameNode* armazena todas as informações de Metadados e é um dos pontos únicos de falha do sistema. Todas as informações armazenadas neste *daemon* são relacionadas com quais *DataNodes* contêm quais blocos do sistema de arquivos. Um usuário pode definir *checkpoints*, onde toda a informação contida no *NameNode* é transferida para um *SecondaryNameNode*. Todas as computações realizadas entre uma falha e um *checkpoint* são necessariamente perdidas. O *NameNode* e o *SecondaryNameNode* não armazenam blocos de dados nem realizam computações de MapReduce, uma vez que eles devem resguardar sua performance para manter a integridade do *framework*.

2.3.2. JobTracker e TaskTrackers

A implementação dos *Master* e *Workers* no Hadoop ocorre de maneira similar a descrita em (DEAN; GHEMAWAT, 2004). Normalmente na máquina que executa o *NameNode*, um *daemon* chamado *JobTracker* também opera. O *JobTracker* se comporta de maneira similar ao *Master* descrito por Dean e Ghemawat (2004), coordenando os chamados *TaskTrackers* (que são equivalentes aos *Workers*). O *JobTracker* recebe *heartbeats*¹⁹ de cada um dos *TaskTrackers*, visando garantir que todos os *Mappers* e *Reducers* estão funcionando corretamente. Caso um dos *TaskTrackers* falhe, o *JobTracker* analisa a causa de falha e determina se um novo *TaskTracker* deve ser alocado para a tarefa ou se o *job* deve ser declarado como falho (LOTZ, 2014).

A Figura 5 demonstra a interação do *framework* do Hadoop. Note que o código do cliente (*client*) somente interage com o *JobTracker* (enviando os arquivos de configuração (XML) e o *Java Archive* (JAR) com código dos *mappers* e *reducers*), desconhecendo diretamente os *TaskTrackers* que estão rodando em outro nível.

¹⁹ *Heartbeats*, batimentos cardíacos, é uma técnica de *pooling* muito utilizada para garantir a duas máquinas que uma delas está funcionando.

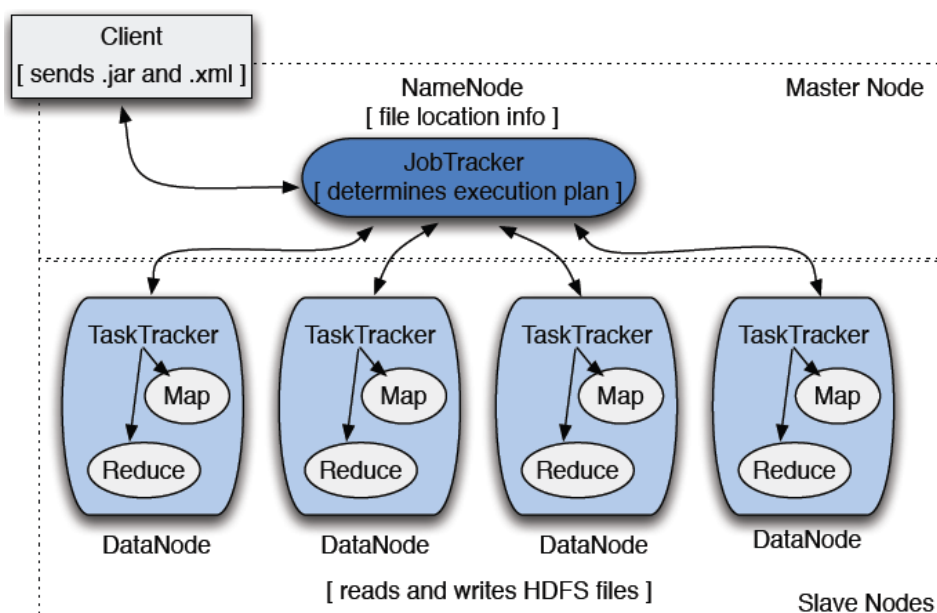


Figura 5 – Hadoop MapReduce 1.0 daemons. Fonte: (GURUZON, 2013).

2.4. YARN / MAPREDUCE 2.0

Depois do lançamento da versão 0.20.203 do Hadoop, todo o *framework* foi reestruturado. Em realidade, quase um *framework* completamente novo foi criado. O nome deste novo *framework* é MapReduce 2.0 (MRv2) ou YARN. Por conseguinte, o *framework* antigo do Hadoop passou a ser chamado de MapReduce 1.0 (MRv1).

A sigla YARN provém de “Yet Another Resources Negotiator”²⁰. A modificação do *framework* teve como foco principal tornar o componente de MapReduce menos dependente do Hadoop em si. Além disso, elas melhoravam as possibilidades de utilizar interfaces de programação além de MapReduce sobre ele e incrementava a escalabilidade do *framework* (WHITE, 2012).

Originalmente, esperava-se que YARN estivesse em produção próximo do término de 2012. No entanto, o projeto só passou da fase alfa para a fase beta no dia 25 de Agosto de 2013 (com a versão 2.1.0-beta). No dia 7 de Outubro de 2013 a

²⁰ Ainda Outro Negociador de Recursos, tradução livre. Algumas fontes o referenciam pela sigla recursiva “YARN Application Resource Negotiator”, ou seja YARN Negociador de Recursos de Aplicação.

primeira versão estável de YARN foi lançada com o Hadoop 2.2.0. Embora as versões alfa e beta fossem amplamente distribuídas ao menos um ano antes da versão estável ser lançada, alguns *vendors* – como a Cloudera – preveniam seus usuários contra possíveis instabilidades deste *framework* (LOTZ, 2013).

A principal razão para a criação do YARN veio de um limiar de escalabilidade que acontecia para o MapReduce 1.0. Para clusters de grande tamanho²¹, Hadoop não era uma solução de grande confiabilidade (MURTHY, 2011).

As principais diferenças entre o MapReduce 1.0 e o 2.0 é que o *JobTracker* da versão original foi separado em dois *daemons* diferentes: O *ResourceManager* e o *ApplicationMaster*. Além disso, o *TaskTracker* foi transformado no chamado *NodeManager*. Uma das principais vantagens disso é que ele remove o ponto único de falha que existia no *JobTracker* e o coloca no *ResourceManager*. Com isso, separa-se a administração da arquitetura do MapReduce em si.

A Figura 6 apresenta um diagrama da arquitetura lógica do Hadoop 2.0. Os parágrafos a seguir descrevem esta arquitetura em detalhes.

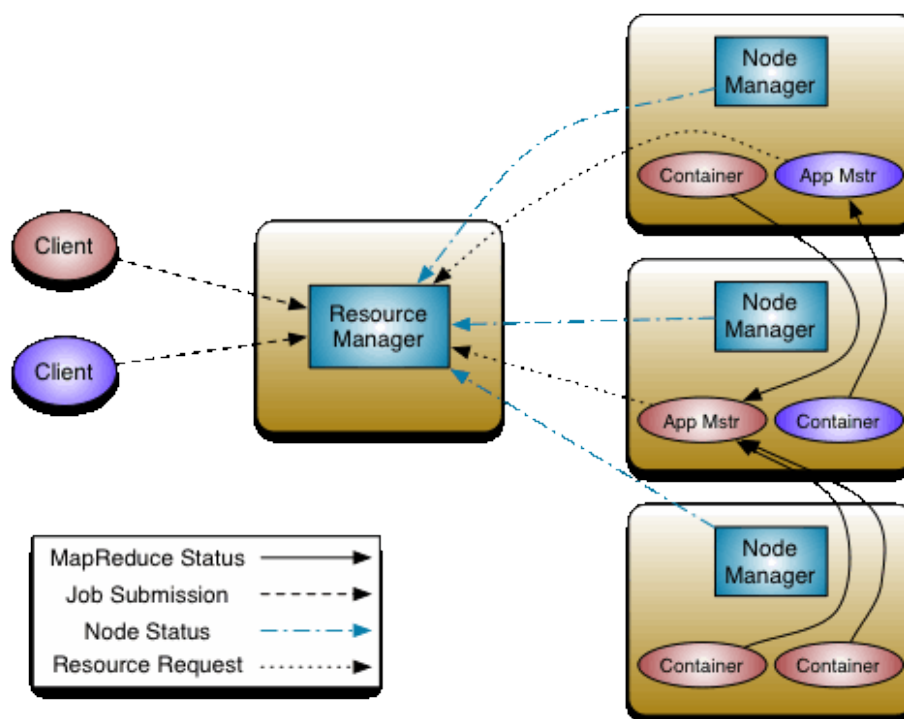


Figura 6 – Hadoop MapReduce 2.0 daemons. Fonte: (APACHE FOUNDATION, 2014)

²¹ Por grande tamanho leia-se com mais de 4 mil nós.

O *ResourceManager* serve para monitorar o status do cluster, indicando quais recursos estão disponíveis em quais nós. Isso somente é possível através do conceito de *container* (que será explicado posteriormente). Pode-se dizer que o *ResourceManager* arbitra a utilização de recursos entre as várias aplicações no sistema. Estas aplicações não se limitam a somente MapReduce.

O conceito de *container* – também chamado de *ResourceContainer* – incorpora elementos como memória, CPU, disco e rede. Um mesmo nó – e portanto um mesmo *NodeManager* – pode ter mais de um *container*. Ele serve como uma abstração de uma unidade independente onde uma máquina virtual Java será executada.

O antigo *TaskTracker* virou o chamado *NodeManager*. Este e o *ResourceManager* compõem a parte de computação de dados do *framework*. Ele é um agente do *framework* individual por máquina e toma conta dos *containers* monitorando o uso de recursos pelos *containers* ali presentes – como por exemplo memória, disco, CPU e rede²². Os resultados dos recursos monitorados são reportados para o *Scheduler* pertencente ao *ResourceManager*. Com isso é possível fazer com que outros *frameworks* de *Big Data* da Apache utilizem o *NodeManager* para alocar recursos, evitando contato com o *TaskTracker* que era específico para o MapReduce.

O *ResourceManager* é composto por dois componentes: O *Scheduler* e o *ApplicationManager*. O primeiro aloca os recursos para as várias aplicações e leva em consideração a capacidade e limitações de fila de execução. Como o nome próprio diz, o *Scheduler* age como um escalonador, de maneira que ele não monitora ou verifica o status de uma aplicação. Além disso, ele não fornece garantias sobre o reinício de tarefas que falharam devido a erro de aplicação ou hardware.

O *ApplicationManager*, por sua vez, é responsável por aceitar os *jobs* que são submetidos, negociar a execução do primeiro *container* – que será o

²² Até o presente momento, existe somente a implementação do gerenciamento de memória. Mas o gerenciamento dos outros recursos está sendo elaborado para as próximas versões.

ApplicationMaster específico daquela aplicação – e prover um serviço de reinício do *ApplicationMaster* caso o *container* onde ele esteja sendo executado falhe.

O *ApplicationMaster*, por sua vez, gerencia a execução paralela de um *job*. Na arquitetura YARN, isso é feito separadamente para cada *job* individual. Este isolamento permite com que o *ApplicationMaster* seja uma biblioteca específica do *framework* que roda sobre o YARN - o que viabiliza a utilização de vários *frameworks* diferentes sobre o YARN, como mostra o diagrama da Figura 7:

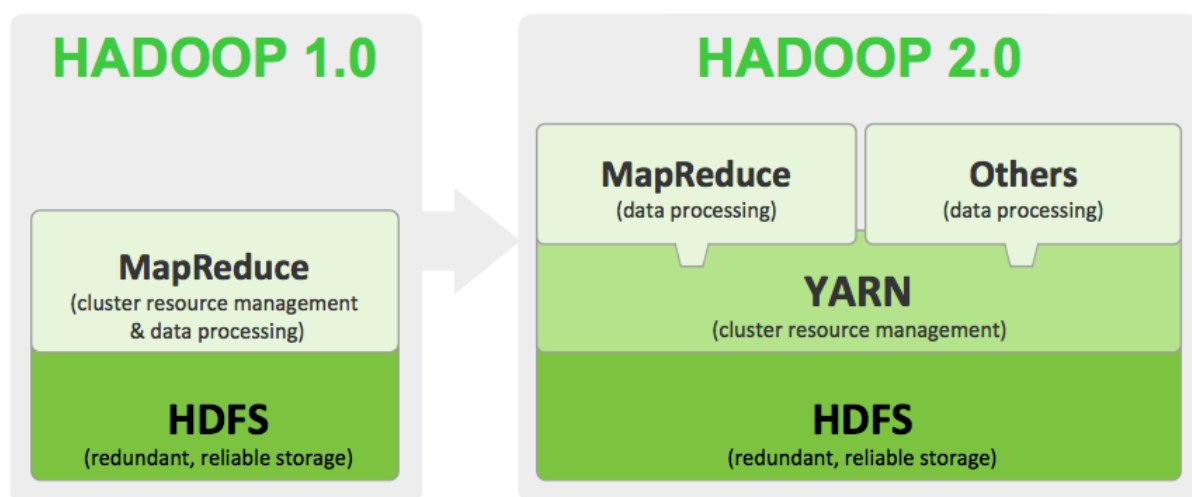


Figura 7 – Arquitetura do Hadoop 1.0 e Hadoop 2.0. Fonte: (HORTONWORKS,2014b).

Conforme os diagramas da Figura 7, no Hadoop 1.0 (MapReduce 1.0) o MapReduce controlava diretamente o HDFS. Já o Hadoop 2.0 (MapReduce 2.0) utiliza o YARN como uma camada de abstração para gerenciamento de recursos do cluster, permitindo assim que outros *frameworks* se conectem sem ter que interagir com o MapReduce. Em outras palavras, o MapReduce passa a ser um *plugin*.

Voltando aos *daemons* ilustrados na Figura 6, observa-se que o *ApplicationMaster* realiza a negociação de recursos disponíveis com o *ResourceManager* e trabalha em conjunto com o *NodeManager* para executar e monitorar as tarefas.

Entre algumas das características da arquitetura YARN tem-se que ela é retro compatível com a versão 1.0, de tal maneira que aplicações já escritas para o *framework* antigo não necessitam ser recompiladas.

2.5. CLOUDERA MANAGER

Cloudera é uma companhia de com sede em Palo Alto, nos Estados Unidos, e conta atualmente com aproximadamente 600 funcionários. Sua principal atividade é prover softwares baseados em Apache Hadoop, bem como suporte, serviços e treinamento (CLOUDERA, 2014).

A companhia possui uma distribuição *open-source* de Apache Hadoop chamada de *Cloudera Distribution Including Apache Hadoop* (CDH). Esta distribuição tem como meta principal fazer implantação em nível empresarial de softwares de *Big Data*. Segundo a companhia, ao menos 50% da produção de seus engenheiros é como *committers* de projetos *open-source* da Apache Foundation (como Apache HBase, Apache Avro, Apache Hive, etc). Além disso, ela é patrocinadora da Apache Software Foundation.

Um dos seus principais produtos é o chamado Cloudera Manager. Este é um software que inclui um GUI²³ para instalação em algumas distribuições de Linux²⁴. Uma vez instalado, o programa oferece uma web UI²⁵ onde o usuário pode selecionar quais computadores deseja incluir no cluster, quais aplicativos cada um dos computadores deve possuir e quais papéis eles devem desempenhar dentro destes aplicativos.

A última versão disponível durante a redação deste documento é chamada de CDH5, que inclui os seguintes aplicativos de *Big Data*:

- Yarn (MapReduce 2.0)
- Apache HBase
- Apache HDFS
- Apache Hive
- Apache HUE

²³ Graphic User Interface, do inglês: Interface Gráfica do Usuário.

²⁴ Algumas versões específicas dos seguintes sistemas operacionais: Red Hat, CentOS, SUSE Linux Enterprise, Debian e Ubuntu.

²⁵ Web User Interface: Interface de usuário web, tradução livre.

- Cloudera Impala
- Apache Oozie
- Apache Spark
- Apache Zookeeper

A descrição do que é cada um desses aplicativos foge do escopo deste documento, uma vez que somente será utilizado o pacote de YARN (MapReduce 2.0) e Apache HDFS.

Os aplicativos que o usuário decide instalar são baixados de maneira transparente pela aplicação e instalados. Posteriormente, o usuário pode configurar cada aplicação através da interface web do Cloudera Manager, sem a necessidade de modificar diretamente os arquivos XML utilizados normalmente. Além disso, o Cloudera Manager oferece diversos gráficos customizáveis para mostrar o desempenho e uso de recursos do cluster. A figura Figura 8 apresenta a tela principal do Cloudera Manager sendo executado no cluster institucional:

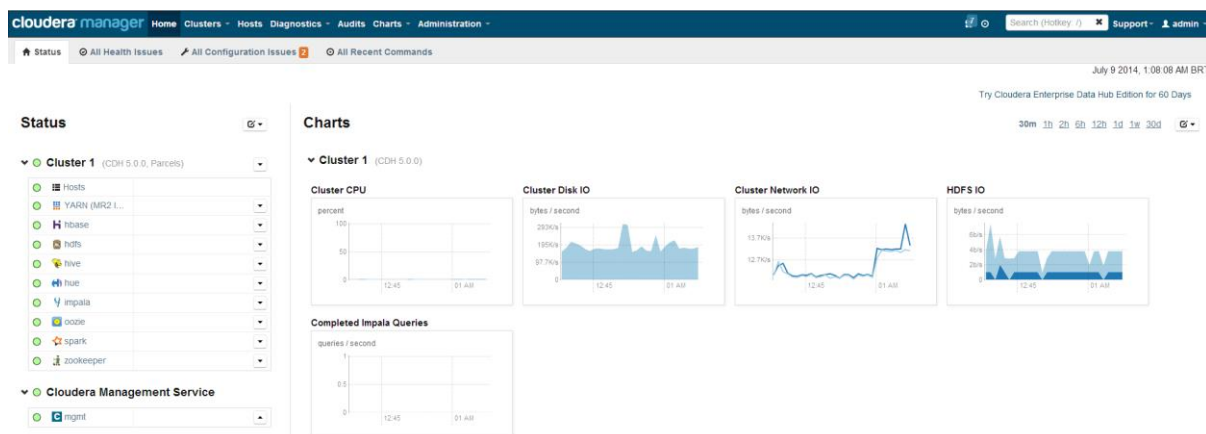


Figura 8 – Cloudera Manager web UI. Fonte: Autoria própria.

2.6. PROCESSAMENTO DIGITAL DE IMAGENS

Para realizar a detecção de nódulos pulmonares, as imagens tiveram que ser processadas digitalmente. Alguns conceitos de processamento digital de imagens devem ser compreendidos para permitir ao leitor uma fácil compreensão de como o processo de tratamento transcorreu.

2.6.1. Profundidade de Bits

A profundidade de bits (*bit depth*) se refere à cor armazenada em um pixel. Assim, quanto maior a profundidade de bits, mais cores podem ser representadas nele. Uma imagem com profundidade de um 1 bit, por exemplo, pode ter somente duas cores (TECH-EASE, 2011). Uma imagem com profundidade de oito bits, por sua vez, oferece 256 possibilidades de cores para um mesmo pixel. O leitor pode notar que, ao se incrementar a profundidade de bits de uma imagem, seu tamanho total também aumenta, uma vez que o tamanho de uma imagem pura (matriz imagem) – ou seja, sem metadados, cabeçalhos ou compressão – é dado pela Equação 1:

$$tamanho = profundidade\ de\ bits \times número\ de\ pixels\ presentes\ na\ imagem \quad (1)$$

2.6.2. Aplicação de Máscaras binárias

Em alguns momentos, somente algumas regiões de interesse (ROI) da imagem desejam ser extraídas da imagem original. Para tal, cria-se a chamada máscara binária²⁶.

Para se extrair as regiões de interesse da imagem se realiza a operação de AND lógico pixel a pixel entre a imagem original e a máscara. Os pixels que possuem todos os bits iguais a 1 na máscara são transferidos para a nova imagem, enquanto os que possuem valor zero são ignorados.

²⁶ Vale ressaltar que, embora o nome máscara binária leve o leitor a crer que sua profundidade de bits é 1, ela pode ter uma profundidade maior desde que somente dois extremos da escala sejam representados nessa máscara.

2.6.3. Segmentação por Threshold²⁷

A segmentação por *threshold* gera uma máscara binária levando em conta os valores dos pixels da imagem original. Ela segue a seguinte função matemática aplicada pixel-a-pixel na imagem original. Ela é realizada de acordo com a função mostrada a seguir aplicada:

$$\begin{cases} 0 & \text{se } pixel < Threshold \\ 1 & \text{se } pixel \geq Threshold \end{cases}$$

Com isto, pode-se assim criar uma máscara binária com o valor de pixel 1, para o caso de do valor de pixel da imagem original seja maior ou igual que o *threshold* ou 0 caso contrário.

2.6.4. Operações Morfológicas

São transformações que resultam na alteração da forma da forma (*shape*) de objetos binários na imagem. As operações morfológicas aplicam um operador sobre a imagem original, chamado de elemento estruturante, que pode possuir um formato arbitrário, dependente da aplicação. O ponto de referência do elemento estruturante é chamado de *hot spot* (BURGER; BURGE, 2010).

As principais operações morfológicas são descritas a seguir, sendo B uma imagem binária e S um elemento estruturante binário.

Dilatação ($B \oplus S$): Cada vez que o *hot spot* de S encontra um pixel '1' de B, S é replicado na imagem de saída. Como exemplo, a Figura 11 apresenta o resultado de uma operação de dilatação para uma imagem sintética simples, com o objetivo de facilitar a visualização. O elemento estruturante, representado na Figura 10, tem seu

²⁷ Do inglês, limiar. Tradução livre.

hot spot no pixel superior esquerdo (demarcado com um X). Os pixels de cor verde possuem valor 1, enquanto o de cor branca possui valor zero.

1	0	0	0	0	1
0	0	1	1	0	0
0	0	1	0	0	0
0	1	1	1	1	0
0	1	1	1	0	0
0	0	0	0	0	0

Figura 9 – Imagem de entrada. Fonte: Autoria Própria

X	

Figura 10 – Elemento estruturante. Fonte: Autoria Própria.

1	0	0	0	0	1
1	1	1	1	0	1
0	0	1	1	1	0
0	1	1	1	1	0
0	1	1	1	1	1
0	1	1	1	1	0

Figura 11 – Imagem após dilatação. Fonte: Autoria Própria.

Erosão ($B \ominus S$): O pixel '1' de B correspondente ao hot spot de S só é mantido na imagem de saída se todos os pixels '1' de S possuem pixel '1' correspondente em B. A YY apresenta o resultado de uma operação de erosão para o mesmo elemento estruturante (Figura 10) e a mesma imagem sintética simples (Figura 9) anteriores.

0	0	0	0	0	0
0	0	0	0	0	0
0	0	1	0	0	0
0	1	1	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Figura 12 – Imagem após a erosão. Fonte: Autoria Própria.

Fechamento ($B \bullet S$): É a dilatação seguida da erosão, isto é, aplicar a dilatação na imagem B usando S e depois aplicar a erosão na imagem dilatada usando S. A operação de fechamento atua nos objetos da imagem principalmente das seguintes formas: suaviza contornos, funde quebras estreitas, elimina pequenos buracos, preenche espaços de contornos (GONZALEZ; WOODS; EDDINS, 2008).

$$B \bullet S = (B \oplus S) \ominus S$$

Abertura ($B \circ S$): É a erosão seguida da dilatação, isto é, aplicar a erosão na imagem B usando S e depois aplicar a dilatação na imagem erodida usando S. A operação de abertura atua nos objetos da imagem principalmente das seguintes formas: suaviza contornos, quebra ligações estreitas, elimina protrusões finas (GONZALEZ; WOODS; EDDINS, 2008).

$$B \circ S = (B \ominus S) \oplus S$$

2.6.5. Rotulagem de Componentes Conectados

O objetivo da rotulação (*labeling*) é discriminar (nomear) cada objeto da imagem binária. Na convenção adotada neste trabalho, objetos possuem pixel '1' e fundo pixel '0'. Um objeto é um conjunto de pixels conectados pela vizinhança-4 ou vizinhança-8. A saída da rotulação é uma imagem na qual cada objeto é identificado por um valor de pixel único, como ilustrado no exemplo da Figura 13.

Na maioria das aplicações práticas, a rotulação geralmente ocorre após uma segmentação (por *Threshold*, por exemplo, como mencionado anteriormente) e faz parte do processo de análise da imagem. As características a serem analisadas de cada objeto dependem da aplicação. Dentre as mais diretas estão a contagem dos objetos na imagem e a medida da área em pixels, mas há uma grande quantidade de métodos disponíveis na literatura para a extração de outros atributos dos objetos binários. Por exemplo: centróide, perímetro, excentricidade e muitos outros (SHAPIRO; STOCKMAN, 2001; GONZALEZ; WOODS; EDDINS, 2008; MARQUES, 2011).

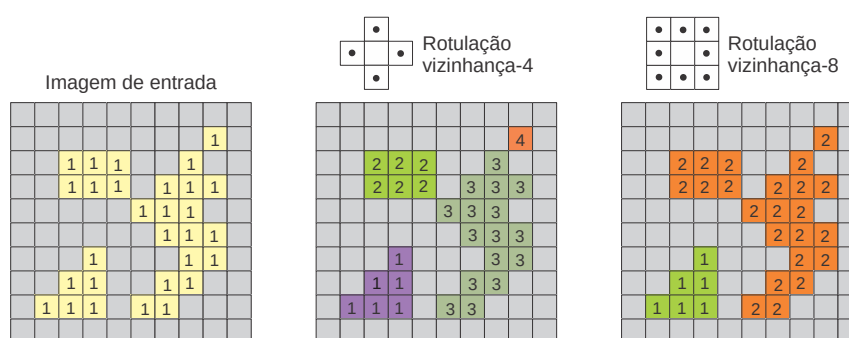


Figura 13 – Rotulagem levando em conta a vizinhança do pixel. Fonte: Autoria Própria.

2.7. DICOM

Sigla para *Digital Imaging and Communication in Medicine*²⁸, consiste em um padrão para o manuseio, impressão, armazenamento e transmissão de informação de exames médicos por imagem. O padrão inclui um *standard* de comunicação de rede e um de arquivo.

Todas as imagens médicas utilizadas neste trabalho estavam no padrão DICOM. Junto a cada uma delas, existe um conjunto de metadados da imagem, identificados por sua TagID. Somente algumas TagID são utilizadas neste documento.

²⁸ Imageamento Digital e Comunicação na Medicina, tradução livre.

Existem quatro níveis de hierarquia no padrão DICOM: *Patient*, *Study*, *Series* e *Image*. Cada paciente pode ter um ou mais *Studies*. Cada *Study*, por sua vez, pode ter um ou mais *Series*. Finalmente, uma *Series* pode ter uma ou mais *Images*. A Figura 14 apresenta um diagrama de hierarquia do formato DICOM:

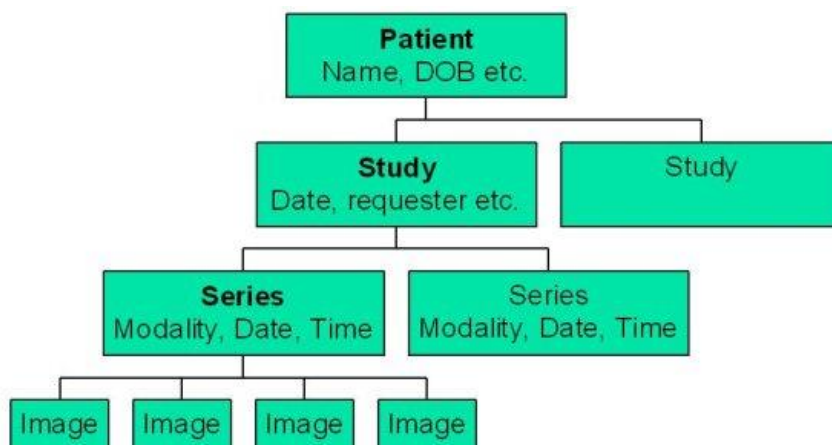


Figura 14 – Hierarquias no padrão DICOM. Fonte: (MEDICAL CONNECTIONS, 2014).

As imagens DICOM utilizadas possuem uma profundidade de pixel de 16 bits. No entanto, os valores ali presentes não representam diretamente uma escala de cinza, mas sim uma referência com a escala Hounsfield (FANNING, 2006). Vale destacar que uma *series* é o equivalente a um exame médico.

2.7.1. Escala Hounsfield

Em tomografia, é a escala quantitativa para descrever a intensidade de radiação X. Também é comumente chamada de valor de CT. A Tabela 1 apresenta valores médicos de Hounsfield (HU) para algumas substâncias e tecidos.

Tabela 1– Níveis de referência para a escala de unidades Hounsfield (HU). Fonte: (MURPHY, 2005)

Substância	HU
Ar	-1000
Pulmão	-500
Gordura	-100
Rins	+30
Sangue	De +30 até +45
Osso	De +700 até + 3000

2.7.2. DICOM Tags

Neste trabalho, foram analisadas algumas TagIDs armazenadas no cabeçalho de uma imagem no padrão DICOM. O TagID, seu respectivo nome e seu significado estão descritos na Tabela 2:

Tabela 2– DICOM tags analisados neste documento. Fonte: Autoria Própria.

Tag ID	Nome	Significado
0008,0016	SOPClassUID	Identifica o tipo de serviço para o qual a imagem é utilizada.
0008,0018	SOPInstanceUID	Identificador único da imagem
0008,0020	StudyDate	Dia, mês e ano do <i>Study</i>
0008,0021	SeriesDate	Dia, mês e ano da <i>Series</i>
0008,0023	ImageDate	Dia, mês e ano da <i>Image</i>
0008,0030	StudyTime	Horário do <i>Study</i>
0008,0031	SeriesTime	Horário da <i>Series</i>
0008,0033	ImageTime	Horário da <i>Image</i>
0008,0060	Modality	Modalidade
0008,0070	Manufacturer	Fabricante do equipamento
0008,0080	InstitutionName	Nome da instituição onde o equipamento médico se situa.
0008,0081	InstitutionAddress	Endereço da instituição
0008,1010	StationName	Nome do equipamento
0008,1030	StudyDescription	Descrição do exame
0010,0010	PatientsName	Nome do paciente
0010,0020	PatientsID	Identificar do paciente
0018,0015	BodyPartExamined	Parte do corpo analisada
0018,0050	SliceThickness	Espessura da fatia
0018,0060	KVP	kiloVoltagem de pico
0018,0088	SpaceBetweenSlices	Espaçamento entre fatias
0020,000E	SeriesInstanceUID	Identificador único da <i>series</i> que uma imagem pertence.
0020,0012	AquisitionNumber	Identificador que varia para cada tipo de exame
0020,0013	ImageNumber	Número da imagem na série
0020,0032	ImagePosition	Posicionamento daquela imagem
0020,0037	ImageOrientation	Orientação daquela imagem
0028,0030	PixelSpacing	Espaço representado por um pixel

2.8. PROTOCOLO PXE

Algumas máquinas possuem o recurso de *boot* pela rede (Lan Boot) diretamente configurável na BIOS. Esse recurso, em geral, é sinal de que o hardware da máquina suporta o protocolo PXE. A sigla significa Preboot eXecution Environment²⁹ e é um padrão criado pela Intel. De maneira simplificada, ele permite que máquinas iniciem e sejam configuradas pela rede, sem necessidade de intervenção humana (INTEL CORPORATION, 1999).

O protocolo em si é a combinação dos protocolos DHCP³⁰ e TFTP³¹. O primeiro é utilizado para localizar os servidores de *boot* apropriados. O outro serve para permitir o *download* do programa de *bootstrap*³² inicial.

O DHCP é, em realidade, um protocolo de rede que atua sobre a camada IP, de maneira a distribuir dinamicamente parâmetros de rede. Entre esses parâmetros, pode-se citar o IP de interfaces e outros serviços. Com isso, não há necessidade de um administrador de redes de configurar o IP de cada máquina manualmente.

O TFTP, por sua vez, é utilizado para permitir a transferência de arquivos de *boot* e configuração entre máquinas de uma rede local. Vale ressaltar que, devido a sua simplicidade de implementação, este não utiliza qualquer tipo de autenticação.

Um programa de *bootstrap* nada mais é do que o primeiro programa, de uma sequência de outros programas de *boot*, que irão participar do projeto de inicialização da máquina.

²⁹ Ambiente de execução pré-boot, tradução livre.

³⁰ Dynamic Host Configuration Protocol. Protocolo dinâmico de configuração de hospedeiro - tradução livre.

³¹ Trivial File Transfer Protocol. Protocolo Trivial de Transferência de Arquivos, tradução Livre.

³² Mais detalhes sobre o bootstrap serão comentados em seções posteriores.

3. DESENVOLVIMENTO

3.1. O PROBLEMA

Como mencionado anteriormente, o câncer de pulmão é atualmente a maior causa de mortes masculinas no Brasil (INCA, 2014). Um de seus principais sintomas é a presença de nódulos na região pulmonar. A detecção de nódulos pulmonares no entanto não é um processo trivial, necessitando muitas vezes de considerável esforço analítico por parte dos médicos, associado a grande concentração. Isto porque os exames específicos para este tipo de diagnóstico podem conter facilmente mais de 1000 imagens de tomografia. Além disso, os vasos sanguíneos que irrigam a região pulmonar muitas vezes atuam como ruído na busca de um nódulo (ZHAO *et al.*, 2003).

Tendo em vista a possibilidade de erros deste processo, bem como artigos atuais neste ramo de estudo e tecnologias aplicadas para o processamento de grandes quantidades de dados, o autor deste documento propõe e analisa um conjunto de aplicações modulares – ou seja, que posteriormente podem ser melhoradas através de *plugins* por outros autores – para fazer este tipo de detecção anterior ao laudo médico. Assim torna-se possível que as imagens do exame sejam processadas logo após sua coleta, fornecendo informações aos médicos que posteriormente validarão o diagnóstico, de maneira a agilizar este processo. Além disto, os resultados de todos os exames antigos armazenados em um banco de dados podem ser analisados, permitindo uma possível reavaliação de diagnósticos pretéritos não conformes com o encontrado pela solução.

3.2. O MÉTODO

A proposta do autor consiste na elaboração de quatro soluções de software para processamento dos dados de tomografia. Estas soluções serão descritas posteriormente na seção de software. Nesta seção é apresentada a visão geral do método.

3.2.1. As imagens pulmonares

As imagens utilizadas neste documento foram fornecidas por uma empresa curitibana da área de diagnóstico por imagem, chamada de DAPI (DAPI, 2014). Nesta parceria, foram fornecidos os exames de 43 pacientes portadores de nódulo pulmonares, com ao menos um nódulo por paciente com uma das dimensões superior a 8 milímetros. Os diagnósticos destes pacientes foram feitos por médicos do DAPI e foram utilizados como critério final de avaliação de desempenho do método. Também forneceu-se os exames de 10 pacientes sem qualquer tipo de nódulo pulmonar.

Todas as imagens estavam no padrão DICOM e com resolução de 512x512 ou 1024x1024 pixels. Cada paciente – identificados pelo seu PatientName e PatientID de sua Tag DICOM – possui um conjunto de exames realizados no passado. Cada um desses exames é chamado de *series* segundo a nomenclatura DICOM e está associado aos seguintes campos:

- SeriesData
- SeriesTime
- SeriesInstanceUID

A descrição da função de cada campo está presente na seção anterior. As imagens fornecidas pelo DAPI têm profundidade de 16 bits. Devido à grande quantidade de equipamentos lá disponíveis, as *series* foram geradas em dispositivos de diversos fabricantes. Estes dispositivos estavam, no entanto, calibrados seguindo o mesmo referencial para seus valores de penetração da radiação (escala Hounsfield).

Quando as imagens foram originalmente providas ao autor deste documento, existiam outros exames de áreas que não a de interesse. Assim, o volume original de dados DICOM para os 53 pacientes era de aproximadamente 115,65 Gigabytes (157.792 imagens), o que resulta em aproximadamente 2,18 Giga bytes por paciente (2977 imagens/paciente).

Foi necessário então uma filtragem manual por parte do autor. Conservou-se todas as imagens que eram de tomografias de pulmão, ou seja, as que tinham este órgão como região de interesse. Exames de mediastino e outras regiões (mesmo sendo na mesma localidade corpórea) foram removidos. Como resultado final, filtrou-

se a entrada original para 26,62 Gigabytes (totalizando 41305 imagens), o que resulta em aproximadamente 500 Megabytes por paciente (779 imagens/paciente em média).

Vale ressaltar que não houve qualquer tipo de filtragem para definir a quantidade mínima de imagens em uma mesma *series* de região relevante, ou mesmo verificar se todas as series eram de fato contínuas na sequência de imagens. Isto porque cada paciente possuía, em geral, ao menos três exames. E, em ao menos um destes, assume-se que o médico conseguiu diagnosticar a patologia. Com isto, espera-se que a solução consiga, em ao menos um deles, desempenhar o mesmo papel na detecção de nódulos.

3.2.2. A sequência de processamento completa

As principais etapas do processo de detecção de nódulos são ilustradas no diagrama da Figura 15. Conforme o diagrama, as imagens originalmente estão armazenadas no formato DICOM em um *file system*³³. Um aplicativo lê e processa estas imagens, gerando o que o autor nomeou de *Series Data*. Este termo denota que imagem em si é descartada e somente alguns dados, relevantes para descrever as principais características daquela imagem, são mantidos. Com isto pode-se reduzir o volume de dados, permitindo que somente dados importantes sejam posteriormente analisados. O resultado desta fase de processamento é novamente armazenado no *file system*.

Em um segundo momento, existe a etapa de pós-processamento. Neste instante, as descrições das imagens são levadas em conta e analisadas de maneira interdependente por *series*. Com isso, pode-se levar em conta a disposição tridimensional dos candidatos, bem como a morfologia destes para classificá-los em nódulos ou não. Nesta etapa pode-se ainda traçar dados de verdadeiros positivos/falsos positivos e realizar-se aferições de desempenho da solução.

³³ Por hora, pormenores funcionais do *file system* serão abstraídos para facilitar a compreensão do leitor. Mais detalhes serão dados nas seções posteriores.

Vale ressaltar que tentou-se abstrair o máximo possível a atividade de cada fase no método de processamento utilizado pelo autor. Através dessa característica, usuários posteriores poderão definir outros critérios de processamento para a detecção de candidatos e resolução de nódulos e não nódulos.

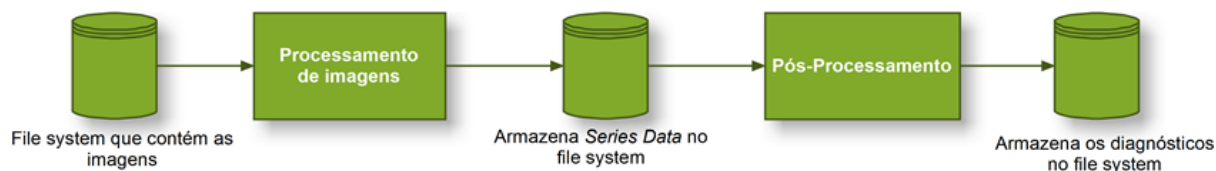


Figura 15 – Sequência das operações principais para a detecção de nódulos. Fonte: Autoria própria.

3.2.3. Processamento de Imagens

O diagrama da Figura 16 ilustra os três componentes fundamentais da seção de processamento de imagem. Cada um destes componentes será abordado individualmente na sequência.

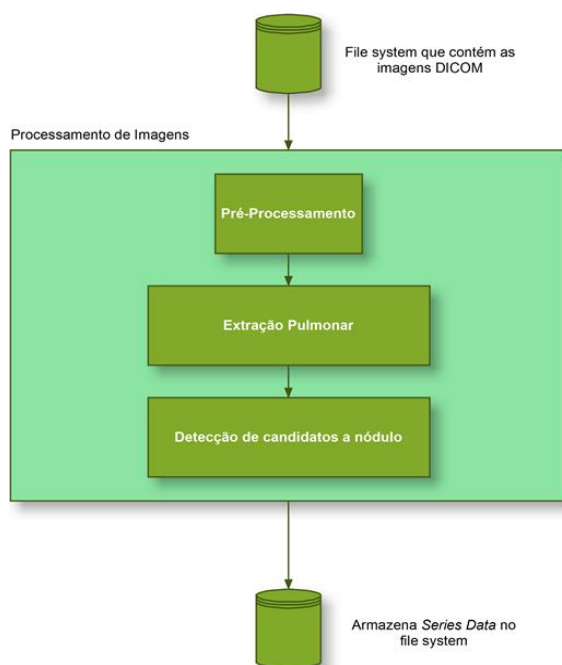


Figura 16 – Etapas do processamento de imagens. Fonte: Autoria própria.

3.2.3.1. Pré-Processamento

Neste momento, um usuário pode realizar algum tipo de pré-processamento com a imagem. Pode-se, por exemplo, reduzir a profundidade de bits de uma imagem DICOM original de 16 para 8 bits. Na implementação descrita neste documento, o autor não realizou qualquer tipo de pré-processamento nas imagens, mantendo assim o valor original das imagens DICOM.

3.2.3.2. Extração dos pulmões

Este bloco tem como finalidade extrair os pulmões do paciente dos restos da imagem e pode ser redefinido futuramente. O funcionamento deste módulo, na implementação do autor, pode ser visualizado no fluxograma da Figura 17:



Figura 17 – Fluxograma de Extração dos Pulmões. Fonte: Autoria própria.

No início da seção de extração de pulmões se tem a imagem de entrada (a possibilidade de ser uma imagem DICOM ou não depende das opções de pré-processamento utilizadas). Na saída ter-se-á somente os pulmões. A Figura 18 mostra uma imagem original, ou seja, sem qualquer tipo de processamento, como presente nos dados de entrada:



Figura 18 – Imagem DICOM original. Fonte: Autoria própria.

Na etapa de geração de máscara, cria-se uma máscara binária com as mesmas dimensões da imagem original e com o valor de todos seus pixels como falso. Conforme áreas de interesse vão sendo encontradas, os pixels equivalentes na máscara serão configurados para verdadeiro, permitindo assim a extração do pulmão.

Durante a segmentação de escala de Hounsfield, todos os pixels cujos valores estão abaixo dos limiares de -375 da escala Hounsfield são selecionados. Este valor foi utilizado tendo-se como referência (ZHAO *et al.*, 2003). O valor está acima do limiar mínimo para a área do pulmão, conforme referenciado na seção de fundamentação teórica.

As áreas extraídas neste processo podem ser visualizadas na imagem da Figura 19. Verifica-se que junto com os valores abaixo de -375 estão tanto os pulmões quanto o ar no exterior do paciente.

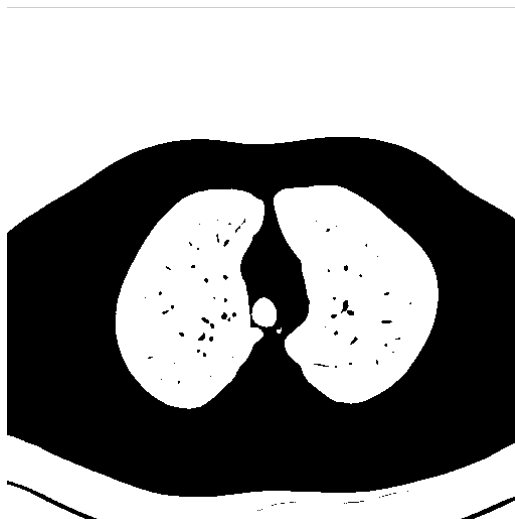


Figura 19- Imagem após segmentação por nível na escala Hounsfield. Fonte: Autoria própria.

Muitas vezes os componentes presentes na máscara apresentam pequenos buracos ou fissuras, devido ao fato de que a distribuição de HU não é homogênea. Assim, utiliza-se fechamento para garantir a inclusão de regiões que antes estavam contidas em pequenos buracos dentro da representação pulmonar nas máscaras. Além disso, pequenas possíveis fissuras também são removidas no processo. Para realizar esta transformação morfológica usa-se, como elemento estruturante, um disco com seis pixels de diâmetro. Os resultados podem ser visualizados na imagem da Figura 20.

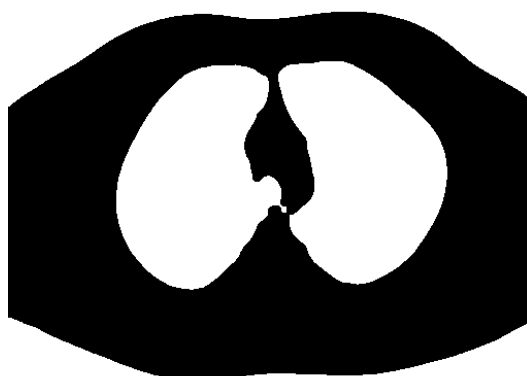


Figura 20 – Imagem após o processo de fechamento. Fonte: Autoria própria.

Finalmente, aplica-se a técnica de análise de componentes conectados. Caso um componente conectado possua um pixel em comum com as bordas da imagem, ele será eliminado da máscara. Neste trabalho, este processo também é

chamado remoção de fundo. Qualquer buraco nos componentes conectados finais são removidos, para garantir que espaços não preenchidos no fechamento original sejam levados em consideração. A Figura 21 mostra o resultado deste processo:

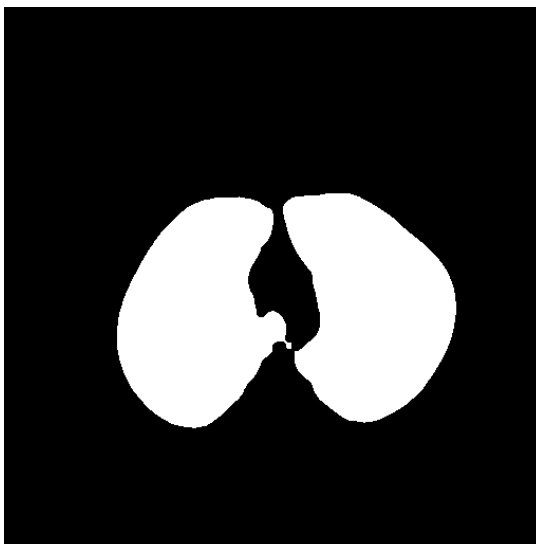


Figura 21 – Imagem com o fundo removido. Fonte: Autoria própria.

Uma vez que tem-se a máscara dos pulmões, basta realizar o equivalente à um AND lógico com a imagem de entrada para retirar somente os pulmões desta. O resultado final do bloco de extração dos pulmões pode ser visualizado na imagem da Figura 22:

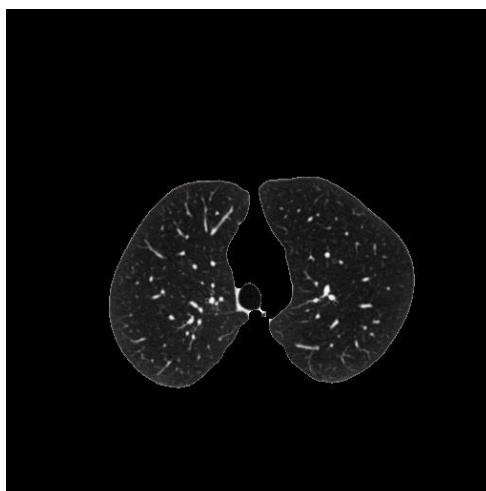


Figura 22 - Pulmões extraídos. Fonte: Autoria própria.

3.2.3.3. Detecção de candidatos a nódulos

Tendo como princípio uma imagem contendo somente os pulmões, muitos critérios podem ser utilizados para encontrar os candidatos a nódulo. Por exemplo, (KIM *et al.*, 2003) descreve a aplicação de um algoritmo chamado *Local Density Maximum*³⁴. Na aplicação do autor, utilizou-se uma seleção de escala de cinza, bem como um limitador de tamanho mínimo do candidato em pixels (ZHAO *et al.*, 2003). Um fluxograma indicando a sequência de processamentos deste bloco pode ser visualizado na Figura 23:



Figura 23 – Fluxograma de detecção de candidatos a nódulo. Fonte: Autoria própria.

A parte de configuração de parâmetros serve para que o módulo de detecção de candidatos possa ler possíveis variáveis definidas em um arquivo de configuração fornecido pelo usuário. Embora os módulos anteriores também possam acesso a este arquivo de configuração, nenhuma configuração foi realizada. Por tal razão, este bloco foi omitido em diagramas pretéritos.

³⁴ Densidade local máxima, tradução livre.

Após isso, assim como na parte de extração pulmonar, gera-se uma máscara binária com as mesmas dimensões da imagem de entrada neste bloco. Na sequência, ocorre um seletor de *threshold*. Somente pixels com valores entre os mínimos e máximos definidos na etapa anterior são selecionados. Por padrão, os valores de cinza mínimo e máximo padrões são 110 e 120, respectivamente (ZHAO *et al.*, 2003).

Como os valores dos pixels da imagem utilizada aqui estão em realidade em valores Hounsfield, o autor realiza um mapeamento de valores levando em conta esses valores de cinza e um fundo de escala. Este mapeamento é dado pela formula:

$$Threshold \text{ mínimo em HU} = \text{valor mínimo de cinza} * \frac{Limite superior DICOM}{Valor \text{ máximo para 8 bits}}$$

O limite superior DICOM é o maior valor representativo neste tipo de imagem DICOM. Ele tende a ser próximo do valor 3000 e representa um fundo de escala. Empiricamente, utilizou-se o valor de 2235. O valor máximo para 8 bits é $2^8 - 1$, ou seja, 255. O *threshold* máximo é calculado de maneira similar com o valor máximo de cinza.

Assim, tem-se os seguintes valores de *threshold* mínimos e máximos linearizados a partir de valores de cinza 110 e 120:

$$Threshold \text{ mínimo em HU} = 964,11$$

$$Threshold \text{ máximo em HU} = 1051,76$$

A máscara tem seus valores atualizados para tornar brancos somente os pixels que estão entre estes valores. O resultado pode ser visualizado na imagem da Figura 24:



Figura 24 - Candidatos dentro do nível de cinza. Fonte: Autoria própria.

Finalmente são encontrados os componentes conectados da imagem e filtra-se estes por tamanho. Todos os componentes com menos de cinco pixels (valor obtido empiricamente) nos eixos x e y são descartados. Com isso, obtêm-se os candidatos a nódulo de uma dada imagem, conforme ilustra a Figura 25:

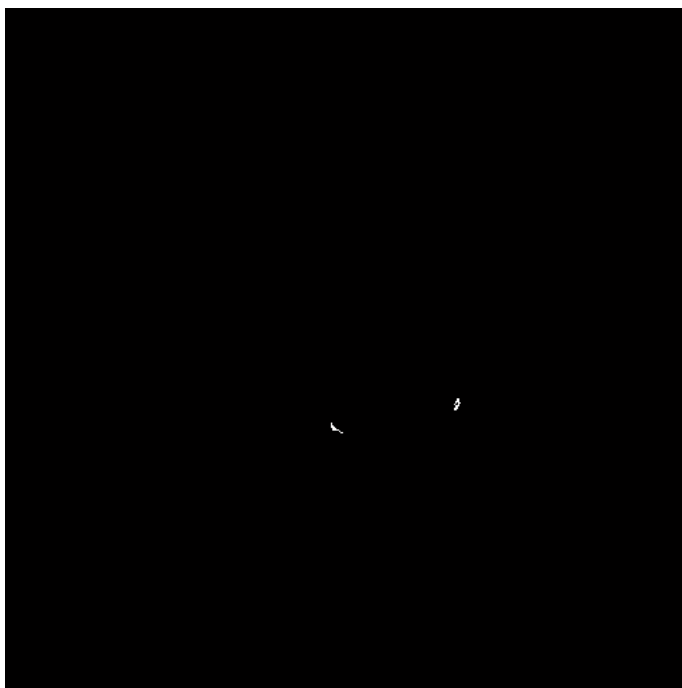


Figura 25- Candidatos maiores que o tamanho mínimo. Fonte: Autoria própria.

O resultado deste bloco de processamento são os metadados da imagem, que contêm dados da imagem original e informações a respeito dos candidatos a nódulo. A imagem original em si é descartada. Com isso, reduz-se o volume de dados externados por esta fase.

3.2.4. Os Metadados da Imagem

O formato definido pelo autor para descrever somente uma imagem contém algumas informações da imagem e outras dos candidatos a nódulo nela presentes. As informações referentes à imagem são os seguintes Tags DICOM:

- SOPInstanceUID
- ImageDate
- ImageTime
- AquisitionNumber
- ImageNumber
- ImagePosition
- ImageOrientation

Além disso, cada imagem tem uma lista de candidatos a nódulo. Cada um destes candidatos contém as seguintes informações:

- Coordenadas x e y do canto esquerdo superior de uma *bounding box* contendo o candidato.
- Largura do objeto
- Altura do objeto
- Circularidade do candidato, que é definida pela seguinte fórmula:

$$circularidade = \frac{perímetro^2}{Área}$$

- Área, em pixels.
- Perímetro, em pixels.

3.2.5. O Pós-processamento

Por último, realiza-se a etapa de pós-processamento para diferenciar estruturas nodulares de vasos sanguíneos e outros componentes. Esta etapa lê todos os metadados das imagens de uma mesma *series* e procura por objetos tridimensionais que apresentem características nodulares. A Figura 26 apresenta o fluxograma desta etapa. A solução de pós-processamento extrai todas as *series* presentes no arquivo de entrada e as processa uma a uma. Por fim, ela apresenta os resultados ao usuário. O processamento tridimensional das imagens é descrito pelo fluxograma da Figura 27.

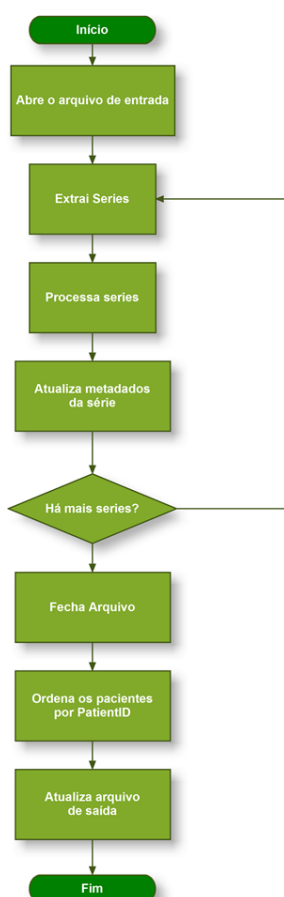


Figura 26 - Fluxograma de pós-processamento. Fonte: Autoria própria.

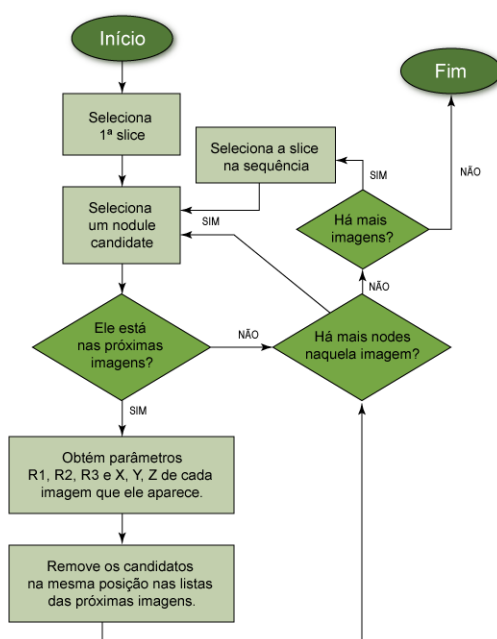


Figura 27 - Fluxograma para comparação tri-dimensional de candidatos. Fonte: Autoria própria.

A primeira imagem (*slice*) de um exame é selecionada. Verifica-se então, candidato por candidato desta imagem, se estes aparecem na imagem adjacente, uma a uma, até que este não seja mais detectado em imagens subsequentes. Dois candidatos em duas imagem adjacentes são considerados como componentes do mesmo nódulos se possuírem os seus centroides³⁵ em uma distância máxima menor que um dado valor definido nas configurações.

A distância entre dois centroides é definida segundo a seguinte fórmula:

$$d_x = |X_{\text{candidato A}} - X_{\text{candidato B}}| < \text{parâmetro limitador}$$

Por padrão, o parâmetro limitador é dois (pixels). A fórmula equivalente é aplicada para as coordenadas Y dos candidatos. Para eles serem considerados um nódulo, ambas as distância necessitam estar dentro deste parâmetro limitador.

³⁵ A função Centróide(N) de um Nódulo N retorna as coordenadas do topo esquerdo da imagem, mais a metade de sua largura para a coordenada das abcissas e a metade da sua altura para a coordenada das ordenadas.

Uma vez que todas as imagens de uma mesma *series* sejam avaliados buscando objetos tridimensionais e os classificando como nódulos, é necessário eliminar prováveis falsos positivos. Assim, cada um dos nódulos até então encontrado é filtrado pela combinação dos seguintes parâmetros:

$$\begin{aligned} & (R1 < 0.3) \text{ OR} \\ & (R2 < 0.2) \text{ OR} \\ & (R3 > 1.5) \text{ OR} \\ & \{[(dx) * pixel_size_x < 2.0] \text{ AND } [(dy) * pixel_size_y < 2.0]\} \text{ OR} \\ & \{[(dx) * pixel_size_x > 15] \text{ AND } [(dy) * pixel_size_y > 15]\} \end{aligned}$$

Caso a expressão acima seja verdadeira, o nódulo é considerado como um objeto não compacto (ZHAO *et al.*, 2003) (esférico) ou não pertencente ao limites de interesse deste documento. Vale ressaltar que estes parâmetros são configuráveis e o desempenho destes na detecção será posteriormente analisado.

O parâmetro R1 representa a razão do volume do objeto com o volume de *bounding box* modificada contendo o objeto. Ele é calculado pela seguinte fórmula:

$$R1 = \frac{\text{numero de voxels do objeto}}{\max(dx, dy) * \max(dx, dy) * dz}$$

dx , dy e dz representam o comprimento máximo das projeções do objeto (em número de pixels) nos eixos x, y e z respectivamente. A função max retorna o valor máximo entre os seus dois argumentos. O número de voxels de objeto é calculado levando em conta que a distância entre cada uma das imagem em z corresponde a um pixel no eixo z, indiferentemente da resolução utilizada neste eixo. Assim, multiplica-se a área do candidato a nódulo, por um pixel em z para cada imagem que este ocupa.

A modificação entre o volume comum de uma *bounding box*, calculado pela trivialmente por $dx * dy * dz$, com o utilizado no denominador do parâmetro permite que R1 diferencie um objeto de formato circular (nódulo) de um objeto em formato de bastão (vaso sanguíneo). Os valores baixos de R1 tendem a corresponder a objetos em forma de bastão. A Figura 28 ilustra a *bounding box* modificada e o cálculo de R1 para um objeto bidimensional.

Como pode-se visualizar na Figura 28, os objetos (A) e (B) possuem formas bastante distintas. O primeiro possui um formato quase circular, enquanto o segundo se assemelha a uma elipse no plano xy. Caso o denominador de R1 fosse a *bounding box* normal, os dois valores seriam muito próximo. Isto porque R1 seria dado pela fórmula:

$$R1_{\text{Bounding box normal}} = \frac{\text{numero de voxels do objeto}}{dx * dy * dz}$$

A razão dessa semelhança é que a soma dos valores dx e dy em ambos os casos seriam praticamente o mesmo³⁶. No entanto, caso seja utilizada a fórmula originalmente proposta para o *bounding box* modificado, os dois objetos apresentam valores distintos de R1 (Com R1 baixo para objetos do tipo bastão).

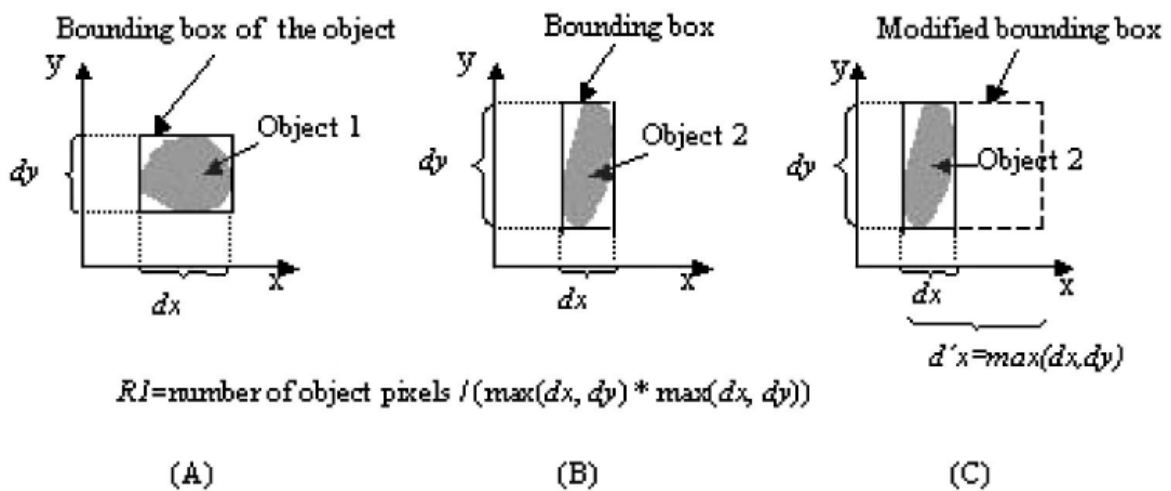


Figura 28 – Comparação de bounding boxes. Fonte: (ZHAO et al., 2003)

Pode-se definir o parâmetro R2 como a razão do máximo valor do comprimento do objeto no eixo z (em unidades de distância) pelo máximo valor do objeto nos eixos x ou y (o que for maior, em unidades de distância).

³⁶ O leitor deve se atentar que o maior valor de dy na imagem (B) é compensado pelo menor comprimento em dx.

$$R2 = \frac{dz * pixel_size_z}{\max(dx * pixel_size_x, dy * pixel_size_y)}$$

Com este tipo de abordagem, possíveis característica anisotrópicas das tomografias computadorizadas são levadas em consideração através de multiplicação do comprimento – em pixels – pela valor correspondente do pixel em unidades de medida (*pixel_size*). Vale ressaltar que o autor, neste parâmetro, tomou a liberdade de realizar uma pequena correção por preciosismo. Originalmente, descrevia-se o parâmetro como:

$$R2 = \frac{dz * pixel_size_z}{\max(dx, dy) * pixel_size_x}$$

Para o caso hipotético de um exame em que o tamanho de pixel nos eixos x e y não sejam idênticos, este parâmetro seria falho. Por isso, cada uma das componentes foi multiplicada pelo seu tamanho de pixel dentro da função de máximo.

Finalmente, define-se o parâmetro R3 como a razão da dimensão máxima do objeto no eixo x ou y (o que for maior) pela dimensão máxima deste no eixo x ou y (o que for menor). Ou seja, o parâmetro é dado pela fórmula:

$$R3 = \frac{\max(dx, dy)}{\min(dx, dy)}$$

A função $\min(dx, dy)$ retorna o valor mínimo entre seus dois argumentos. A sequência de condições para que o nódulo não seja apontado como um falso positivo garante que ele possui formato compacto (circular/esférico) em duas dimensões (R2 e R3) e em três (parâmetro R1). Os outros parâmetros definem seus tamanhos mínimos e máximos nos eixos x e y em unidades de distância. Usualmente, a unidade tende a ser milímetros.

Por fim, todos os exames de um mesmo paciente são computados e, caso um de seus exames seja positivo para a presença de nódulos, este paciente é considerado como portador de nódulos pulmonares.

3.3. PROCESSANDO IMAGENS COM MAPREDUCE

A parte que mais demanda poder computacional é a de processamento digital de imagens. Conforme será comparado posteriormente, a parte de pós-processamento é praticamente desprezível temporalmente quando comparada com a parte do processamento de imagem.

Para reduzir este tempo, o autor propôs a utilização do modelo computacional MapReduce no processamento de imagens e agrupamento de metadados. Assim, todas as imagens no formato DICOM ficariam armazenadas no HDFS. Aos *Mappers* seriam designadas as imagens locais, ou seja, aquelas que estão armazenadas no disco rígido da máquina que irá processá-las - de maneira a evitar sobrecarga de I/O na rede.

Pode-se representar o processamento de imagens utilizando MapReduce através do diagrama em blocos da Figura 29:

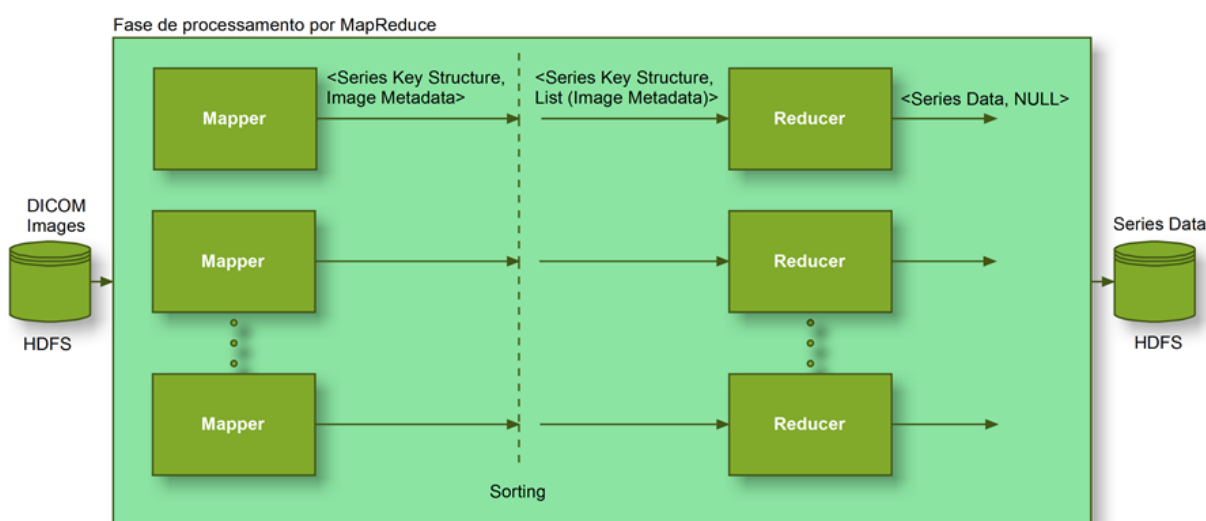


Figura 29- Diagrama da etapa de MapReduce. Fonte: Autoria própria.

3.3.1. Mappers

Cada um dos *Mappers* lerá as imagens do HDFS. Para manter o formato *key-value* do paradigma computacional, utilizar-se-á um valor nulo de *key* e a imagem DICOM como o *value*. Cada um dos *records* lidos pelos *Mappers* conterá todos os bytes de uma imagem DICOM original. Caberá ao *Mapper* reinterpretar os

bytes presentes em um *record* para o formato de imagem e então processa-los de acordo. O processamento de cada uma das imagens é individual e nenhum estado é mantido entre a computação de duas imagens distintas. Assim, pode-se representar a sequência de processamento do *Mapper* conforme o diagrama da Figura 30:

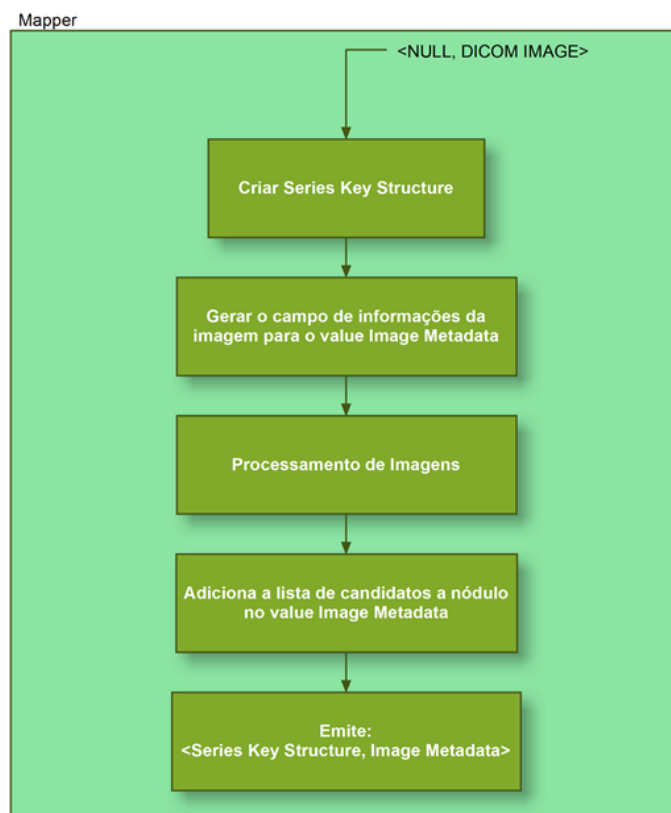


Figura 30 – Sequência de processamento do Mapper. Fonte: Autoria própria.

Pode-se visualizar que a primeira coisa que o *Mapper* fará é gerar a *key* utilizada. Ela é do tipo *Series Key Structure*, pois contém informações sobre uma série de imagens. As informações contidas na *key* são:

- seriesInstanceUID
- studyDate
- seriesDate
- studyTime
- seriesTime
- modality
- manufacture
- institutionName
- institutionAddress
- stationName
- studyDescription
- patientsName
- patientsID
- bodyPartExamined
- sliceThickness
- kVP

- spaceBetweenSlices
- pixelSpacing

Estes dados são únicos para todas as imagens de uma mesma *series* e, portanto, serão utilizados como *key*. O campo *seriesInstanceUID* será utilizado para comparar duas *keys* na verificação de igualdade. O *spaceBetweenSlices*, o *pixelSpacing*, O *patientsID* e o *patientsName* serão utilizados posteriormente na fase de pós-processamento.

Em seguida, o *Mapper* coletará as informações da imagem em si, que serão utilizados no *value* Image Metadata. As metainformações pertencentes ao *value* serão:

- SOPInstanceUID;
- imageDate;
- imageTime;
- acquisitionNumber;
- imageNumber;
- imagePosition;
- imageOrientation;

O *imageNumber*, o *acquisitionNumber*, o *imagePosition* e o *imageOrientation* será utilizado posteriormente no pós-processamento. O campo *imageNumber* será empregado posteriormente para a realização de ordenação pelo *reducer*.

Finalmente, as imagens são processadas como descrito na seção anterior e uma lista com candidatos a nódulo é adicionada ao conteúdo original do *value*. Por fim, o par *key-value* na forma <Series Key Structure, Image Metadata> é emitido pelo *Mapper*.

3.3.2. Sorting

Os pares *key-value* emitidos pelos *Mappers* serão, ao término da fase de *mapping*³⁷, agrupados em listas de *values* que possuem a mesma *key*. Com isso, o autor garante que todas as imagens de uma mesma *series* estarão no mesmo *reducer*. Esta característica permite que a entrada de um *reducer* seja independente

³⁷ Em realidade, a implementação desta solução espera até que 80% dos *Mappers* terminem seu processamento para iniciar a parte de *Reducer*. Esta característica pode ser configurada.

dos outros pares e que estas entradas sejam completas – ou seja, todas as informações relativas a uma mesma *series* estarão necessariamente no mesmo *reducer*.

3.3.3. Reducer

O comportamento do *reducer* pode ser visualizado no diagrama da Figura 31:

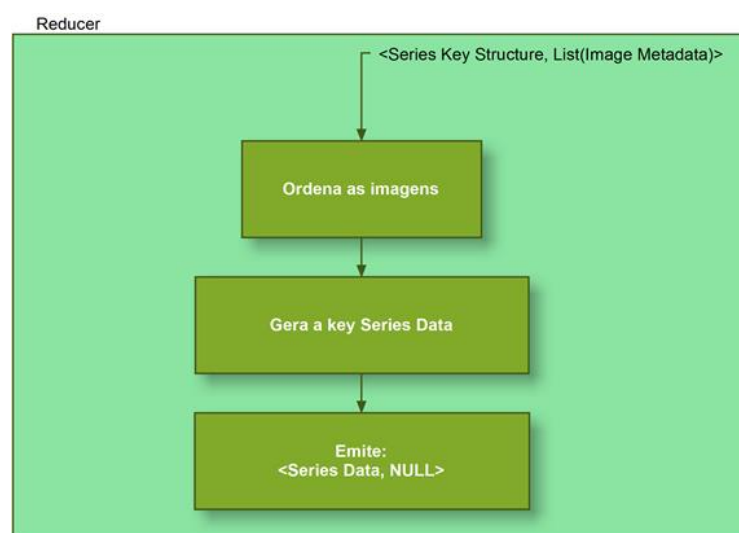


Figura 31 – Sequência de Processamento do Reducer. Fonte: Autoria própria.

O *reducer* primeiro ordenará todas as imagens pertencentes a uma mesma *series* de forma crescente levando em consideração o número daquela imagem dentro da *series*. Isto será feito utilizando-se o campo *imageNumber*, contido no Image Metadata. Posta esta organização, este bloco do modelo computacional gerará um novo tipo de chave chamado de Series Data, que nada mais é do que um invólucro para a *key* original (Series Key Structure) e a sua lista de *values* ordenada (lista de Image Metadata). Será então emitido o par <Series Data, null>. A saída do reducer irá para o HDFS no formato definido pelo usuário de maneira a facilitar o pós-processamento.

3.4. HARDWARE

3.4.1. Especificações

Para o desenvolvimento das soluções descritas neste documento foram utilizados o cluster institucional do departamento de física e um computador pessoal. Seguem as especificações de cada um destes.

Especificações do cluster institucional do Departamento Acadêmico de Física:

- 4 *blades*, cada *blade* contendo:
 - 12 processadores Intel(R) Xeon(R) CPU E5-2620 0 @ 2.00GHz
 - 12 processadores virtuais gerados pela tecnologia HyperThreading.
 - 3x SATA III Seagate Constellation de 4TB (ST4000NM0033) em Raid 0 por software.
 - 32 Gigabytes de memória RAM DDR3 @ 1333 MHz, compartilhada pelos processadores da mesma *blade*.
 - Interface de rede eth0 (*blade*-servidor) atuando em Gigabit, 1000Mb/s.
 - Conexão nós-servidor feita com cabos CAT-6.
- Um servidor, com a seguinte configuração:
 - 12 processadores Intel(R) Xeon(R) CPU E5-2620 0 @ 2.00GHz.
 - 12 processadores virtuais gerados pela tecnologia HyperThreading.
 - 24x SATA III Seagate Constellation de 4TB (ST4000NM0033) em Raid 6 por hardware.
 - Contraladora LSI MegaRAID LSI9271-8I 8-Port 6GB/s 1GB DDR3 PCIe 3.0
 - 32 Gigabytes memória RAM DDR3 @ 1333 MHz.
 - Interface de rede eth0 atuando em Gigabit, 1000Mb/s.
 - Interface de rede eth0 (servidor-*gateway*) atuando em Gigabit, 1000Mb/s.
 - Interface de rede eth1 (servidor-*blades*) atuando em Gigabit, 1000Mb/s.

- Conexão em ambas as interfaces utilizam cabos CAT-6.
- Switch HPN V1910 24 x 10/100/1000 + 4 x SFP - JE006A - HP

As especificações do computador Apple utilizado pelo autor durante a fase de pós-processamento são:

- 1 núcleo Intel Core i3 @ 3,2 Ghz
- 4 Gbytes DDR3 @ 1333 MHz
- 1 Terabyte de disco rígido

3.4.2. Configuração

Os itens a seguir descreverão como o cluster institucional foi configurado de maneira a possibilitar a implantação das plataformas de processamento distribuído e assim viabilizar a detecção de nódulos pulmonares com a abordagem proposta.

3.4.2.1. Configuração inicial

Tendo como meta permitir que no futuro outros usuários e administradores do cluster institucional pudessem configurá-lo rapidamente, o autor implementou a configuração através de *boot* pela rede e do protocolo PXE. Utilizou-se o sistema operacional CentOS 6.5, já almejando a compatibilidade deste com o Cloudera Manager. Os scripts desenvolvidos e os parâmetros de configuração podem ser encontrados em (LOTZ, 2014).

3.4.2.1.a. Definição da Rede Interna do Cluster

O cluster consiste de 4 *blades* e de um servidor. Assim, o servidor foi chamado de *server.cluster* e conectado através de sua interface *eth1* a um *switch*. Este, por sua vez, possuía conexão com as quatro *blades*. O *server* funcionava de *gateway* para rede externa, através de uma interface *eth0*. Assim, os nós podem acessar a internet mas não podem ser acessados externamente.

Para acessar uma dada máquina, o cliente primeiro necessita conectar-se ao *server* e deste seguir para o nó. A Figura 32 representa um diagrama simplificado da camada lógica da rede. Vale ressaltar que, em realidade, tem-se quatro nós. Estes foram abstraídos da representação gráfica para viabilizar a rápida compreensão.

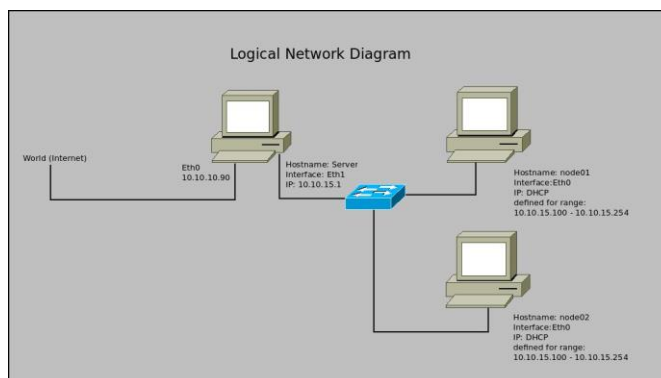


Figura 32- Diagrama da rede montada no cluster. Fonte: Autoria Própria.

3.4.2.1.b. Configuração de DHCP

No servidor do cluster colocou-se um servidor de DHCP para definir o IP das máquinas ali presentes. Ele responde aos pacotes do tipo DHCPDISCOVER lançados pelos nós quando estão sendo iniciados pela opção de *Boot on Lan*. O servidor de DHCP fornece, além das opções de IP disponíveis, o menu de boot pelo protocolo PXE. Este menu é importante quando há mais de uma possibilidade de instalação de sistema operacional na mesma rede local.

3.4.2.1.c. Configuração de TFTP

O TFTP irá disponibilizar o programa de *bootstrap*. Assim, os nós com a BIOS compatível com o protocolo PXE irão fazer download deste software e dar início a sequência de *boot*. Vale ressaltar que o *boot* de um sistema é, em realidade, a chamada de vários programas sucessivos até que o *kernel* do sistema operacional seja finalmente instalado.

3.4.2.1.d. Instalação do servidor Apache

O servidor Apache é utilizado para disponibilizar a ISO completa de instalação do CentOS 6.5. O nó rodando o programa de *bootstrap* acessará a porta 80 da interface eth1 no servidor em busca da imagem de instalação.

3.4.2.1.e. Instalação do Syslinux

Este software é um projeto que contém um conjunto de *bootloaders*. Através dele, é possível realizar o *boot* do sistema operacional (SO) através de múltiplas fontes, entre elas CDs e rede. Para iniciar pela rede, utilizar-se-á o *bootloader* chamado de PXELinux. As configurações deste não entram no escopo deste documento.

3.4.2.1.f. Kickstart File

Por fim configurou-se o arquivo *kickstart*. Este arquivo define as configurações de instalação do sistema operacional da máquina sem qualquer interação do usuário com ela. Assim, no momento de instalação do CentOS, todas as diretivas de configuração serão lidas deste arquivo. Pode-se definir, através dele, *passwords* padrão de *root*, o particionamento do disco rígido e outras configurações.

3.4.2.2. Configuração da plataforma Cloudera Manager

Uma vez que o CentOS 6.5 estava corretamente configurado, instalou-se o Cloudera Manager 6.5 Express. Este software apresenta uma WebUI, permitindo ao usuário com as permissões corretas a configuração do cluster. O manager tem como porta padrão a 7180 e pode ser acessado por qualquer *browser* comum.

O autor configurou o gateway do Departamento Acadêmico de Física para realizar *portforwarding*³⁸ de conexões na porta 7180 para o servidor do cluster. Assim, qualquer usuário autenticado se conectando do Brasil (ou de países cuja faixa de IP estão liberadas para conexão pelo *firewall* do departamento) pode se conectar ao Cloudera Manager. Basta abrir um *browser* e digitar o seguinte endereço:

dafis.ct.utfpr.edu.br:7180

Vale ressaltar que o usuário deve estar previamente cadastrado para conseguir acesso.

Todos os aplicativos descritos na seção do Cloudera Manager da fundamentação teórica foram instalados no cluster, inclusive Apache YARN, HBASE, Hue e Spark. Muitos softwares não utilizados neste documento foram instalados e configurados tendo como meta que usuários posteriores possam munir-se de seus recursos sem se deparar com os pormenores técnicos de configuração.

3.4.2.3. Acesso remoto ao Cluster

Configurou-se o *gateway* do Departamento Acadêmico de Física para fazer *portforwarding* de conexões SSH³⁹ na porta 1090 para o cluster. Assim, serão aceitas as conexões de um usuário previamente autorizado, se estas forem realizadas com a seguinte sintaxe na linha de comando remota:

```
ssh -p 1090 [usuário]@dafis.ct.utfpr.edu.br
```

³⁸ *Port Forwarding* é o nome de uma técnica que permite computadores remotos se conectarem em um computador específico de uma rede local.

³⁹ Secure Shell (SSH), é um protocolo de rede criptografado que permite comunicação segura de dados e login de linha de comando remotos.

3.5. SOFTWARE

Os principais componentes de software desenvolvidos durante este projeto foram: O *script* em MATLAB, o *framework* de máquina única para processamento de imagem (chamado Lung Processor), o aplicativo de Hadoop para processamento de imagem (chamado HadoopLung) e o aplicativo de pós processamento (chamado Lung PostProcessing). Alguns scripts de shell para tratamento de dados de entrada e saída também foram escritos. Os softwares e os scripts serão descritos nessa seção.

3.5.1. Script de MATLAB

O script de MATLAB foi a primeira implementação desenvolvida pelo autor, como uma maneira de verificar a viabilidade da extração pulmonar seguindo os critérios descritos em artigos científicos selecionados. Ele segue exatamente a sequência de processamento descrita previamente neste capítulo e está em anexo a este documento.

Devido a grande quantidade de janelas externadas ao usuário, é possível verificar como cada uma das fases de processamento atua sobre a imagem DICOM. Esta característica permite assim a sintonia fina de parâmetros de maneira ágil, viabilizando rápida comparação de resultados no processamento da imagem. O valor máximo de fundo de escala da imagem DICOM, utilizado no momento da linearização da escala de cinza para detecção dos candidatos a nódulos, provém exatamente de resultados empíricos encontrados nesta solução. O código fonte desta implementação está disponível em (LOTZ, 2014).

3.5.2. Lung Processor

O Lung Processor é um *framework* de máquina única que atua como a intersecção entre o código presente no script de MATLAB e a aplicação de Hadoop. Similarmente ao MATLAB, ele permite a visualização das imagens conforme elas vão sofrendo o tratamento e realiza a mesma sequência de operações para o

processamento. Como a maior parte do código do Hadoop é escrito em Java, esta foi a linguagem adota para o *framework* de máquina única.

3.5.2.1. Principais Características do Lung Processor

Este *framework* possui algumas características muito importantes que foram criteriosamente planejadas e desenvolvidas. Entre elas, pode-se citar:

- **Compatibilidade com a aplicação em Hadoop:** Quando este software foi projetado, era uma das principais prioridades do autor permitir que módulos (no caso da linguagem Java, leia-se *classes* e *packages*) fossem facilmente conectados com o software de Hadoop e depurados. Assim, um módulo de extração de pulmão poderia ser testado e configurado no *framework* de máquina única em um conjunto de imagens antes de ser colocado no HadoopLung. Tendo em vista a complexa depuração de aplicações distribuídas usando Hadoop - que envolve, entre outras ferramentas, a utilização de logs e valores agregados - é desejável que os erros possíveis de ocorrerem durante o processamento de imagem sejam todos depurados em máquina única. O código uma vez pronto deveria ser somente anexado ao Hadoop, sem necessidade de modificação alguma.
- **Configurabilidade:** Muitas vezes um usuário deseja passar vários parâmetros para um objeto de uma dada classe. No Hadoop, utiliza-se um arquivo de configuração em formato XML para realizar este tipo de tarefa. Portanto, incluiu-se um arquivo XML no diretório *conf*⁴⁰ um arquivo de configuração com extensão mencionada anteriormente . Através deste método, qualquer tipo e quantidade de parâmetros podem ser passados pelo usuário. Este recurso foi anexado ao Lung Processor para manter a semelhança ao HadoopLung.

⁴⁰ Diretório presente dentro de um JAR do *framework*.

- **Customização:** Todas as classes que realizam o processamento de imagem - seja a extração ou a detecção de candidatos - derivam de uma classe mãe abstrata. Utilizando o polimorfismo presente em linguagens orientada-objeto, outros autores podem definir, testar e comparar classes por eles implementadas, sem necessidade de grandes modificações de código. Além disso, elas possuem acesso às configurações de sistema, permitindo assim a atribuição de valores de configurações específicos.
- **Gerenciamento de operações:** Uma classe de gerenciamento de processamento foi criada para evitar que um usuário posterior se preocupasse com os detalhes da sequência de processamento em si. Somente é necessário que o futuro desenvolvedor defina quais classes realizarão as tarefas de pré-processamento, extração-pulmonar e detecção de candidatos. Finalmente, uma vez terminado os processamentos, esta classe contém os candidatos a nódulo detectados naquela imagem.
- **Padronização *key-value*:** Durante o desenvolvimento, o autor se preocupou em tentar manter o formato *key-value* que seria utilizado posteriormente por uma aplicação de Hadoop. Assim, criou-se classes base para depois serem utilizadas no desenvolvimento de seus equivalentes compatíveis com o Hadoop⁴¹.
- **Padronização de saída:** A saída do *framework* de processamento de máquina única foi uma primeira versão da saída prevista para o Hadoop. Definiu-se o formato JSON⁴² na saída, por sua baixa verbosidade quando comparada com arquivos HTML e por ser facilmente legível a seres humanos.

⁴¹ Em geral, a grande diferença destas classes para as implementadas em Hadoop é que no segundo estas classes implementam as classes do tipo Writable e Comparable, que serão mencionadas posteriormente no documento.

⁴² É a abreviatura para JavaScript Object Notation e ele é a ferramenta principal para a transmissão de dados entre servidores e aplicações web.

3.5.2.2. Bibliotecas Utilizadas

Bibliotecas de código livre e *plugins* foram utilizados. Abaixo tem-se a lista deles e a descrição de suas funcionalidades:

- **ImageJ:** Biblioteca utilizada como base para o processamento de imagem digitais e leitura de arquivos DICOM. Originalmente o ImageJ é uma aplicação para o tratamento de imagens (IMAGEJ, 2014). No entanto, o código fonte é aberto e este pode ser utilizado como biblioteca. Este código permite a utilização de módulos criados por usuários. Neste caso, utilizou-se os *plugins*:
 - **ijblob:** Tem como foco a análise de componentes conectados e texturas. Foi utilizada para a detecção de pulmões e detecção de candidatos a nódulos (WAGNER; LIPINSKI, 2013).
 - **Gray Morphology:** Contém o código necessário para realizar as operações morfológicas - como por exemplo fechamento - nas máscaras binárias e imagens com profundidade de cinza de 8 bits (PRODANOV, 2013).
- **Google Gson:** É uma biblioteca Java desenvolvida pela Google para converter objetos Java em sua representação equivalente no formato JSON (GOOGLE, 2014).

3.5.2.3. Descrição de classes chave

O javadocs⁴³ e o código fonte final do Lung Processor estão disponíveis em (LOTZ, 2014). Esta seção se conterà a uma rápida descrição das classes/packages mais importantes nesta aplicação.

⁴³ Javadoc, desenvolvido pela Oracle Corporation, cria a documentação de uma API no formato HTML utilizando o código fonte Java.

- **Package com.marcolotz:**

- DicomReader: Esta classe realiza a leitura dos arquivos DICOM utilizando a API do ImageJ. Além disso, ela gera a *key Series KeyStructure* utilizada no processamento de MapReduce e as informações da *image* para o *value* Image Metadata.
- ImageViewer: Sua principal função é externar imagens, utilizando Java Swing⁴⁴, para que o usuário possa verificar o desempenho de seus módulos de processamento de imagem.

- **Package com.marcolotz.imageprocess:**

- ImageProcessor: Gerencia o processamento de imagens em geral. No momento em que ele é criado o usuário deve definir as classes de pré-processamento, extração, detecção de candidatos e um objeto de configuração.
- ImageProcessingClass: É uma classe abstrata que necessita ser estendida por qualquer classe do processamento de imagens. O método *process* deve ser definido em extensões desta classe. Somente objetos que estendem esta classe são aceitos pelo ImageProcessor.

Os outros pacotes do código tem como meta principal manter a compatibilidade do código fonte desta solução com o código que será posteriormente utilizado no HadoopLung. Nos anexos, está disponível o diagrama UML da solução.

3.5.2.4. Demonstração do formato de saída

Conforme mencionado anteriormente, utilizou-se JSON como formato de saída. O Quadro 3 apresenta um exemplo de saída para o Lung Processor. No

⁴⁴ Java swing é o principal kit de ferramentas para a geração de interfaces gráficas de usuário na linguagem Java.

quadro estão disponíveis somente os candidatos a nódulo de uma imagem. As informações de *key* descrevem uma *series*, enquanto os *values* deste *key* é uma lista de metadados da imagem e dos candidatos a nódulo em cada uma delas. No caso do *framework* de máquina única, a lista contém somente uma imagem.

```
{
  "keyStructure": {
    "seriesInstanceUID": "
1.2.840.113704.7.32.07.5.1.4.64127.30000014051406391514000019084",
    "studyDate": " 20140514",
    "seriesDate": " 20140514",
    "studyTime": " 112021.984000 ",
    "seriesTime": " 112530.750000 ",
    "modality": " CT",
    "manufacturer": " SIEMENS ",
    "institutionName": " DAPI - PALLADIUM",
    "institutionAddress": " Street ",
    "stationName": " CTAWP64127",
    "studyDescription": " Thorax^TORAX_AR_DAPI (Adult)",
    "patientsName": " [OMITTED] ",
    "patientsID": " 0006025503",
    "bodyPartExamined": " CHEST ",
    "sliceThickness": " 1 ",
    "kVP": " 120 ",
    "pixelSpacing": " 0.72265625\\0.72265625 "
  },
  "reducedValue": {
    "reducedList": [
      {
        "SOPInstanceUID": " 1.2.840.113704.7.1.0.16914518613371.1401913167.11860",
        "imageDate": " 20140514",
        "imageTime": " 112129.801000 ",
        "aquisitionNumber": " 2 ",
        "imageNumber": " 140 ",
        "imagePosition": " -185.638671875\\-376.138671875\\-183.3",
        "imageOrientation": " 1\\0\\0\\0\\0\\1\\0 ",
        "CandidatesList": [
          {
            "width": 5.0,
```

```

        "height": 9.0,
        "xCoord": 334,
        "yCoord": 291,
        "circularity": 21.475789473684216,
        "enclosedArea": 19.0,
        "perimeter": 20.200000000000003
    },
    {
        "width": 9.0,
        "height": 8.0,
        "xCoord": 243,
        "yCoord": 309,
        "circularity": 219.6984384,
        "enclosedArea": 2.5,
        "perimeter": 23.436
    }
]
}
]
}

```

Quadro 3 – Exemplo de saída para o Lung Processor

3.5.3. HadoopLung

Esta aplicação Java implementa o conceito do processamento de imagens distribuído descrito anteriormente. Seu desempenho será descrito em uma sessão posterior. Conforme mencionado em páginas pretéritas, o autor atentou para garantir que todo o código de processamento de imagem, ou seja, qualquer classe que siga os padrões do package *com.marcolotz.imageprocess* possa ser incluída sem qualquer modificação do projeto Lung Processor para o projeto HadoopLung.

3.5.3.1. Configurações

Usou-se um objeto da classe *Configuration* para a definição de parâmetros no processamento digital de imagens. Como os *Mappers* e os *Reducers* tem código independente e isolados entre si, a única maneira de se passar parâmetros de linha

de comando para máquinas virtuais executando estes agentes é através de arquivos de configuração. Um Hadoop *job*, quando é iniciado, é executado inicialmente em máquina única. Neste período, pode-se realizar configurações e análise de parâmetros na linha de comando. Somente após a chamada do método *WaitForCompletion()* é que o Hadoop *job* é distribuído.

Para diminuir a necessidade de configurações por linha de comando, criou-se um arquivo XML, presente dentro do diretório *conf*, chamado *LungConfiguration.xml*. Aqui o usuário pode incluir todas as configurações a serem posteriormente lidas na fase de processamento de imagem.

O arquivo XML é lido enquanto o processamento está em máquina única. Alguns parâmetros, como os níveis de cinza dos candidatos, podem ser definidos em linha de comando e irão se sobrepor aos valores padrão já definidos no arquivo. Esta funcionalidade foi criada para facilitar a execução do Hadoop por script⁴⁵. O arquivo de configuração está disponível nos anexos.

3.5.3.2. Principais Parâmetros

A seguir, os principais atributos e valores utilizados no arquivo XML serão comentados:

- `Mapreduce.job.reduces`
 - Valor configurado: 48.
 - Tentou-se utilizar um reducer por processador físico. Um incremento desse valor afeta muito pouco o desempenho deste Hadoop job em específico, uma vez que a parte mais lenta é a de Map.
- `Mapreduce.reduce.memory.mb`
 - Valor configurado: 1024.

⁴⁵ Uma maneira mais elegante de se atualizar configurações por linha de comando será discutida posteriormente neste documento.

- O *reducer* em si utiliza pouca memória, devido ao fato de processar e ordenar pequenas quantidades de metadados.
- `Mapred.input.dir.recursive`
 - Valor configurado: `true`.
 - Isso possibilita o processamento de todos os arquivos dentro de pastas presentes no diretório de entrada. Ou seja, habilita a recursividade.
- `com.marcolotz.grayNoduleCandidates.bottomThreshold`
 - Valor configurado: 110
 - Valor mínimo padrão utilizado na extração de candidatos a nódulo.
- `com.marcolotz.grayNoduleCandidates.topThreshold`
 - Valor configurado: 120
 - Valor máximo padrão utilizado na extração de candidatos a nódulo.

3.5.3.3. Diferenças entre Lung Processor e o HadoopLung

As classes que representam pares *key-value* no HadoopLung são ligeiramente diferentes das utilizadas no Lung Processor. Isto porque as presentes na primeira solução, além de conter todo o código presente na segunda solução, necessitam implementar as classes *Writable* e *Comparable*. A primeira serve como uma maneira de possibilitar a rápida serialização⁴⁶ e deserialização de dados dentro do *framework*. A segunda, por sua vez, é utilizada principalmente para procedimento de comparação, permitindo assim verificar se dois objetos são iguais ou ordená-los.

Na solução desenvolvida pelo autor, duas *keys* são consideradas iguais se o valor presente no campo *SeriesInstanceUID* é o mesmo. Assim, as imagens de uma mesma série serão todas agrupadas no *reducer* em formato de lista. Analisando

⁴⁶ No campo da ciência da computação, mais especificamente na área de armazenamento de dados, serialização é definida como processo de tradução de estrutura de dados (ou estados de objetos) em um formato que possa ser corretamente armazenado.

tecnicamente, caso as classes aqui delineadas para HadoopLung fossem utilizadas no Lung Processor, a solução funcionaria corretamente. A recíproca, no entanto, não é verdadeira.

3.5.3.4. Saída da solução distribuída

A saída da solução distribuída é similar à de máquina única em formato. Ou seja, ambas utilizam JSON. No entanto, o HadoopLung agrupa as imagens de uma mesma *series* e o Lung Processor computa somente uma imagem. O resultado final é a serialização de objetos do tipo *SeriesDataWritable*, que nada mais é do que uma maneira de representar os metadados de uma *series* e das imagens que a compõe.

Uma vez terminada a fase de *reducing*, suas saídas são agrupadas em um único destino. O resultado final é o conjunto de vários objetos JSON serializados em somente um arquivo. Este tipo de entrada não segue os padrões de um arquivo JSON normal – pois ele contém vários arquivos JSON dentro de um único. Este tipo de comportamento será posteriormente contornado no instante de pós-processamento. O tamanho de saída para os 53 pacientes analisados pode alcançar até 100 megabytes, dependendo dos parâmetros utilizados durante o processamento

3.5.3.5. Diagrama UML e código

O código fonte e os javadocs da solução estão disponíveis em (LOTZ, 2014). O diagrama UML encontra-se em anexo.

3.5.4. Lung Post-Processor

Essa solução Java implementa o bloco de pós-processamento descrito anteriormente. Em média o código aqui desenvolvido demora em torno de 5 segundos para pós-processar todos os exames dos 53 pacientes. Suas principais características são descritas nos próximos tópicos.

3.5.4.1. Leitura da Saída do HadoopLung

Como mencionado anteriormente, os arquivos de entrada da solução de pós-processamento (e portanto a saída do HadoopLung) não seguem os padrões JSON. Por tal razão, o autor utilizou um leitor de fluxo (*stream*) da biblioteca GSON. Isso possibilitou a extração de uma *series* por leitura.

3.5.4.2. Otimização de utilização de Memória

Caso no futuro essa solução venha a ser utilizada para ler arquivos de entrada ainda maiores, o autor decidiu que a execução da aplicação deve ser intercalada entre ciclos de leitura e pós-processamento. Isso diminui a quantidade de memória alocada e garante que somente a última *series* lida está ocupando espaço em RAM. Estima-se que, em média, dois candidatos a nódulo são encontrados por imagem. Uma *series* contém em torno de 500 imagens. Cada string de caracteres contém 32 caracteres, com um byte cada. Um inteiro ou ponto flutuante é representado por 32 bits (4 bytes). Existem em média 5 *series* por paciente. Assim, pode-se estimar o consumo de memória por:

$$\text{Tamanho de uma string} = 32 * 1 \text{ bytes} = 32 \text{ bytes}$$

$$\text{Tamanho de metadados de series} = 18 \text{ strings} * 32 \text{ bytes} = 576 \text{ bytes}$$

$$\text{Tamanho de informações por candidato}$$

$$= (5 \text{ pontos flutuantes} + 2 \text{ inteiros}) * 4 \text{ bytes} = 28 \text{ bytes}$$

$$\text{Tamanho de informações por imagem} = 2 \text{ candidatos} + 7 \text{ Strings} = 280 \text{ bytes}$$

$$\begin{aligned} \text{Tamanho por SeriesDataWritable} &= 500 \text{ imagens} + 1 \text{ metados de series} \\ &= 137,28 \text{ kBytes} \end{aligned}$$

$$\text{Tamanho por paciente} = 5 * \text{tamanho por SeriesDataWritable} = 686,40 \text{ kBytes}$$

$$\text{Tamanho para todos os pacientes} = 53 * 686,40 \text{ kBytes} = 35,52 \text{ Mbytes}$$

Embora para o número de pacientes aqui analisados a quantidade de memória pareça baixa⁴⁷, caso o número de pacientes fosse somente uma ordem de grandeza maior, a quantidade de memória alocada ofereceria um entrave considerável dependendo da máquina utilizada para o processamento. Por essa razão, fez-se a leitura em etapas⁴⁸.

3.5.4.3. Agrupamento

Ao término do pós-processamento de cada *series*, os resultados são armazenados e ordenados por *PatientID*. Caso uma *series* aponte o resultado positivo de nódulos pulmonares, aquele paciente será considerado com nódulos.

3.5.4.4. Verificação de desempenho

Próximo ao final da execução da solução, uma classe realiza a validação dos resultados. Ou seja, ela compara o resultado de cada paciente com os presentes nos laudos médicos dos exames. Assim, o resultado do diagnóstico de um paciente pode ser classificado em um dos quatro grupos:

- Verdadeiro positivo: Quando o resultado esperado do paciente é positivo e o método aqui descrito indicou positivo para nódulos pulmonares.
- Falso positivo: Quando o resultado esperado do paciente é negativo e o método indicou positivo.
- Falso negativo: Quando o resultado esperado é positivo e o método indicou negativo
- Verdadeiro negativo: Quando o resultado esperado é negativo e o método indicou negativo.

⁴⁷ Vale ressaltar que nesse cálculo não inclui-se a descrição da quantidade de memória utilizada para JVM para descrever cada um desses objetos - o que é maior que o valor descrito.

⁴⁸ Uma solução melhor será sugerida posteriormente neste documento.

A solução então calcula a taxa de verdadeiros positivos e falsos positivos, a partir das fórmulas:

Taxa de Verdadeiros Positivos

$$= \frac{\text{número de verdadeiros positivos}}{\text{número de verdadeiros positivos} + \text{número de falsos negativos}}$$

Taxa de Falsos Positivos

$$= \frac{\text{número de falsos positivos}}{\text{número de falsos positivos} + \text{número de verdadeiros negativos}}$$

Além disso, calcula-se a distância euclidiana do ponto de ótimo – ou seja, o ponto com a taxa de verdadeiros positivos 1 e falsos positivos zero. O calculo é feito pela seguinte fórmula:

distância

$$= \sqrt{(1 - \text{taxa de verdadeiros positivos})^2 + (0 - \text{taxa de falsos positivos})^2}$$

Com isto, pode-se comparar parâmetros de pós-processamento para encontrar o menos distante do ideal.

3.5.4.5. Chamada do programa via linha de comando

Existe a possibilidade de se utilizar dois ou cinco argumentos através da linha de comando. No caso da primeira opção, os dois argumentos são o arquivo de entrada (ou seja, o arquivo .json da saída do HadoopLung) e o arquivo de saída para armazenar os resultados

Para o caso de cinco argumentos, a solução permite que sejam determinadas a distância entre os centróides de dois candidatos de imagens adjacentes (em pixels), o tamanho mínimo de um nódulo (em milímetros) e o tamanho máximo (também em milímetros). Para gerenciar tal característica o autor implementou uma classe abstrata que mantém os parâmetros de referência

utilizados para pós-processamento. Os valores presentes nessa classe são lidos do arquivo de configuração *postConfiguration.xml*.

Os atributos que podem ser configurados por este arquivo são listados na Tabela 3:

Tabela 3 – Atributos configuráveis em PostConfiguration.xml

Nome do atribute	Valor padrão	Descrição
com.marcolotz.nodule.centroidDistance	2	Define a distância de centróides para considerar dois candidatos como integrantes do mesmo nódulo.
com.marcolotz.nodule.r2Above	5.0	Nódulos com R2 acima deste valor são eliminados
com.marcolotz.nodule.r2Below	0.2	Nódulos com R2 abaixo deste valor são eliminados
com.marcolotz.nodule.r3Minimum	1.5	Nódulos com R3 acima deste valor são eliminados
com.marcolotz.nodule.dxPsizeBelow	2.0	Nódulos com dimensões em x abaixo destas são eliminados
com.marcolotz.nodule.dyPsizeBelow	2.0	Nódulos com dimensões em y abaixo destas são eliminados
com.marcolotz.nodule.dxPsizeAbove	40	Nódulos com dimensões em x acima destas são eliminados
com.marcolotz.nodule.dyPsizeAbove	40	Nódulos com dimensões em y acima destas são eliminados

3.5.4.6. Saída

O primeiro tipo de saída para essa solução é no arquivo de saída definido na linha de comando. Os resultados de verificação são adicionados ao fim do arquivo e possui o seguinte formato em linha única:

```
111      123      5.0      10.0      30.0      0.2558  0.3000  0.802379
```

A interpretação destes dados é mostrada na Tabela 4.

Tabela 4 – Interpretação dos resultados de pós-processamento.

Mínimo valor de cinza	Máximo valor de cinza	Distância máxima entre centróides	Dimensões mínimas do nódulo	Dimensões máximas do nódulo	Taxa de verdadeiros positivos	Taxa de falsos positivos	Distância do ponto ideal:
111	123	5.0	10.0	30.0	0.2558	0.3000	0.8023

Este formato foi utilizado devido a sua facilidade na interpretação pelos scripts AWK, que serão comentados na sessão posterior.

O segundo tipo de saída são os dados externados na saída padrão do sistema. Nesta saída é apresentado o resultado de cada um dos pacientes, bem como quais exames apontaram para o resultado de positivo. Além disso, é possível verificar o número de positivos e negativos.

3.5.4.7. Javadocs e diagrama UML

O código fonte desta solução, bem como seu javadocs estão disponíveis em (LOTZ, 2014). Nos anexos encontra-se um diagrama de classes UML da aplicação.

3.5.5. Scripts

Abaixo serão detalhados os funcionamentos dos principais scripts desenvolvidos no teste de desempenho da solução. Todos os scripts desenvolvidos estão disponíveis nos anexos deste documento.

3.5.5.1. ROCProcess.sh

Este script possui dois valores de cinza, um mínimo e um máximo e é utilizado para modificar o resultado do processamento de imagem para vários parâmetros de cinza. Para cada par de valores chama-se um Hadoop *job* e realiza-se a detecção de candidatos com eles. Por padrão os valores mínimos e máximos do script são 110 e 135 respectivamente. Ele inicia o processamento utilizando estes dois valores. Uma vez que o *job* termina, ele copia e armazena os resultados disponíveis no HDFS para o diretório de saída determinado no script. O arquivo é nomeado com o seguinte padrão: output[Valor mínimo]_[Valor máximo].json

A cada iteração, ele decrementa o valor máximo em 1 até que o valor máximo seja 5 valores inteiros maior que o mínimo. O valor mínimo então é incrementado em um e o valor máximo é reiniciado ao valor padrão original (no caso 135). O mesmo processo é repetido até que todos os valores possíveis sejam

calculados. O critério de valor mínimo foi estabelecido para reduzir a necessidade de processamento desnecessário, uma vez que uma janela de 5 níveis de cinza, em geral, é muita pequena para detectar candidatos relevantes.

O tempo médio de execução do script é 18 horas no cluster e gera 231 arquivos de saída no formato JSON. O volume total de dados na saída é de 12,15 Gbytes. Isso permitirá, posteriormente, a verificação de qual nível de cinza na seleção de candidatos possui melhor desempenho.

3.5.5.2. PostProcess.sh

Este script pós-processa todos os arquivos JSON gerados na saída do script anterior, utilizando variações nos valores de parâmetros para a mesma entrada. Os parâmetros de pós-processamento levados em conta são:

- **distância de centróides de candidatos:** São os valores em que dois candidatos, em imagens adjacentes e com uma distância de centróide menor que o parâmetro, serão considerados componentes do mesmo nódulo pulmonar. Os valores aqui testado são: 1, 2, 3, 4 e 5 pixels.
- **O valor mínimo da componente máxima de um nódulo nos eixos x e y:** Também referidos anteriormente como $dx * p_size_x$ e $dy * p_size_y$. Nódulos que tenham os valores abaixo deste limite serão considerados falsos positivos. Os valores utilizados no teste são: 2, 4, 6 e 8 milímetros
- **O valor da componente máxima de um nódulo nos eixos x e y:** Similarmente ao item acima, os nódulos que tenham os valores acima deste limite serão considerados falsos positivos. Os valores utilizados no teste são: 40, 30, 20 e 15 milímetros.

Através da utilização de todas as faixas de cinza disponíveis de entrada, é possível ter uma estimativa do valor de falsos positivos e verdadeiros positivos para cada combinação de parâmetros, avaliando-se assim o desempenho do algoritmo.

Este script foi executado no computador Apple descrito na sessão de Hardware. Seu tempo de execução foi de aproximadamente 20 horas.

3.5.5.3. FindSolutions.sh

Uma vez que todos os dados tenham sido processados, aplica-se este script para facilitar a visualização dos melhores candidatos dentro de um arquivo de saída de pós-processamento. Sua análise é sobre a última coluna disponível no arquivo de saída, que contém a distância euclidiana entre resultado do pós-processamento e o ponto ideal.

Com ele, o usuário pode selecionar o arquivo de entrada que contenha resultados de pós-processamento e o número de melhores resultados que ele deseja. O primeiro resultado sempre será o mais próximo do ideal - e portanto o com a menor distância euclidiana. O script em realidade contém um script AWK⁴⁹ que irá processar os dados e aplicar o algoritmo de seleção⁵⁰ sobre eles.

Um exemplo de saída pode ser visualizado no Quadro 4:

Looking for the best 10 solutions on input file: tmp/ROCValues.data							
# AWK Script written by Marco Lotz (marcolotz.com)							
# The output syntax is:							
# Bottom grey treshold Upper grey threshold Maximum centroid distance Minimum nodule dimensions Maximum nodule dimensions rate of true positives rate of false positives Distance from top left ROC result							
# Printing the most effective parameters, from most effective to least effective:							
111	129	1.0	8.0	15.0	0.6744186282157898	0.2000	0.38210368
111	128	1.0	8.0	15.0	0.604651153087616	0.1000	0.40779984
111	128	1.0	8.0	20.0	0.604651153087616	0.1000	0.40779984
111	123	4.0	2.0	30.0	0.7209302186965942	0.3000	0.40973154
111	123	4.0	2.0	40.0	0.7209302186965942	0.3000	0.40973154
110	129	1.0	8.0	15.0	0.6279069781303406	0.2000	0.42243725
111	123	4.0	2.0	20.0	0.6976743936538696	0.3000	0.4259117
111	125	2.0	2.0	15.0	0.6976743936538696	0.3000	0.4259117
111	125	3.0	2.0	15.0	0.6976743936538696	0.3000	0.4259117
115	127	1.0	2.0	40.0	0.6976743936538696	0.3000	0.4259117

Quadro 4 - 10 melhores resultados para o script ROCProcess.sh

⁴⁹ AWK é uma linguagem interpretada de programação projetada para processamento de texto e comumente utilizada para a extração de dados e criação de relatórios.

⁵⁰ Algoritmo que permite encontrar o k-ésimo menor valor em um lista ou vetor.

3.5.5.4. IterateSolution.sh

Uma vez que, através do findSolutions.sh, tenham sido encontrados alguns dos níveis de cinza com resultados mais promissores, o script iterateSolution.sh foi desenvolvido para encontrar de um mínimo local⁵¹ (uma vez que é somente para aqueles níveis de cinza). De maneira similar ao script de pós-processamento, este script utiliza vários valores para os três parâmetros anteriormente descritos. Uma vez que o tamanho dos dados de entrada é bem menor (por se restringir a somente um arquivo), é possível utilizar uma maior gama de parâmetros na avaliação de desempenho. Agora os valores para cada um dos parâmetros são:

- **distância de centróides de candidatos:** 0.01; 0.1; 0.2; 0.3; 0.4; 0.5; 0.6; 0.8; 1; 1.5; 2; 3; 4; 5. Os valores estão em pixels.
- **O valor mínimo da componente máxima de um nódulo nos eixos x e y:** Também referidos anteriormente como $dx * p_size_x$ e $dy * p_size_y$. Nódulos que tenham os valores abaixo deste limite serão considerados falsos positivos. Os valores utilizados no teste são: 2; 4; 6; 8; 10; 12; 14. Os valores estão em milímetros.
- **O valor da componente máxima de um nódulo nos eixos x e y:** Similarmente ao item acima, os nódulos que tenham os valores acima deste limite serão considerados falsos positivos. Os valores utilizados no teste são: 40; 35; 30; 25; 20; 15 milímetros.

⁵¹ Um mínimo local, também chamado de mínimo relativo, é um conceito de inteligência artificial para designar o mínimo valor em uma vizinhança. Este mínimo pode, ou não, ser um mínimo global (ou seja, um mínimo para todas as vizinhanças).

4. RESULTADOS E DISCUSSÕES

O *framework* de processamento de máquina única teve sua lógica de processamento de imagem completamente baseada no script de MATLAB. O autor se atentou em garantir que os resultados finais de processamentos de ambos fossem idênticos, caso os mesmo parâmetros fossem aplicados.

A implementação distribuída, também chamada de HadoopLung, é compatível integralmente com os pacotes desenvolvidos para o Lung Processor (conforme foi mencionado anteriormente). Com isto, pode-se esperar que o resultado de processamento de uma única imagem no processamento distribuído será idêntico ao realizado em máquina única.

4.1. OTIMIZAÇÕES DE PROCESSAMENTO

Uma vez que a aplicação HadoopLung foi terminada por completo, ela foi testada para o *data set* completo, ou seja, em todas as imagens DICOM. Devido à necessidade de configurações de pequenos parâmetros do cluster, seu desempenho esteve muito aquém do ótimo nas primeiras iterações. Os parâmetros que foram configurados serão discutidos a seguir.

4.1.1. Configurações de *Container*.

Como mencionado anteriormente, o *data set* completo contém 41305 imagens. Durante a primeira execução de HadoopLung, o autor verificou que o código demorava 5 horas e 30 minutos (aproximadamente) para finalizar todo o processamento de imagens.

Através das análises descritas a seguir, verifica-se que este valor está longe do ideal:

$$\frac{41305 \text{ imagens}}{330 \text{ minutos}} = 125,16 \text{ imagens/minuto}$$

Vale lembrar que o cluster possui 48 processadores reais e 48 virtuais (HyperThreading). Neste exemplo, vamos assumir que somente processadores reais⁵² estejam sendo utilizados, tem-se que:

$$\frac{125,16 \text{ imagens/minuto}}{48 \text{ processadores}} = 2,60 \frac{\text{imagens}}{\text{minuto}} \text{ por processador}$$

Com isso comparou-se o desempenho por processador individual do cluster com o do *Lung Processor*. Uma imagem, na segunda solução, tomava em torno de 1 segundo para ser processada, aferidos pela classe *ImageProcessor*.

Após analisar os parâmetros do cluster, o autor finalmente encontrou o fator gerador desta performance, que remete aos conceitos de *container* relatados na sessão de fundamentação teórica. Na verificação da utilização de cada máquina, cada nó, em média, possuía somente 8% de sua memória sendo utilizada e 34% sua capacidade de CPU sendo consumida. Uma vez que os nós deste cluster não estavam alocados em outras atividades, era desejável que a aplicação consumisse integralmente a CPU ou a memória disponível (o que acontecesse primeiro).

Na plataforma YARN, ao invés de se definir a quantidade de *Tasks* disponíveis dentro de um *TaskTracker* como ocorria no MapReduce 1.0, define-se a quantidade de *containers* que será disponibilizada pelo *NodeManager*. Para se definir o número destes, utiliza-se a quantidade de memória disponível e a quantidade de memória sugerida. Dentro de cada container, existirá uma Máquina Virtual Java (JVM) isolada e independente das demais. Esta máquina virtual contém ou um *Mapper* ou um *Reducer* para o caso do MapReduce 2.0.

Por padrão, o Cloudera Manager configura o *NodeManager* para permitir que no máximo 8 Gbytes de memória sejam alocados para *containers*. Esta característica é definida pelo parâmetro (BAKHSI, 2013):

yarn.nodemanager.resource.memory-mb

⁵² Em realidade, pode-se utilizar o total de processadores reais e virtuais. Este tipo de aproximação levaria a um número que seria a metade do valor encontrado. Como o valor encontrado será comparado com o desempenho do processador único computador Apple descrito na sessão de hardware, utilizar-se-á neste momento somente o número de processadores reais.

Além disso, por padrão define-se que um container utilizará ao menos 1 Gbyte de memória. Isto é feito pelo parâmetro:

yarn.scheduler.minimum-allocation-mb

Com isso, tem-se que:

$$\frac{8 \text{ Giga bytes disponíveis para containers por nó}}{1 \text{ Giga byte por container}} = 8 \text{ containers por nó}$$

Assim, somente 8 *containers* iriam ser alocados simultaneamente em um nó. Recomenda-se a alocação de 1 a 2 *containers* por disco por núcleo (HORTONWORKS, 2014a). Assim, tem-se a seguinte formula:

$$\text{número de containers} = \text{minimum}(2 * \text{número de núcleos}, \\ 1.8 * \text{número de discos}, \frac{(\text{RAM total disponível})}{\text{Tamanho mínimo do container}})$$

Substituindo os valores para cada nó, tem-se:

$$\text{número de containers} = \text{minimum}(24, \quad 5,4, \quad 24)$$

Embora o nó tenha 32 Gbytes de memória, o autor decidiu manter 4 Gbytes de memória RAM para o sistema operacional e outros 4 para manter outros softwares funcionando em background (como o Cloudera Manager e o HBase).

O número de discos entra na conta devido a tempo de busca⁵³ para cada um dos arquivos locais. Embora, o resultado para o cálculo do número máximo de

⁵³ Do inglês *seek time*. É o tempo que leva para um controlador de disco rígido para localizar um fragmento de dado armazenado.

containers tenha sido 5,4, foi utilizado 24⁵⁴ *containers*. Foi mantido o valor original mínimo de memória por *container* e incrementou o valor de memória disponível de 8 para 24 Gbytes.

Estas configurações dos *containers* resultaram em um processamento bem mais eficiente em termos de uso das CPUs e velocidade, conforme a Figura 33 e a Figura 34. A capacidade do processador é consumida totalmente, permitindo afirmar que este tipo de computação é de natureza de utilização intensiva da CPU, ao invés de ser de uso intenso de memória – uma vez que somente 14 Gigabytes desta são utilizados por nó. Vale ressaltar que a utilização de CPU total do cluster fica em média, limitada a 80% porque o servidor em si é considerado na avaliação dos dados estatísticos. Este contém o *NameNode*, o Cloudera Manager e o *ResourcesManager*, não sendo utilizado para a computação dos dados. Essa percentagem indica que quatro dos cinco *hosts* avaliados pelo programa estão utilizando integralmente suas CPU.

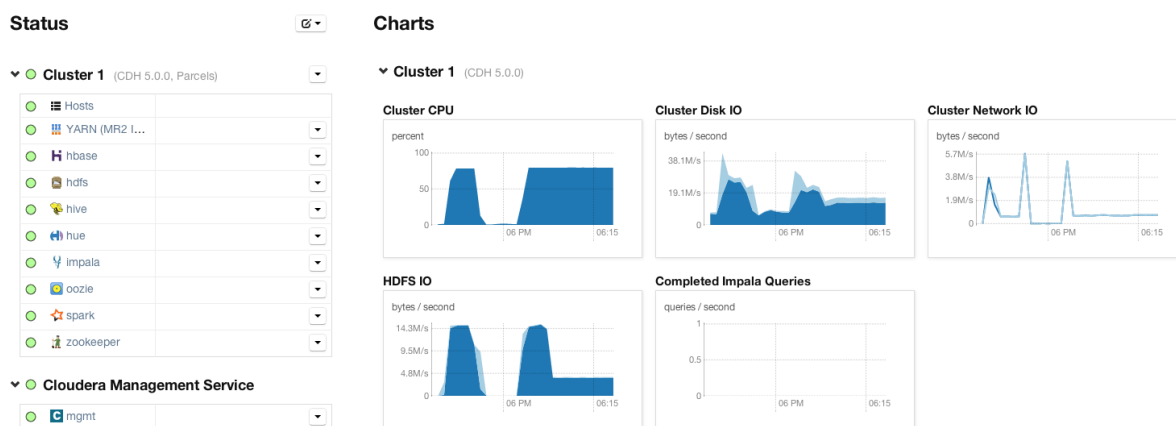


Figura 33 – Métricas de utilização do Cluster. Fonte: Autoria própria.

⁵⁴ A razão da utilização do valor 24 provém da otimização realizada na sequência, que aglutinou os arquivos. O disco rígido em si não seria um limitante uma vez que haveria poucas transferências entre a memória e o disco local.

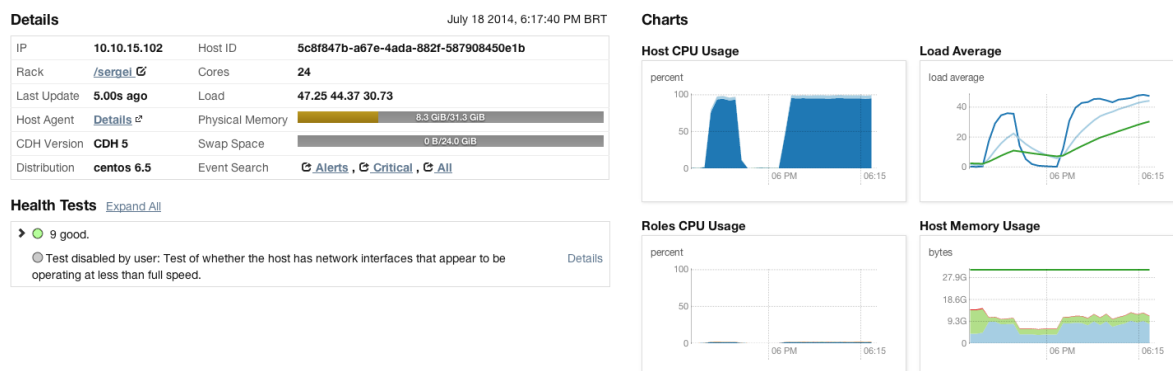


Figura 34 – Métricas de utilização do nó 1. Fonte: Autoria própria.

Com todas as otimizações, o tempo final de processamento caiu de 5 horas e 30 minutos para 1 hora e 40 minutos. Com isto, pode-se calcular o *speed up*⁵⁵ da solução por:

$$\text{speed up} = \frac{\text{Tempo de execução antigo}}{\text{Tempo de execução novo}} = \frac{330}{100} = 3,3$$

O valor acima indica que o tempo final para o processamento de imagens após a otimização do número de containers é 3,3 vezes menor que o original.

Com estas configurações, o novo valor para o número de imagens processadas por minuto é:

$$\frac{41305 \text{ imagens}}{100 \text{ minutos}} = 413,05 \text{ imagens/minuto}$$

Novamente, estimou-se o *throughput*⁵⁶ por processador do cluster. Apenas processadores reais para o cálculo.

⁵⁵ *Speed up* é uma métrica utilizada para o cálculo de melhoria de desempenho na execução de uma task. Originalmente, esta noção foi definida na lei de Amdahl - que será comentada na sequência.

⁵⁶ Este termo é definido, no ramo da tecnologia, como a quantidade de trabalho que um computador pode fazer em um certo período de tempo.

$$\frac{413,05 \text{ imagens/minuto}}{48 \text{ processadores}} = 8,60 \frac{\text{imagens}}{\text{minuto}} \text{ por processador}$$

O desempenho ainda estava muito abaixo do desempenho esperado quando comparado com os resultados de tempo de processamento para o Lung Processor, que utiliza somente um processador. Os tópicos a seguir descrevem o estudo dos procedimentos para a melhoria deste desempenho.

4.1.1.1. Número de Input Splits

Nos relatórios de funcionamento do Hadoop *job* que estava sendo submetido ao cluster havia a indicação de 41305 *input splits*. Ou seja, devido ao código original da aplicação, cada imagem a ser processada representava um *input Split*.

4.1.1.2. O Input Split

O número e a lógica para a criação de um *input split* é definida em código no Hadoop. Esta definição está associada ao objeto que implementa a classe abstrata *InputFormat*. De maneira simples, um *input split* é um pedaço de dados que será processado por um único *Mapper*.

Cada *input split* é dividido no que são chamados *records*. O *Mapper* recebe um *input split* e executa a função de *Map* para cada um dos *records* nele presentes. Uma característica muito importante dos *input splits* e dos *records* é que eles devem ser lógicos. Ou seja, eles não devem estar associados à maneira com que o conteúdo da entrada será interpretada diretamente. Esta é uma característica muito importante, uma vez que os *input splits* são definidos em máquina única⁵⁷, e, portanto, em código não concorrente. A seguir são apresentadas explicações sobre a necessidade de manter o tempo concorrente ao mínimo possível.

⁵⁷ Esta definição ocorre antes do *job* ser distribuído.

4.1.1.3. Lei de Amdahl

Segundo a lei de Amdahl (BROWN, 2004), tem-se a seguinte fórmula de *speed up* para o caso de paralelização:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Onde P é a parte paralelizável do código executante (ou seja, aquela que pode se beneficiar de ser paralelizada) e (1-P) é a porção que não pode ser paralelizada (ou seja, continua com execução concorrente). O maior *speedup* que pode ser alcançável com N processadores⁵⁸ é encontrado pela fórmula acima. No limite, quando o número de processadores N tende a infinito, tem-se que:

$$S(N) = \lim_{N \rightarrow \infty} \frac{1}{(1 - P) + \frac{P}{N}}$$

O resultado desta equação é:

$$S(N) = \frac{1}{(1 - P)}$$

Ou seja, o *speed up* máximo é limitado pelo tempo da parte concorrente (não paralela) do código, representada na equação acima pelo denominador. Essa é a principal razão pela qual utiliza-se o particionamento lógico para a definição de *input splits*. Por tal razão, a parte concorrente dura o menor tempo possível, visto que não há necessidade de analisar o conteúdo dos dados de entrada mas sim sua estrutura.

⁵⁸ Supõe-se que cada processador somente realiza uma tarefa por vez.

4.1.1.4. Relação de Input Splits com Máquinas Virtuais Java

Cada *input split* alimenta um *Mapper* e, portanto, um *container*. Assim, quanto maior o número de *input splits*, maior o número de *containers* e de Java Virtual Machines alocadas.

O tempo de alocação e desalocação de uma JVM pode ser estimado em aproximadamente um segundo (WHITE, 2012). Por tal razão, pode-se estimar o tempo aproximado de alocação de máquinas virtuais quando se tem uma imagem por *input split*.

$$\frac{41305 \text{ input splits}}{1 \text{ segundo por split}} = 41305 \text{ segundos}$$

$$\frac{41305 \text{ segundos}}{48 \text{ processadores}} = 860 \text{ segundos} = 14,34 \text{ minutos}$$

Portanto 14,34 minutos de tempo paralelizável estavam sendo utilizados para a inicialização de JVMs. Além disso, há o tempo de busca dos arquivos. Sempre que um *Mapper* for utilizar um *input split*, ele irá tentar transferir todo o conteúdo do *split* do disco para a memória. Assim, quanto maior o número de *input splits*, mais tempo será gasto em *seek time*. Tentando assim reduzir este período não útil para processamento, recomenda-se diminuir o número de *input splits*.

4.1.1.5. O problema dos arquivos pequenos

Além do tempo de alocação de máquinas virtuais Java, há outros problemas atrelados ao armazenamento de arquivos pequenos⁵⁹ para o Hadoop. Estes problemas são descritos nos tópicos a seguir:

⁵⁹ Na comunidade, este problema é conhecido como *small files problem*.

4.1.1.5.a. Problemas com o HDFS

Como o leitor deve se recordar, o HDFS utiliza blocos para representar os arquivos. Um arquivo pode conter um ou mais blocos, que, por padrão possuem o tamanho de 64 Megabytes⁶⁰. Arquivos muito menores que o tamanho de um bloco ocupam um espaço de dados que poderia ser utilizado por completo.

Um arquivo, diretório ou bloco no HDFS representa um objeto na memória do *NameNode*. Estima-se que cada entrada consome aproximadamente 150 bytes de memória (WHITE, 2009). Assim, no caso desta solução em especial todos os arquivos utilizam:

$$150 \text{ bytes} * 41305 \text{ arquivos} = 6195750 \text{ bytes} = 6 \text{ Mbytes de memória}$$

Embora, para a quantidade de arquivos descrita neste documento, este valor seja pequeno, para um número maior (e não incomum) de arquivos de entrada, pode-se facilmente consumir toda a memória de um *NameNode*. Por exemplo, 10 milhões de arquivos pequenos consomem aproximadamente 1,5 Gbytes de memória do *NameNode*. Caso esta solução venha a ser extrapolada para todos os arquivos disponíveis em uma clínica de diagnóstico por imagem, recomenda-se a compactação e aglutinação de arquivos. No caso abordado por este documento, pode-se verificar que este fator não incorreu em um problema severo.

4.1.1.5.b. Problemas com MapReduce

Além do *overhead* de criação e destruição de máquinas virtuais Java, há o problema de agendamento e registro de cada um dos *Mappers* lançados. No MapReduce 1.0 havia o recurso de reutilização de máquinas virtuais, que foi retirado quando a versão YARN foi lançada. Como solução, o autor implementou um objeto da classe abstrata *CombileFileInputFormat*, que será descrito na sequência.

⁶⁰ No caso do código descrito nessa solução, utilizou-se uma granularidade de HDFS de 128 Mbytes.

4.1.1.6. CombileFileInputFormat

Este tipo de InputFormat utiliza detalhes da arquitetura do HDFS para a geração de *input splits*. Por padrão, ele define que o tamanho de *split* será do tamanho da granularidade do HDFS – no caso utilizado 128 Mbytes. Em seguida, ele aglutina várias imagens em somente um *input split*. As imagens são fundidas até que o tamanho do *split* se aproxime da granularidade do sistema distribuído de arquivos. Finalmente, ele faz com que cada *record* deste *input split* seja em realidade um arquivo – e, portanto uma imagem DICOM.

Além disso, este formato de entrada garante que os *records* não poderão ser divididos logicamente. Com isto garante-se que todos os arquivos estarão completos em um *input split* e não separados em dois ou mais.

4.1.1.7. Desempenho

Uma vez modificada a maneira com que o Hadoop lê os arquivos de entrada, fez-se novamente uma análise de desempenho. Observou-se, em um primeiro momento que o número total de *input splits* caiu para 197. O tempo total de processamento foi de exatos 4 minutos e 42 segundos, ou seja, 282 segundos. O relatório deste Hadoop Job está disponível nos anexos. Comparando-se com os resultados anteriores, tem-se que:

$$\frac{41305 \text{ imagens}}{4.7 \text{ minutos}} = 8788,3 \text{ imagens/minuto}$$

Novamente, aferiu-se o *throughput* por processador real do cluster.

$$\frac{8788,3 \text{ imagens/minuto}}{48 \text{ processadores}} = 183,08 \frac{\text{imagens}}{\text{minuto}} \text{ por processador}$$

$$\text{speed up} = \frac{\text{Tempo de execução antigo}}{\text{Tempo de execução novo}} = \frac{330}{4,7} = 70,2$$

Por fim, atingiu-se um *speedup* maior que 70. Os tempos dos três níveis de otimização podem ser comparados visualmente no gráfico da Figura 35.

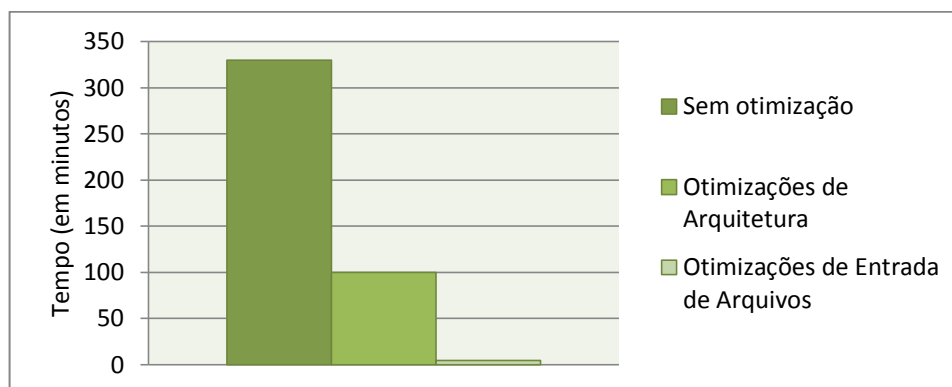


Figura 35 - Tempo total de processamento para cada um dos níveis de otimização. Fonte: Autoria própria.

4.2. COMPARAÇÕES DE PÓS-PROCESSAMENTO

Para verificar o desempenho da solução contra os exames fornecidos, a aplicação de pós-processamento foi avaliada em uma ampla gama de parâmetros. Originalmente, calculou-se todos os candidatos que poderiam ser encontrados levando em conta os níveis de cinza entre 110 e 135. Posteriormente, essas saídas foram pós-processadas com alguns parâmetros para a caracterização de nódulos. Por último, os resultados mais promissores foram iterados com menor granularidade dos parâmetros para garantir que o melhor valor fosse encontrado⁶¹.

Considerando todas as saídas geradas pelo script PostProcess.sh, tem-se o seguinte gráfico da Figura 36.

⁶¹ Os parâmetros utilizados para estas análises são descritos na sessão de scripts e estes estão disponíveis em anexo. Aqui, são enfatizadas as comparações dos resultados.

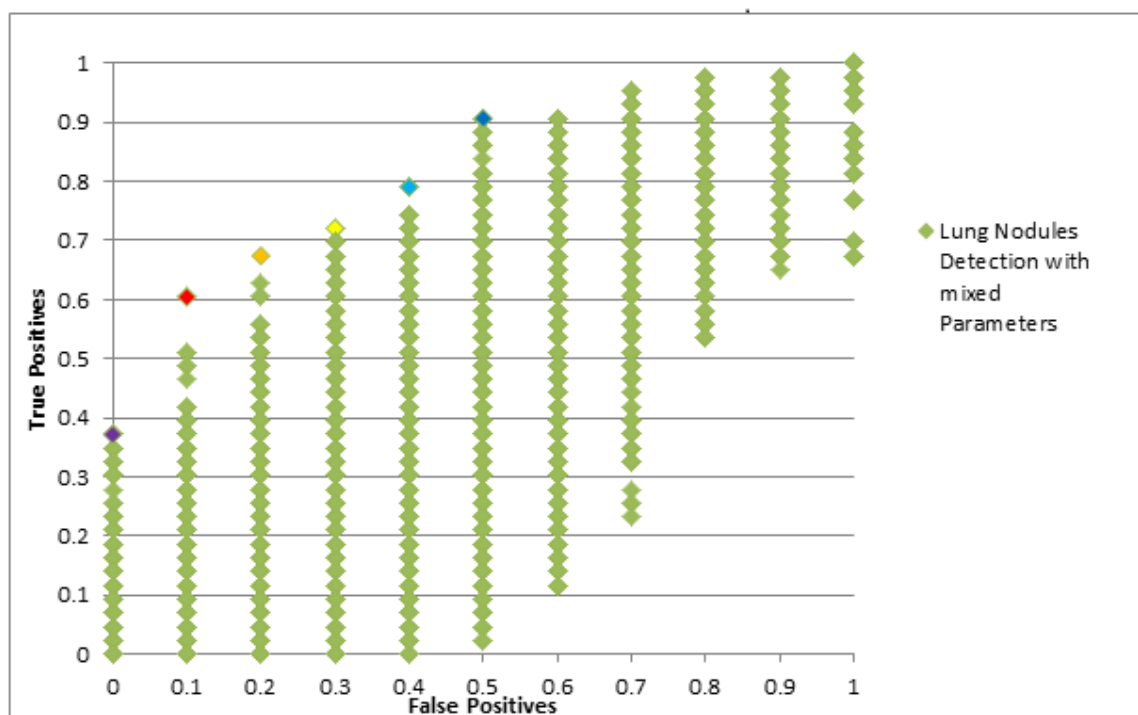


Figura 36 - Comparação de Verdadeiros Positivos e Falsos Positivos. Fonte: Autoria própria.

Tabela 5 – Locais de interesse.

Cor	Mínimo valor de cinza	Máximo valor de cinza	Distância máxima entre centróides	Dimensões mínimas do nódulo	Dimensões máximas do nódulo	Taxa de verdadeiros positivos	Taxa de falsos positivos	Distância do ponto ideal
Red	111	129	1.0	8.0	15.0	0.6744	0.20	0.3821
Orange	111	128	1.0	8.0	15.0	0.6046	0.10	0.4077
Yellow	111	123	4.0	2.0	30.0	0.7209	0.30	0.4097
Light Blue	111	126	1.0	2.0	20.0	0.7906	0.40	0.4514
Dark Blue	111	128	2.0	2.0	15.0	0.9069	0.5	0.5085
Purple	110	125	1.0	8.0	40.0	0.3720	0.0	0.6279

As combinações de parâmetros que resultaram em melhor desempenho são apresentadas na Tabela 5. Os três primeiros valores da tabela foram selecionados devido a sua proximidade com a situação ideal. Os dois seguintes pela sua alta taxa de verdadeiros positivos. O último, finalmente, foi selecionado pela sua baixa taxa de falsos positivos. Os próximos tópicos apresentam, para os pontos obtidos, os resultados da análise dos parâmetros com maior granularidade.

4.2.1. Candidatos entre 111 e 129

Ao aumentar a faixa dos parâmetros de pós-processamento para o arquivo com candidatos encontrados entre os limiares de cinza 111 – 129 na parte de processamento de imagens foram obtidos os resultados mostrados na Figura 37:

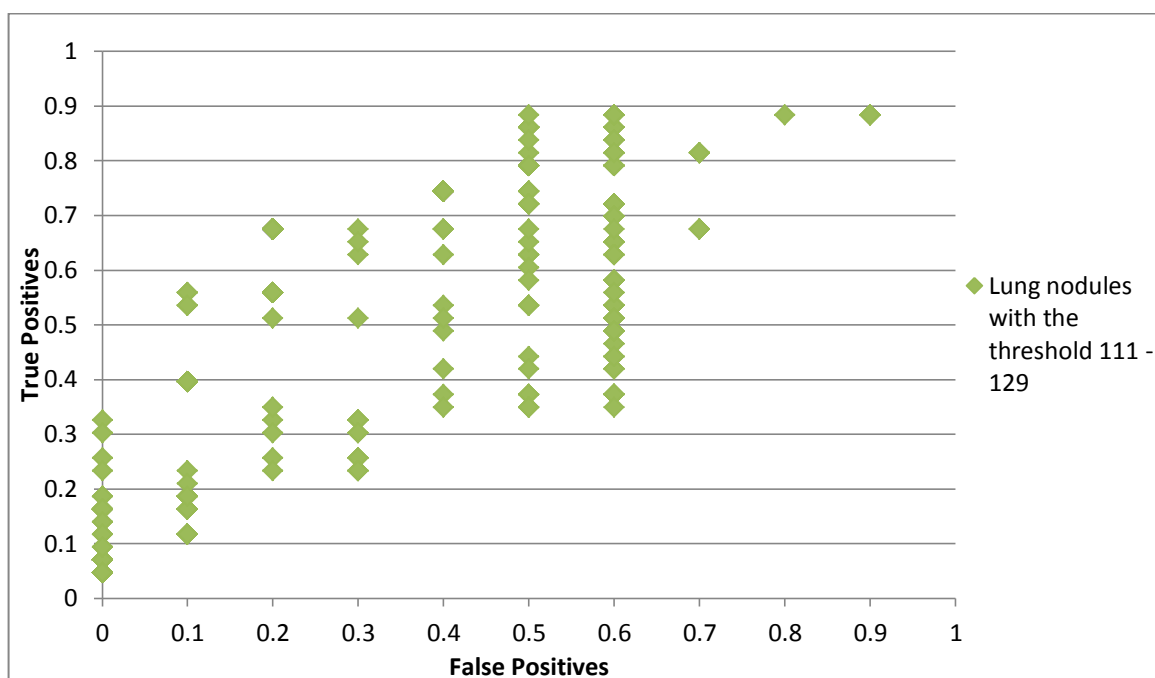


Figura 37 – Desempenho de candidatos encontrados com os limiares 111-129. Fonte: Autoria própria.

O ponto que mais se aproxima da solução ideal foi realmente o mostrado anteriormente. Para esta faixa de cinza, os melhores três resultados (encontrados pelo script *findSolutions.sh*) foram os mostrados na Tabela 6:

Tabela 6– melhores valores para o limiar 111-129

Distância máxima entre centróides	Dimensões mínimas do nódulo	Dimensões máximas do nódulo	Taxa de verdadeiros positivos	Taxa de falsos positivos	Distância do ponto ideal:
1.0	8.0	15.0	0.6744	0.20	0.3821
0.01	2.0	40.0	0.6744	0.20	0.3821
0.01	2.0	35.0	0.6744	0.20	0.3821

4.2.2. Candidatos entre 111 e 128

Os resultados para esta faixa de valores são mostrados no gráfico da Figura 38.

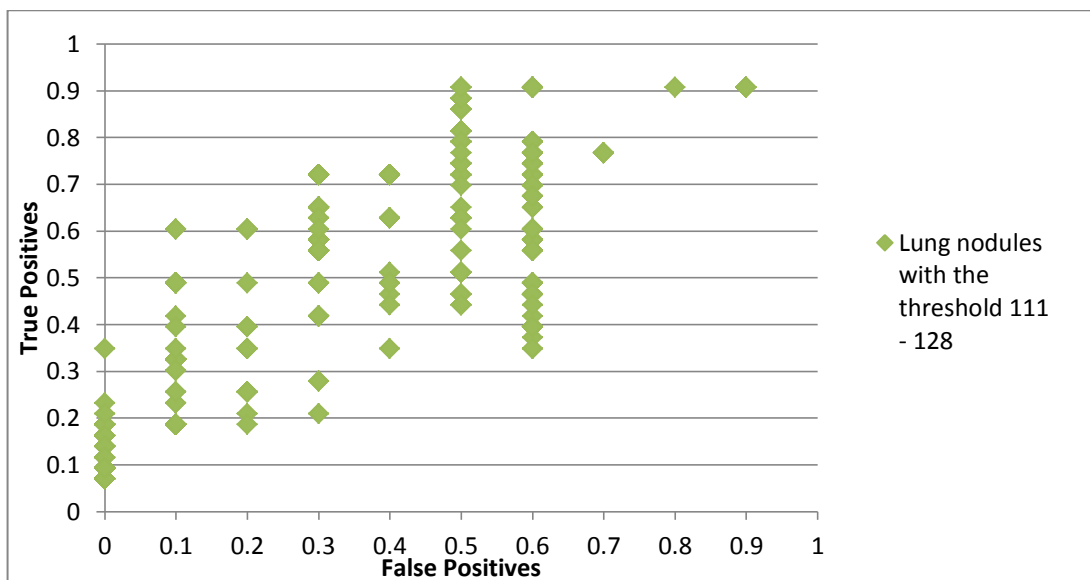


Figura 38 – Desempenho de candidatos encontrados com os limiares 111-128. Fonte: Autoria própria.

Verifica-se que, mesmo com a iteração dos valores, os pontos mais interessantes nessa faixa de cinza mantiveram-se os mesmos da Tabela 5, conforme mostra a Tabela 7:

Tabela 7 – melhores valores para o limiar 111-128

Distância máxima entre centróides	Dimensões mínimas do nódulo	Dimensões máximas do nódulo	Taxa de verdadeiros positivos	Taxa de falsos positivos	Distância do ponto ideal:
1.0	8.0	20.0	0.6046	0.10	0.4077
1.0	8.0	15.0	0.6046	0.10	0.4077
4.0	2.0	30.0	0.7209	0.30	0.4097

4.2.3. Candidatos entre 111 e 123

Os resultados para esta faixa de valores são mostrados no gráfico da Figura 39.

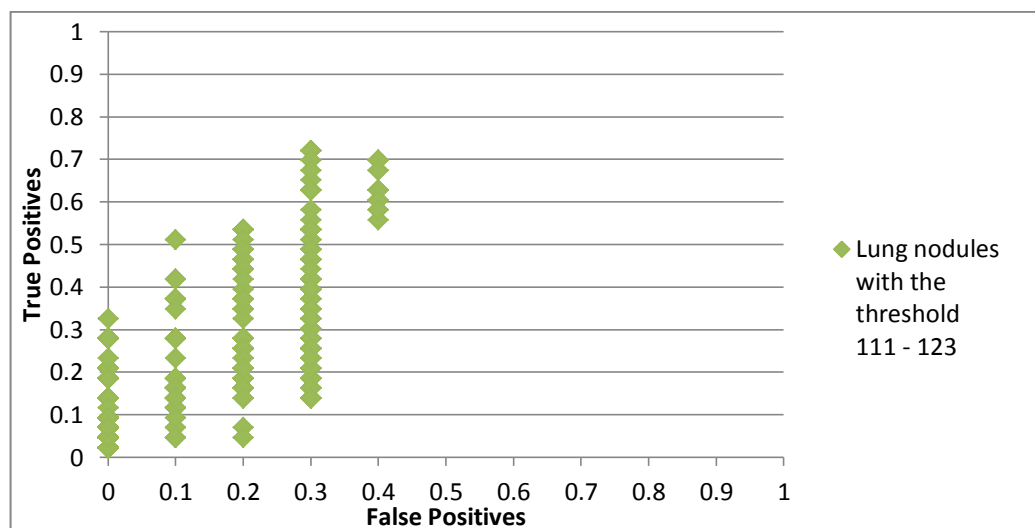


Figura 39 – Desempenho de candidatos encontrados com os limiares 111-123. Fonte: Autoria própria.

Verifica-se que, mesmo com a iteração dos valores, os pontos mais interessantes nessa faixa de cinza mantiveram-se os mesmos da Tabela 5, conforme mostra a Tabela 8:

Tabela 8 – melhores valores para o limiar 111-123

Distância máxima entre centróides	Dimensões mínimas do nódulo	Dimensões máximas do nódulo	Taxa de verdadeiros positivos	Taxa de falsos positivos	Distância do ponto ideal:
4.0	2.0	40.0	0.7209	0.30	0.4097
4.0	2.0	35.0	0.7209	0.30	0.4097
4.0	2.0	30.0	0.7209	0.30	0.4097

4.2.4. Candidatos entre 111 e 126

Os resultados para esta faixa de valores são mostrados no gráfico da Figura 40.

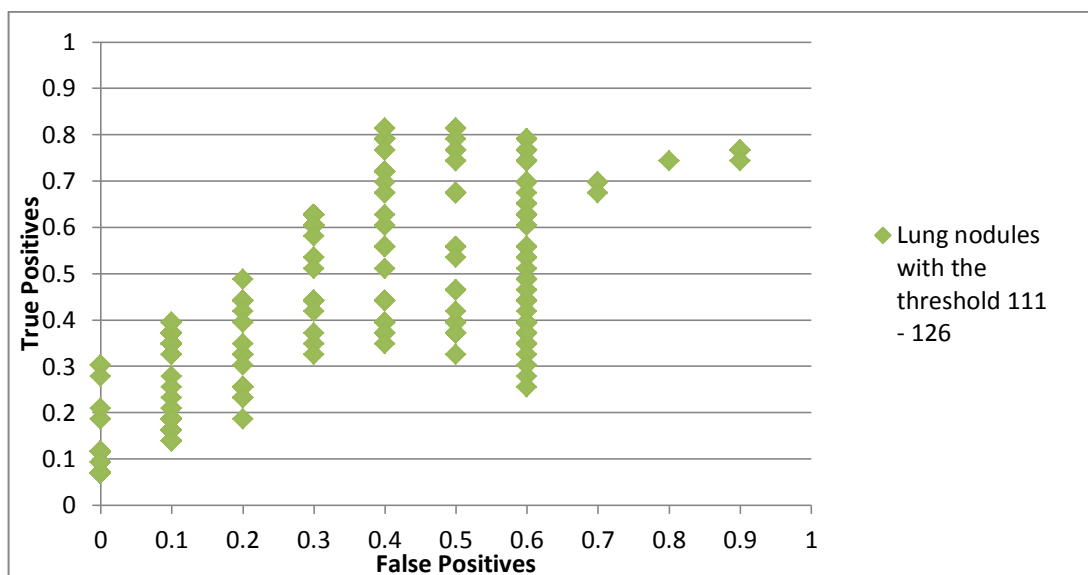


Figura 40 – Desempenho de candidatos encontrados com os limiares 111-126. Fonte: Autoria própria.

Verifica-se que, com a iteração dos valores, os três pontos mais interessantes nessa faixa de cinza são os mostrados na Tabela 9. Nesta faixa de cinza percebe-se que houve melhorias devido à iteração de parâmetros mais detalhados. Os dois valores superiores da Tabela 9 não estavam disponíveis quando as análises de todas as faixas de cinza foram feitas.

Tabela 9 – melhores valores para o limiar 111-126.

Distância máxima entre centróides	Dimensões mínimas do nódulo	Dimensões máximas do nódulo	Taxa de verdadeiros positivos	Taxa de falsos positivos	Distância do ponto ideal:
1.5	2.0	20.0	0.8139	0.4	0.44115
1.5	2.0	15.0	0.8139	0.4	0.44115
1.0	2.0	30.0	0.7906	0.4	0.4514

4.2.5. Candidatos entre 110 e 125

Os resultados para esta faixa de valores são mostrados no gráfico da Figura 41.

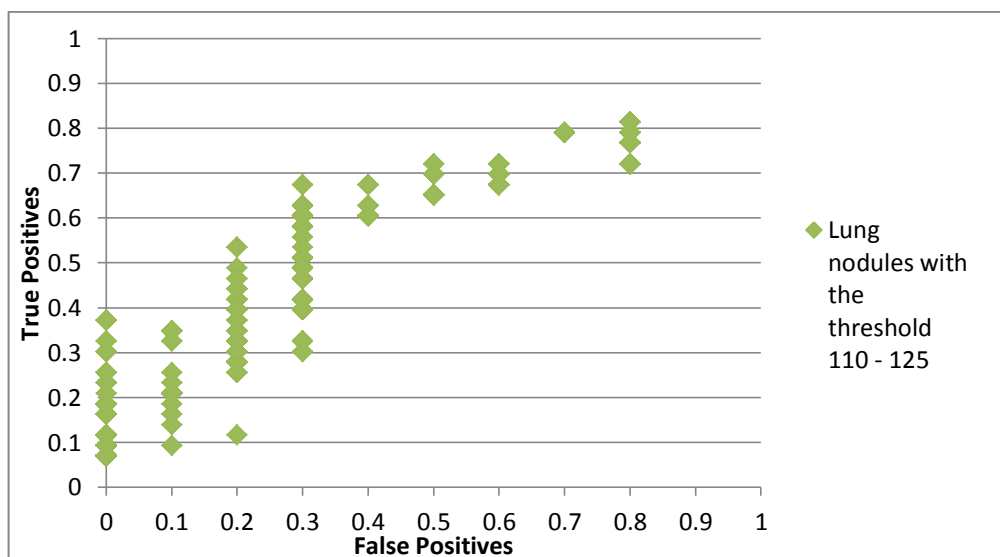


Figura 41 – Desempenho de candidatos encontrados com os limiares 110-125. Fonte: Autoria própria.

Verifica-se que, com a iteração dos valores, os três pontos mais interessantes nessa faixa de cinza são os mostrados na Tabela 10. Embora os valores não estejam mais próximos da solução ideal, o terceiro ponto da tabela apresenta uma característica especial. Ele é o ponto como maior número de verdadeiros positivos sem falsos positivos em todas as interações. Este ponto será comentado em seções posteriores.

Tabela 10 – melhores valores para o limiar 110-125.

Distância máxima entre centróides	Dimensões mínimas do nódulo	Dimensões máximas do nódulo	Taxa de verdadeiros positivos	Taxa de falsos positivos	Distância do ponto ideal:
3.0	6.0	20.0	0.6744	0.30	0.4427
3.0	6.0	15.0	0.6744	0.30	0.4427
1.0	8.0	40.0	0.3720	0.0	0.6279

4.3. CONSIDERAÇÕES FINAIS

Este trabalho demonstrou como o modelo computacional MapReduce pode ser aplicado no âmbito do processamento de imagens médicas em grandes volumes para permitir que as facilidades da computação paralela produzam um pré-diagnóstico. Por pré-diagnóstico leia-se a possibilidade de um diagnóstico antes que uma autoridade com respaldo legal e conhecimento (neste caso um médico) possa dar seu veredito final.

O exemplo aqui descrito para o processamento de imagens pulmonares pode, não somente ser extrapolado facilmente para outros exames, mas também, devido à modularidade do software *HadoopLung* e *Lung Processor*, ser utilizada para comparar inúmeras abordagens desenvolvidas por futuros alunos e pesquisadores. O autor espera com esta produção incentivar futuros desenvolvimentos na área, permitindo melhorar ainda mais os métodos de detecção e classificação de patologias.

Como foi apresentado na sessão anterior – através de gráficos com verdadeiros positivos e falsos positivos – a solução possui parâmetros que permitem a configuração de um ponto de funcionamento. Este ponto deve levar em conta o tipo de análise que se deseje realizar. A Tabela 11 apresenta alguns exemplos.

Tabela 11 – Tabela de comparação de pontos.

Referência	Mínimo valor de cinza	Máximo valor de cinza	Distância máxima entre centróides	Dimensões mínimas do nódulo	Dimensões máximas do nódulo	Taxa de verdadeiros positivos	Taxa de falsos positivos	Distância do ponto ideal:
(A)	111	128	2.0	2.0	15.0	0.9069	0.5	0.5085
(B)	111	129	1.0	8.0	15.0	0.6744	0.20	0.3821
(C)	110	125	1.0	8.0	40.0	0.3720	0.0	0.6279

Caso se almeje um alto grau de verdadeiros positivos, sem compromisso com a possibilidade de falsos negativos, pode-se escolher os parâmetros definidos na referência (A) da Tabela 11. Por outro lado, caso deseje-se o melhor ponto entre verdadeiros positivos e falsos positivos, ou seja, o ponto mais próximo do ideal de 100% de verdadeiros positivos e 0% positivos, deve-se escolher o ponto (B). Finalmente, caso vise-se o maior número de verdadeiros positivos sem falsos positivos, o ponto ótimo seria o (C).

Considerando também o desempenho da solução como um todo, incluindo seu tempo de processamento, o autor acredita que é possível o processamento de todo o banco de dados de uma empresa de diagnósticos clínicos, quando trata-se de tomografias de pulmão.

Além disso, a solução apresenta flexibilidade para que outros pesquisadores e alunos possam, no futuro, desenvolver outros métodos de processamento, tanto de pulmão quando outros exames que possam ser baseados nos princípios da plataforma. A modularização do *framework* permite que vários códigos sejam

implementados e comparados com as soluções já existentes, tanto em tempo de processamento quanto taxa de verdadeiros positivos/falsos positivos.

Com relação ao tempo de processamento, de uma maneira simplificada, pode-se dizer que a solução implementada é 146,5 vezes mais rápida, se comparada à execução em uma única máquina.

4.4. TRABALHOS FUTUROS

A seguir, são sugeridos alguns pontos a serem explorados para a continuidade do trabalho.

4.4.1. No âmbito de processamento de imagem:

4.4.1.1. Extração pulmonar

Devido aos níveis Hounsfield utilizado para a extração pulmonar, muitas vezes nódulos juxta-pleurais são considerados área externa quando a extração pulmonar é realizada. Assim, o autor sugere que um futuro desenvolvedor utilize, além do método atual, um recurso de análise de contorno.

4.4.1.2. Detecção de nódulos candidatos

Embora a análise de uma faixa de níveis de cinza (convertidos a partir da escala HU) para a detecção de candidatos tenha apresentado bons resultados, pesquisadores futuros podem desenvolver *plugins* que utilizem algoritmos como o *Local Density Maximum* (ZHAO *et al.*, 2003), ou que até mesmo tenha valores de referência de escala Hounsfield para nódulos pulmonares. Com este último talvez seja possível não somente detectar nódulos, mas também classificá-los como benignos ou malignos.

4.4.1.3. Desenvolvimento de *Parser* de configuração

Além de aprimoramentos nos métodos de processamento de imagem, seria desejável que o usuário conseguisse modificar todas as configurações de processamento. Isto pode ser possível através da análise de parâmetros passados por linha de comando, atualizando o valor daquele atributo anteriormente lido no arquivo XML. Um *parser* validaria a sintaxe de passagem de configurações e permitiria assim a atualização daquele atributo. Seu funcionamento basearia na classe *GenericOptionsParser*, que interpreta linhas de comando comuns para o Hadoop e as configura no objeto do tipo *Configuration*.

4.4.1.4. Configuração das classes de processamento

Ao invés de modificar o código fonte para incluir quais classes serão utilizadas em cada módulo do processamento de imagem, estes valores poderiam estar presentes no arquivo XML de configuração. Assim, o usuário não precisaria interagir com nenhum outro segmento de código que não fosse implementações de módulos como o de extração pulmonar ou o de detecção de candidatos.

4.4.2. No âmbito de pós-processamento

Embora o tempo de pós-processamento seja desprezível quando comparado com o tempo de processamento digital de imagens, ele pode ser facilmente paralelizado em outro MapReduce *job*. Isto seria ideal para quando o número de *series* é muito alto. O autor sugere então a utilização de Apache Oozie⁶² para este tipo de processamento. Cada *Mapper* poderia receber uma *series* (ou

⁶² Apache Oozie é uma ferramenta que permite o agendamento de fluxo de *jobs* para o Hadoop.

melhor ainda, a saída integral de um reducer do passo anterior⁶³), onde seriam realizados os passos descritos neste documento. Posteriormente, uma chave seria emitida contendo o *PatientID*. Assim, no *reducer* estariam o resultado de todas as *series* de um mesmo paciente. Finalmente, os resultados deste paciente seriam atualizados levando por base o resultado do pós-processamento da lista de *series*.

Há a possibilidade, também, de incluir o pós-processamento no mesmo MapReduce *job* que realiza o processamento de imagens. Para tal, seria necessário que a implementação garantisse, na etapa de *sorting*, que todas as *series* de um mesmo paciente estejam presentes por completo em um mesmo *Reducer*. Na fase de *Reduce* seria, então, realizado o pós-processamento.

⁶³ Essa solução é ainda mais elegante porque não é necessário acessar os conteúdos do arquivo antes de enviá-los para o *mapper*, mantendo assim a limitação de *input split* somente por critérios lógicos.

5. GESTÃO

5.1. ESCOPO

Durante a etapa de planejamento de trabalho de conclusão de curso, os requisitos mínimos foram definidos pelo discente. Entre estes requisitos, pode-se citar:

- O uso de imagens no padrão DICOM.
- O processamento de imagens no padrão acima em busca de determinadas características.
- Uso de Hadoop como a implementação de MapReduce selecionada.
- Utilização do Cluster institucional.
- Munir-se de imagens de tomografias tendo como fonte os bancos de dados do DAPI.
- Utilização de Java como a linguagem principal a ser distribuída entre os nós.
- Preparação do cluster para sua utilização em outras aplicações de *Big Data* por outros discentes no futuro.

Após o início da implementação, o autor tentou se manter fiel às definições originais. Devido ao extenso processo de planejamento, ao conhecimento prévio do autor na área – e claro, porque não, um pouco de fatores estocásticos associados -, não houveram modificações palpáveis no escopo original do projeto. Assim, todos os itens propostos no escopo do projeto foram cumpridos e estão presentes no projeto.

5.2. CRONOGRAMA

Uma das principais características de um projeto é seu tempo de vida. Em prol do desenvolvimento de um projeto funcional, seu ciclo de desenvolvimento deve ser esmiuçado e cada uma das suas atividades que o compõe devem ser

detalhadas de uma maneira realista. Atividades são, por sua vez, subdivididas em tarefas, que constituem a unidade mínima de um projeto.

O projeto detalhado neste documento foi separado em três atividades, cada uma delas composta de várias tarefas. As atividades são: Planejamento e Definição; Implementação e Teste; Depuração e Finalização. O cronograma desenvolvido prevê que o projeto leve ao todo nove meses para ser finalizado passando por cada uma dessas fases na sequência descrita acima.

Na fase de Planejamento e Definição (em azul na **Error! Reference source not found.**), definiram-se os requisitos do projeto – como aspectos tecnológicos, fonte de dados, recursos utilizados, possíveis abordagens. Além disso, adquiriu-se conhecimento sobre a área do processamento de imagens. Como resultados dessas tarefas estipulou-se utilizar o cluster do departamento de física da UTFPR, bem como a base de dados do DAPI e Hadoop como *framework* de processamento distribuído.

Posteriormente, na fase de Implementação e Teste, o autor definiu o sistema operacional e o *framework* que atenderiam melhor a seus requisitos – como facilidade de instalação, possibilidade de depuração, utilização posterior por terceiros. Definiu-se assim o CentOS 6.5 e a distribuição de Hadoop da Cloudera CDH5. Após testes para verificar a integridade do sistema, o autor iniciou a implementação em script de MATLAB da solução, seguido do desenvolvimento de um *framework* modular. Uma vez que ambos estavam com funcionamento satisfatório, prosseguiu-se para a implementação da solução distribuída. Esta atividade pode ser visualizada na cor verde na tabela.

Finalmente, na última atividade terminou-se a documentação e apresentou-se os resultados a banca avaliadora, na atividade chamada de Depuração e Finalização.

A Tabela 12 representa o cronograma das atividades desenvolvidas neste período.

Tabela 12 - Cronograma

Tarefas	Meses								
	Jan	Fev	Mar	Abr	Mai	Jun	Jul	Ago	Set
Pesquisas dos aspectos tecnológicos do projeto.									
Definição dos requisitos do projeto.									
Definição da tecnologias a serem utilizadas.									
Definição de um banco de imagens.									
Definição da interface de processamento de imagens.									
Pesquisas das soluções para o processamento computacional da patologia.									
Estudo de como realizar a implementação do projeto.									
Pesquisas sobre temas teóricos do processamento de imagens.									
Preparação do cluster para a instalação do <i>framework</i> .									
Instalação do <i>framework</i> escolhido no cluster									
Otimização de parâmetros do <i>framework</i> para o cluster.									
Instalação da interface para processamento de imagens.									
Realização de testes para validar o correto funcionamento da interface									
Implementação de <i>softwares</i> de processamento de imagens									
Realizar testes para verificar a eficácia do reconhecimento de patologia.									
Utilizar o <i>software</i> em um conjunto com grande banco de dados.									
Realizar a documentação do projeto.									
Finalizar Documentação									
Apresentação à Banca Avaliadora									

5.3. CUSTOS

Devido principalmente à utilização de recursos de *hardware* públicos, como o cluster da Universidade Tecnológica Federal do Paraná e os computadores do

Laboratório de Processamento de Imagens Médicas, a maior parte dos custos do autor foram para adquirir material de ensino e pesquisa.

Os softwares utilizados nas máquinas são *open-source*, como o caso da distribuição de Linux CentOS e o código fonte do Apache Hadoop. A distribuição de Hadoop utilizado, a Cloudera CDH5, também é gratuita. As imagens processadas foram doadas pelo DAPI.

Foi necessário, no entanto, adquirir um roteador com capacidade de roteamento de QoS⁶⁴ para garantir a comunicação confiável dos computadores no laboratório do processamento de imagens médicas com o cluster. Isto fez-se necessário uma vez que, devido a natureza massiva dos dados de entrada, necessita de uma conexão segura e de alta velocidade entre as máquinas que estão depurando e testando as soluções e o cluster em si.

A Tabela 13 lista os valores gastos para o desenvolvimento deste projeto.

Tabela 13 - Custos Estimados.

Atividade	Custo
Livro: Hadoop Operations	R\$ 130,00
Livro: MapReduce Design Patterns: Building Effective Algorithms and Analytics for Hadoop and other systems	R\$ 120,00
Livro: Hadoop Operations and Cluster Management Cookbook	R\$ 120,00
Livro: Hadoop Real World Solution Cookbook	R\$ 120,00
Roteador Wi-Fi TP-Link 300 MB/s	R\$ 130,00
Total	R\$ 620,00

5.4. RISCOS

A seguir é apresentada uma tabela com alguns riscos do projeto. Existem três possíveis graus para riscos: grau alto (riscos com grau entre 0,5 e 1); grau

⁶⁴ Do inglês *Quality of Service*, recursos que alguns roteadores possuem de priorizar alguns tipos de pacotes perante outros.

médio (riscos com grau entre 0,25 e 0,5); grau baixo (riscos com grau entre 0,0 e 0,25). O grau é calculado pela multiplicação da probabilidade de um risco acontecer pelo impacto daquele risco. Tanto a probabilidade quanto o impacto são normalizados, ou seja, limitados entre zero e um. A Tabela 14 descreve estes riscos:

Tabela 14 – Análise de riscos.

Grau	Riscos	Efeito	Probabilidade	Impacto	Ação Mitigante
0,1	Incompatibilidade de <i>framework</i>	Retorno à fase de pesquisa e Planejamento	0,1	1	Definir outro <i>framework</i>
0,1	Cluster corrompido	Atraso na Implementação	0,1	1	Reinstalar todo o conteúdo
0,25	Problemas de software	Processamento de imagem incorreto	0,5	0,5	Encontrar outra biblioteca de processamento de Imagens
0,3	Problemas de detecção de patologias	Detecção de área incorretas	0,6	0,5	Redefinir o método de detecção
0,08	Banco de dados muito escasso	Pouca diversidade para validação dos testes	0,2	0,4	Definir outra fonte de dados
0,1	Problemas na implementação do software distribuído	Problemas na implementação distribuída do código	0,2	0,5	Depurar o <i>framework</i> .

5.5. CONSIDERAÇÕES

Todo o projeto descrito nessa sessão foi realizado no prazo estipulado. Dos riscos avaliados para o projeto, somente pequenas problemas na implementação do software distribuído ocorreram. Essas foram solucionadas através de métodos de depuração e teste – utilizando principalmente logs e o comportamento do *framework* equivalente para máquina única.

Os custos se contiveram aos descritos na seção acima, sendo principalmente no âmbito de aquisição de conhecimento. Não se pode ignorar o fato de que o hardware utilizado para o desenvolvimento deste documento é de propriedade pública. Assim sendo, quaisquer gastos relativos à energia e refrigeração não foram repassados ao autor.

O âmbito de gestão, se pode afirmar que o projeto aqui delineado seguiu perfeitamente as estimativas iniciais do autor. Isto provém principalmente de experiências passadas do discente na área, bem como o bom alinhamento de fatores estocásticos.

6. REFERÊNCIAS

APACHE FOUNDATION. PoweredBy. **Apache wiki**, 2014. Disponível em: <<http://wiki.apache.org/hadoop/PoweredBy%20>>. Acesso em: 3 Julho 2014.

APACHE FOUNDATION. Welcome to Apache Hadoop. **Hadoop**, 2014. Disponível em: <<http://hadoop.apache.org/>>. Acesso em: 20 Agosto 2014.

BAKHSHI, R. How to Plan and Configure YARN and MapReduce 2 in HDP 2.0. **Hortonworks**, 2013 September 2013. Disponível em: <<http://hortonworks.com/blog/how-to-plan-and-configure-yarn-in-hdp-2-0/>>. Acesso em: 30 Julho 2014.

BROWN, R. G. Amdahl's law. **Beowulf Book**, 2004. Disponível em: <https://www.phy.duke.edu/~rgb/Beowulf/beowulf_book/beowulf_book/node21.html>. Acesso em: 1 Agosto 2014.

BURGER, W.; BURGE, M. **Digital image processing – an algorithmic introduction using Java**. [S.l.]: Springer, 2010.

CLOUDERA. About. **Cloudera: Ask Bigger Questions**, 2014. Disponível em: <<http://www.cloudera.com/content/cloudera/en/about.html>>. Acesso em: 20 Agosto 2014.

DAPI. DAPI - Diagnóstico Avançado por Imagem, 2014. Disponível em: <<http://www.dapi.com.br/>>. Acesso em: 20 Agosto 2014.

DEAN, J.; GHEMAWAT, S. MapReduce: Simplified Data Processing on Large Clusters, 2004. Disponível em: <<http://research.google.com/archive/mapreduce.html>>. Acesso em: 1 Agosto 2013.

FANNING, D. W. Converting CT Data to Hounsfield Units. **IDL Coyote**, 2006. Disponível em: <https://www.idlcoyote.com/fileio_tips/hounsfield.html>. Acesso em: 20 Agosto 2014.

GHEMAWAT, S.; GOBIOFF, H.; LEUNG, S.-T. The Google File System. **Google Inc**, 2003. Disponível em: <<http://static.googleusercontent.com/media/research.google.com/en//archive/gfs-sosp2003.pdf>>. Acesso em: 20 Agosto 2014.

GONZALEZ, R. C.; WOODS, R. E.; EDDINS, S. L. **Digital image processing**. 3rd ed. ed. [S.l.]: Pearson Prentice Hall, 2008.

GOOGLE. Google-gson: A Java library to convert JSON to Java Objects and vice-versa. **Google-gson**, 2014. Disponível em: <<https://code.google.com/p/google-gson/>>. Acesso em: 10 Julho 2014.

GURUZON. What is datanode. **Guruzon.com**, 1 June 2013. Disponível em: <<http://www.guruzon.com/6/hadoop-cluster/architecture/what-is-datanode-functions-hadoop-cluster-limitations>>. Acesso em: 20 Julho 2014.

HORTONWORKS. Determine YARN and MapReduce Memory Configuration Settings. **HortonWorks**, 2014. Disponível em: <http://docs.hortonworks.com/HDPDocuments/HDP2/HDP-2.0.9.1/bk_installing_manually_book/content/rpm-chap1-11.html>. Acesso em: 2014 Julho 2014.

HORTONWORKS. Hortonworks. **Hortonworks**, 2014. Disponível em: <<http://hortonworks.com/>>. Acesso em: 15 Julho 2014.

HURWITZ, J. et al. **Big Data for Dummies**. primeira edição. ed. Hoboken: Wiley, v. 1, 2013.

IMAGEJ. Image Processing and Analysis in Java. **ImageJ**, 2014. Disponível em: <<http://imagej.nih.gov/ij/>>. Acesso em: 10 Julho 2014.

INCA. Mortalidade. **Instituto Nacional de Câncer**, 2014. Disponível em: <http://www2.inca.gov.br/wps/wcm/connect/observatorio_controle_tabaco/site/home/dados_numeros/mortalidade+>. Acesso em: 15 Julho 2014.

INTEL CORPORATION. Preboot Execution Environment (PXE) Specification, 1999. Disponível em: <<http://download.intel.com/design/archives/wfm/downloads/pxespec.pdf>>. Acesso em: 3 Julho 2014.

KIM, D.-Y. et al. Pulmonary Nodule Detection Using Chest CT Images. **Acta Radiologica**, 2003. 252-257.

LENGAUER, C.; APEL, S.; DÖRRE, J. MapReduceFoundation: Typing of MapReduce. **MapReduceFoundation**, 2013. Disponível em: <<http://www.infosun.fim.uni-passau.de/cl/MapReduceFoundation/>>. Acesso em: 20 Julho 2014.

LOTZ, M. A. **Dynamic Graph Computations using Parallel Distributed Computing Solutions**. Queen Mary: University of London. London, p. 69. 2013.

LOTZ, M. A. Configure PXE Boot for CentOS 6.5 using FTP, HTTP and Kickstart. **Marco Aurelio Lotz: Thoughts about Big Data and Embedded Systems**,

2014. Disponível em: <<http://www.marcolotz.com/?p=46>>. Acesso em: 10 Agosto 2014.

LOTZ, M. A. HadoopLung source code and javadocs. **Marco Aurélio Lotz github**, 2014. Disponível em: <<https://github.com/MarcoLotz/HadoopLung>>. Acesso em: 8 Agosto 2014.

LOTZ, M. A. Introduction to Hadoop and MapReduce 1.0. **Marco Lotz: Thoughts about Big Data and Embedded Systems**, 2014. Disponível em: <<http://www.marcolotz.com/?p=77>>. Acesso em: 10 Julho 2014.

LOTZ, M. A. Lung Post Processor source code and javadocs. **Marco Aurélio Lotz github**, 2014. Disponível em: <<https://github.com/MarcoLotz/LungPostProcessor>>. Acesso em: 8 Agosto 2014.

LOTZ, M. A. Lung Processor source code and Javadocs. **Marco Aurélio Lotz github**, 2014. Disponível em: <<https://github.com/MarcoLotz/LungProcessor>>. Acesso em: 8 Agosto 2014.

LOTZ, M. A. Of MapReduce and Men. **Marco Aurélio Lotz: Thoughts about Big Data and Embedded Systems**, 16 May 2014. Disponível em: <<http://www.marcolotz.com/?p=67>>. Acesso em: 20 Julho 2014.

LOTZ, M. A. Source code of Matlab Lung Processor. **Marco Aurélio Lotz Github**, 2014. Disponível em: <<https://github.com/MarcoLotz/MatLung>>. Acesso em: 8 Agosto 2014.

MARQUES, O. **Practical image and video processing using MATLAB**. [S.I.]: Wiley, 2011.

MEDICAL CONNECTIONS. DICOM Hierarchy. **Medical Connections**, 2014. Disponível em: <https://www.medicalconnections.co.uk/kb/DICOM_Hierarchy>. Acesso em: 3 Agosto 2014.

MORRIS, C. Hadoop: Toddler Talk Provides Big Data Name. **CNBC**, 2013. Disponível em: <<http://www.cnbc.com/id/100769719#>>. Acesso em: 20 Agosto 2014.

MURPHY, K. P. Device viewable under an imaging beam. **Google Patents**, 2005. Disponível em: <<http://www.google.com/patents/WO2005055806A2?cl=en>>. Acesso em: 20 Agosto 2014.

MURTHY, A. C. The Next Generation of Apache Hadoop MapReduce. **Yahoo!**, 2011. Disponível em: <<https://developer.yahoo.com/blogs/hadoop/next-generation-apache-hadoop-mapreduce-3061.html>>. Acesso em: 1 Julho 2014.

PRODANOV, D. Grayscale Morphology. **Grayscale Morphology**, 2013. Disponível em: <<http://rsb.info.nih.gov/ij/plugins/gray-morphology.html>>. Acesso em: 04 Julho 2014.

ROSENOW, E. C. Diseases and Conditions: Lung Cancer. **Mayo Clinic**, 2014. Disponível em: <<http://www.mayoclinic.org/diseases-conditions/lung-cancer/expert-answers/lung-nodules/faq-20058445>>. Acesso em: 20 Agosto 2014.

SHAPIRO, L. G.; STOCKMAN, G. C. **Computer Vision**. [S.l.]: Prentice Hall, 2001.

TECH-EASE. What is bit-depth. **Teach-ease**, 2011. Disponível em: <<http://etc.usf.edu/techease/win/images/what-is-bit-depth/>>. Acesso em: 20 Julho 2014.

WAGNER, T.; LIPINSKI, H.-G. ImageJ library to detect and analyse connected components (blobs) in binary images. **IJBlob**, 2013. Disponível em: <<http://openresearchsoftware.metajnl.com/article/view/jors.ae>>. Acesso em: 30 Julho 2014.

WHITE, T. The Small Files Problem. **Cloudera: Ask bigger questions**, 02 February 2009. Disponível em: <<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/>>. Acesso em: 30 Julho 2014.

WHITE, T. **Hadoop: The Definitive Guide**. Terceira edição. ed. Sebastopol: O'Reilly, v. 1, 2012.

YAHOO! Module 4: MapReduce. **Apache Hadoop**, 2014. Disponível em: <<https://developer.yahoo.com/hadoop/tutorial/module4.html>>. Acesso em: 3 Julho 2014.

ZHAO, B. et al. Automatic detection of small lung nodules on CT utilizing a local density maximum algorithm. **JOURNAL OF APPLIED CLINICAL MEDICAL PHYSICS**, v. VOLUME 4, n. NUMBER 3, p. 248-260, 2003.

7. ANEXOS

7.1. SCRIPT DE MATLAB

```
% Lung Nodules Candidates Detector
% Author: Marco Aurélio Lotz
% v. 0.2 23/06/2014

clear all;
close all;

%% 1. Pre-Processing
% I guess it's better to work with raw dicom image
% without any kind of pre-processing.

% Load the DICOM Image:
[image] = dicomread('I1400001');

figure (1)
imshow(image,[]);
title('Original DICOM image');

%% 2. Lung Extraction
% According to the paper, set the gray threshold to 375. All values
% below are considered either background or lung parenchyma.

thresholdHU = -375;
% Reference for this conversion:
% http://imagej.1557.x6.nabble.com/Hounsfield-Unit-conversion-td3686965.html

thresholdGray = thresholdHU + 1000;

% Generates the first mask using Gray Threshold.
mask = zeros(size(image,1),size(image,2));
mask(find(image < thresholdGray)) = 1;

% Change from double
mask = uint16(mask);

figure (2)
imshow(mask,[]);
title('Binary mask');

% It is recommended to use a scanning ball (2D disk) of 50X50 pixels by
% the CT paper. In this case I am using 5, since 50x50 looks too large.

% Note, a larger scanning ball (30x30) allows the detections of juxta
```

```

% pleural nodules. Although it merges the two lungs.
% Check this feature on image I7200000
sDiskSize = 5;
se = strel('disk',sDiskSize);
closedMask = imclose(mask,se);

figure (3)
imshow(closedMask,[]);
title('Closed Binary Mask');

% Now one needs to remove the background. In order to perform this, one
% has to find the connected components of the image and remove the ones
% that touch the margin.

connectedComp = bwconncomp(closedMask);

ccMask = closedMask;

% Removes the components that touch the borders
ccMask = imclearborder(ccMask);

figure(4)
imshow(ccMask,[]);
title('Mask of Connected Components');

% Fills any holes inside the lungs mask
noHolesMask = imfill(ccMask,'holes');
figure (5)
imshow(noHolesMask,[]);
title('Filled Holes');

% Extracted Background and Lung Parenchyma
extracted1 = noHolesMask.*image;
figure (6)
imshow(extracted1,[]);
title('Threshold Extraction');

% Removing Aorta. As one may see, the aorta has a really small
% frequency when compared to the remaining of the lung, so the otsu
% threshold should be enough to remove it. The others regions that may
% be removed from the image may be restaured using opennings.

% otsuLevel = graythresh(extracted1);
% otsuMask = im2bw(extracted1,otsuLevel);

% figure (6)
% imshow(otsuMask,[]);
% title('otsuMask');

% Final image of this section:

```

```

output = extracted1;

%% 3. Nodes Candidates Detection

input = extracted1;

% According to the paper:
% 2003_LUNG_Pulmonary Nodule Detection using chest CT images
% "A gray value between 110 and 120 is used to extract the nodule
% candidates bigger than 5x5 pixels." The paper, however, uses a byte
% per pixel, when the standard DICOM format uses 2 bytes per pixel
% (uint16). Thus, the value needs to be converted.
% TODO: Check the info above.

% According to the paper:
minimumRangeByte = 110;
maximumRangeByte = 120;
byteRange = 255;

% Found this value using a rule of three, one needs to double check
% how valid is this conversion
maximumRangeDICOM = 2235;
minimumRangeDICOM = minimumRangeByte * maximumRangeDICOM / byteRange;
maximumRangeDICOM = maximumRangeByte * maximumRangeDICOM / byteRange;

% Let's find the regions that are within this grey level:
maskRegion = zeros(size(input,1),size(input,2));
maskRegion(find(input > minimumRangeDICOM & input < maximumRangeDICOM)) = 1;

figure (7)
imshow(maskRegion,[]);
title('Mask for regions in the grey boundary');

% Selects only connected regions larger than 5x5 pixels:
% (Connectivity 8)
connectedCandidates = bwconncomp(maskRegion);

% Bouding Box will be used to calculate the size of the component.
CandidatesProps = regionprops(connectedCandidates, 'BoundingBox');

numberOfCandidates = connectedCandidates.NumObjects;

% New mask that will only have objects larger than 5x5 pixels.
largeCandidatesMask = maskRegion;

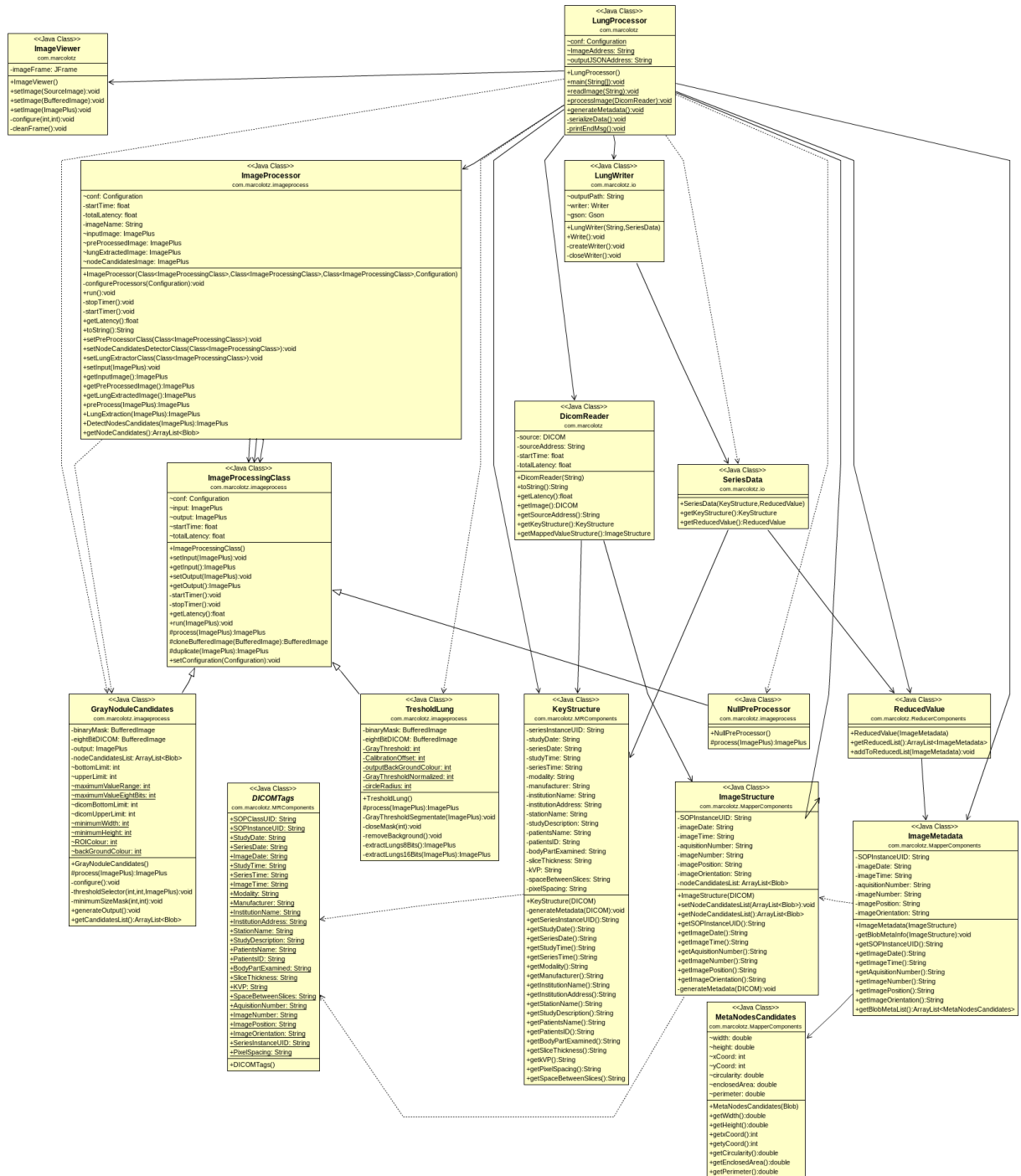
for count = 1:numberOfCandidates
    Buffer = CandidatesProps(count).BoundingBox;
    % If the bounding box is smaller than 5x5
    if (Buffer(3)<5 || Buffer(4) < 5)
        %Erases the candidates that fail the test.
        largeCandidatesMask(connectedCandidates.PixelIdxList{count}) = 0;
    end
end

```

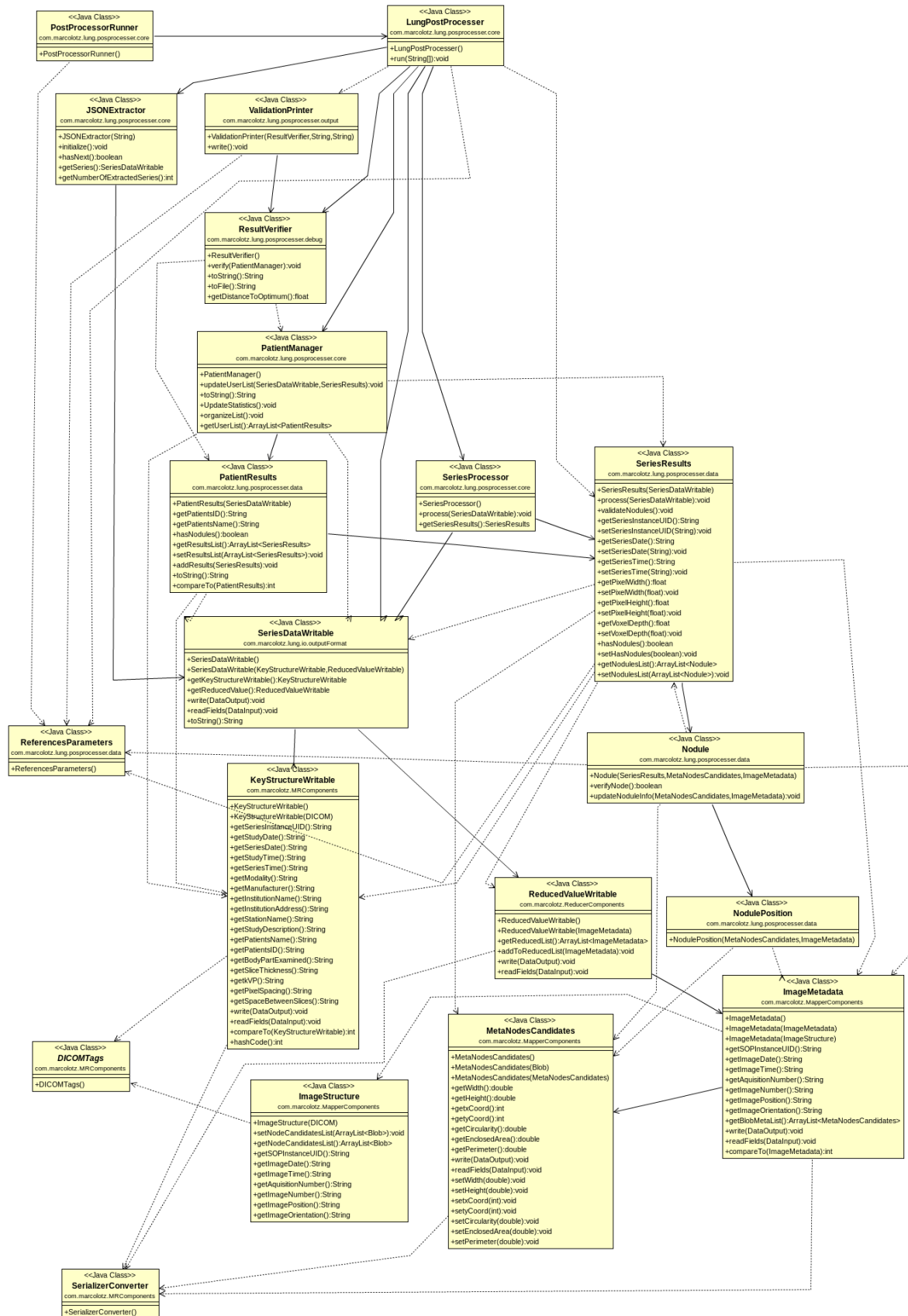
```
    end
end

figure (8)
imshow(largeCandidatesMask,[]);
title('Mask for candidates larger than 5x5 pixels');
```

7.2. UML LUNG PROCESSOR



7.4. UML LUNG POST-PROCESSOR



7.5. ARQUIVO DE CONFIGURAÇÃO DO HADOOP

```
<?xml version="1.0"?>
<!--
XML configuration file written by: Marco Aur lio Lotz
(marcolotz.com)
It configures the HadoopLung processing Hadoop application.
-->
<configuration>

  <property>
    <name>com.marcolotz.author</name>
    <value>Marco Aurelio Lotz (marcolotz.com)</value>
    <description>Application author info</description>
  </property>

  <property>
    <name>mapreduce.job.name</name>
    <value>Hadoop Lung Processor</value>
    <description>Application name</description>
  </property>

  <property>
    <name>mapreduce.job.map.class</name>
    <value>com.marcolotz.lung.mapper.LungMapper</value>
    <description>Lung Processor Mapper Class</description>
  </property>

  <property>
    <name>mapreduce.job.reduce.class</name>
    <value>com.marcolotz.lung.reducer.LungReducer</value>
    <description>Lung Processor Reducer Class</description>
  </property>

  <property>
    <name>mapreduce.job.maps</name>
    <value>50</value>
    <description>Number of mappers tasks</description>
  </property>

  <property>
    <name>mapreduce.job.reduces</name>
    <value>50</value>
    <description>Number of reduce tasks</description>
  </property>

  <property>
    <name>mapreduce.reduce.memory.mb</name>
    <value>1024</value>
    <description>Reduce memory in Mbytes</description>
  </property>

  <property>
    <name>mapreduce.job.inputformat.class</name>
    <value>com.marcolotz.lung.io.inputFormat.MultipleFilesInputFormat</value>
    <description>Class that reads the DICOM files</description>
  </property>

  <property>
```

```

        <name>mapred.input.dir.recursive</name>
        <value>true</value>
        <description>Forces the input directory to be read recursively</description>
    </property>

    <property>
        <name>mapred.input.dir</name>
        <value>/user/marco/TCC/input/onlyLungs</value>
        <description>The input path of the MapReduce Application</description>
    </property>

    <property>
        <name>mapred.output.dir</name>
        <value>/user/marco/TCC/output/onlyLungs</value>
        <description>The output path of the MapReduce Application</description>
    </property>

    <property>
        <name>mapreduce.map.output.key.class</name>
        <value>com.marcolotz.MRComponents.KeyStructureWritable</value>
        <description>Key class on map output</description>
    </property>

    <property>
        <name>mapreduce.map.output.value.class</name>
        <value>com.marcolotz.MapperComponents.ImageMetadata</value>
        <description>Value class on map output</description>
    </property>

    <property>
        <name>mapreduce.job.output.key.class</name>
        <value>com.marcolotz.lung.io.outputFormat.SeriesDataWritable</value>
        <description>Job output key</description>
    </property>

    <property>
        <name>mapreduce.job.output.value.class</name>
        <value>org.apache.hadoop.io.NullWritable</value>
        <description>job output value</description>
    </property>

    <property>
        <name>com.marcolotz.grayNoduleCandidates.bottomThreshold</name>
        <value>110</value>
        <description>Default bottom threshold used by the GrayNodulesCandidates class</description>
    </property>

    <property>
        <name>com.marcolotz.grayNoduleCandidates.topThreshold</name>
        <value>120</value>
        <description>Default top threshold used by the GrayNodulesCandidates class</description>
    </property>
</configuration>

```

7.6. ROCPROCESS.SH

```
# Script created by Marco Lotz (marcolotz.com)
#
# It calls hadoop jobs taking into account the bot and top thresholds for gray
# values.

HadoopJarAddress="/home/marco/Desktop/Hadoop/HadoopLung.jar"
MapReduceOutput="/home/marco/Desktop/TCC/output/onlyLungs/Benchmark/"

bottomGray=110
topGray=135
minRange=5

counterBot=$bottomGray
counterTop=$topGray

echo ""
echo "Script generated for ROC comparison of gray threshold on Lung processing"
echo "author: Marco Lotz (marcolotz.com)"

Hadoop jar address: $HadoopJarAddress
MapReduce local disk output path: $MapReduceOutput
"

for counterBot in `seq $bottomGray $counterTop`
do
    counterTop=$topGray

    #The minimum value is always minRange plus the counterBot
    minCounterTop=`echo "$counterBot + $minRange" | bc`
    for counterTop in `seq $topGray -1 $minCounterTop`
    do
        echo "bottom gray level: $counterBot"
        echo "top gray level: $counterTop"
        echo -n "Calling the Hadoop job for the following threshold:"
        #No need for output to user.
        hadoop jar $HadoopJarAddress $counterBot $counterTop 1>&- 2>&-
        echo " [DONE]"

        echo -n "transferring Hadoop data to local Disk: "
        hadoop fs -getmerge /user/marco/TCC/output/onlyLungs
$MapReduceOutput"/output"$counterBot"_"$counterTop".json"
        echo " [DONE]"
        echo ""
    done
done
done
```

7.7. POSTPROCESS.SH

```
# Script created by Marco Lotz (marcolotz.com)
#
# Post Processes all .json files in given input directory (it should contain the output of
# MapReduce processing step). The Post Processing application has a user defined output
# class, that will write the output in the output Path. In the current class, it writes
# normalized true positives and false positives in order to generate a ROC graphic.

PostProcessingJarAddress="/home/prometheus/Dropbox/TCC/code/Jar/LungPostProcessor.jar"

PostProcessingOutput="/home/prometheus/Dropbox/TCC/output/postProcessing/ROCValues.data"
MapReduceOutputPath="/home/prometheus/Dropbox/TCC/output/hadoop/onlyLungs/Benchmark/"

# Values that will be used for centroid distance
CENTROID_DISTANCES="1 2 3 4 5"

# Values that will be used for minimum size. Remember that all the exams with nodules have
# nodules larger than 8 millimeters
MINIMUM_SIZE="2 4 6 8"
MAXIMUM_SIZE="40 30 20 15"

echo ""
echo "Script generated for ROC graph data generation using post processing application."
echo "author: Marco Lotz (marcolotz.com)"
echo ""

echo "Changing path to :"$MapReduceOutputPath
echo ""
cd $MapReduceOutputPath

for file in *
do
    for centroid in $CENTROID_DISTANCES
    do
        for minimumSize in $MINIMUM_SIZE
        do
            for maximumSize in $MAXIMUM_SIZE
            do
                echo "Post Processing file: $file."
                echo "centroid: $centroid minSize: $minimumSize maxSize: $maximumSize"
                java -jar $PostProcessingJarAddress $MapReduceOutputPath$file $PostProcessingOutput
                $centroid $minimumSize $maximumSize 1>&- 2>&-
            done
        done
    done
done

echo "Post processing ROC values calculation completed."
```

7.8. FINDSOLUTIONS.SH

```
# Script created by Marco Lotz (marcolotz.com)
#
# Iterates over all the solution in the post-processed output and looks for the
# five better solutions (greater true positive lesser false positive). With this
# one can find the better parameter configurations.
#
# The best solutions the lesser value of the last field, that is the euclidean
# distance to the ideal solution.

INPUT_PATH=$1
NUMBERLINES=$2

function wrongUsage(){
    echo "Usage: $0 [input path] [number of top elements to print]"
}

if [ $# -ne 2 ]
then
    wrongUsage
else
    echo ""
    echo "Looking for the best $NUMBERLINES solutions on input file: $INPUT_PATH"
    cat $INPUT_PATH | awk -f solutions.awk -v arraySize=$NUMBERLINES
fi

echo "Script successfully finished."
```

7.9. SOLUTIONS.AWK

```
#!/bin/awk
# AWK script written by Marco Aurelio Lotz (marcolotz.com)
# Selects the three lines with lesser distance from the optimum results.

# The lines have the following fields format:
# (1) Bottom grey treshold (2) Upper grey threshold (3) Maximum centroid distance
# (4) Minimum nodule width/height (5) Maximum nodule width/height (6) rate of
# true positives (7) rate of false positives (8) Distance from top left ROC
# result.

BEGIN {
    #field separator is the \t
    fs=" ";

    # The array size contains the number of elements that will be displayed
    # and is set by the user through the command line.

    for(i=0; i<arraySize;i++)
    {
        #lines with the best solution
        solutions[i] = "";
        #values for the best solution
        values[i] = 2;
    }
}

#For all records:
/{
    #Euclidean distance of that line.
    lineValue = $8;
    completeLine = $0;

    for (counter = 0; counter < arraySize; counter++)
    {
        if (values[counter] > lineValue)
        {
            #saves the line value in a buffer.
            savedValue = lineValue;
            savedLine = completeLine;
            # update the min value for the next iteration of the loop, so the next element also
updates.
            lineValue = values[counter];
            completeLine = solutions[counter];

            #Updates current values.
            values[counter] = savedValue;
            solutions[counter] = savedLine;
        }
    }
}

# At the end, print all the solutions.
END{

    print "# AWK Script written by Marco Lotz (marcolotz.com)"
    print "# The output syntax is:"
    print "# Bottom grey treshold | Upper grey threshold | Maximum centroid distance| Minimum
```

```
nodule dimensions | Maximum nodule dimensions | rate of true positives | rate of false positives |  
Distance from top left ROC result\n"  
    print "# Printing the most effective parameters, from most effective to least effective:\n"  
  
    for(i=0; i<arraySize;i++)  
    {  
        print solutions[i];  
    }  
}
```


7.10. ITERATESOLUTION.SH

```
# Script created by Marco Lotz (marcolotz.com)
#
# Post Processes a specific .json file in given input directory (it should contain the output of
# MapReduce processing step). The Post Processing application has a user defined output
# class, that will write the output in the output Path.
# This application main purpose is to analyse multiple parameters for a selected gray bottom
# and upper threshold in order to find an optimum solution.

MINIMUM_THRESHOLD=$1
MAXIMUM_THRESHOLD=$2

PostProcessingJarAddress="/home/prometheus/Dropbox/TCC/code/Jar/LungPostProcessor.jar"

PostProcessingOutput="/home/prometheus/Dropbox/TCC/output/postProcessing/iterateSolutions/itera
teSolution$MINIMUM_THRESHOLD_"$MAXIMUM_THRESHOLD.data"
MapReduceOutputPath="/home/prometheus/Dropbox/TCC/output/hadoop/onlyLungs/Benchmark/"

# Values that will be used for centroid distance
CENTROID_DISTANCES="0.01 0.1 0.2 0.3 0.4 0.5 0.6 0.8 1 1.5 2 3 4 5"

# Values that will be used for minimum size. Remember that all the exams with nodules have
# nodules larger than 8 millimeters
MINIMUM_SIZE="2 4 6 8 10 12 14"
MAXIMUM_SIZE="40 35 30 25 20 15"

echo ""
echo "Script generated for iterating a single post process threshold parameters to find optimum
points."
echo "author: Marco Lotz (marcolotz.com)"
echo ""

if [ "$#" -ne 2 ];
then
    echo "Wrong usage:"
    echo "$0 [bottom grey threshold] [top grey threshold]"
else

    echo "Changing path to :"$MapReduceOutputPath
    echo ""
    cd $MapReduceOutputPath

    file="output$MINIMUM_THRESHOLD_"$MAXIMUM_THRESHOLD.json"

    if [ -e "$file" ];
    then
        for centroid in $CENTROID_DISTANCES
        do
            for minimumSize in $MINIMUM_SIZE
            do
                for maximumSize in $MAXIMUM_SIZE
                do
                    echo "Post Processing file: $file."
                    echo "centroid: $centroid  minSize: $minimumSize maxSize:
$maximumSize"

                    java -jar $PostProcessingJarAddress
                    $MapReduceOutputPath$file $PostProcessingOutput $centroid $minimumSize $maximumSize 1>&-
```

```
2>&-
done
done
done
echo "Post processing ROC values calculation completed."
else
    echo "File not found: $file"
fi
fi
```

7.11. SAÍDA DO HADOOP APÓS OTIMIZAÇÃO FINAL

```
[marco@server Hadoop]$ hadoop jar HadoopLung.jar
```

```
Application author: Marco Aurelio Lotz (marcolotz.com)
```

```
Configurations stored at: conf/lungConfiguration.xml
```

```
Input path: /user/marco/TCC/input/onlyLungs
```

```
Output path: /user/marco/TCC/output/onlyLungs
```

```
Cleaning output path: [DONE]
```

```
Configuring the job Hadoop Lung Processor: [DONE]
```

```
Distributing the job:
```

```
14/07/19 15:04:39 INFO client.RMProxy: Connecting to ResourceManager at  
server.cluster/10.10.15.1:8032
```

```
14/07/19 15:04:45 INFO input.FileInputFormat: Total input paths to process : 41305
```

```
14/07/19 15:04:45 INFO input.CombineFileInputFormat: DEBUG: Terminated node allocation with :  
CompletedNodes: 4, size left: 69896450
```

```
14/07/19 15:04:45 INFO mapreduce.JobSubmitter: number of splits:197
```

```
14/07/19 15:04:46 INFO mapreduce.JobSubmitter: Submitting tokens for job:  
job_1405742103144_0005
```

```
14/07/19 15:04:46 INFO impl.YarnClientImpl: Submitted application  
application_1405742103144_0005
```

```
14/07/19 15:04:47 INFO mapreduce.Job: The url to track the job:
```

```
http://server.cluster:8088/proxy/application\_1405742103144\_0005/
```

```
14/07/19 15:04:47 INFO mapreduce.Job: Running job: job_1405742103144_0005
```

```
14/07/19 15:04:54 INFO mapreduce.Job: Job job_1405742103144_0005 running in uber mode :  
false
```

```
14/07/19 15:04:54 INFO mapreduce.Job: map 0% reduce 0%
```

```
14/07/19 15:05:09 INFO mapreduce.Job: map 1% reduce 0%
```

```
14/07/19 15:05:14 INFO mapreduce.Job: map 2% reduce 0%
```

```
14/07/19 15:05:19 INFO mapreduce.Job: map 3% reduce 0%
```

```
14/07/19 15:05:24 INFO mapreduce.Job: map 4% reduce 0%
```

```
14/07/19 15:05:28 INFO mapreduce.Job: map 5% reduce 0%
```

```
14/07/19 15:05:32 INFO mapreduce.Job: map 6% reduce 0%
```

```
14/07/19 15:05:35 INFO mapreduce.Job: map 7% reduce 0%
```

```
14/07/19 15:05:38 INFO mapreduce.Job: map 8% reduce 0%
```

```
14/07/19 15:05:41 INFO mapreduce.Job: map 9% reduce 0%
```

```
14/07/19 15:05:44 INFO mapreduce.Job: map 10% reduce 0%
```

```
14/07/19 15:05:46 INFO mapreduce.Job: map 11% reduce 0%
```

```
14/07/19 15:05:49 INFO mapreduce.Job: map 12% reduce 0%
```

```
14/07/19 15:05:52 INFO mapreduce.Job: map 13% reduce 0%
```

```
14/07/19 15:05:55 INFO mapreduce.Job: map 14% reduce 0%
```

```
14/07/19 15:05:58 INFO mapreduce.Job: map 15% reduce 0%
```

```
14/07/19 15:06:01 INFO mapreduce.Job: map 16% reduce 0%
```

```
14/07/19 15:06:03 INFO mapreduce.Job: map 17% reduce 0%
```

```
14/07/19 15:06:06 INFO mapreduce.Job: map 18% reduce 0%
```

```
14/07/19 15:06:09 INFO mapreduce.Job: map 19% reduce 0%
```

```
14/07/19 15:06:12 INFO mapreduce.Job: map 20% reduce 0%
```

```
14/07/19 15:06:14 INFO mapreduce.Job: map 21% reduce 0%
```

```
14/07/19 15:06:17 INFO mapreduce.Job: map 22% reduce 0%
```

```
14/07/19 15:06:20 INFO mapreduce.Job: map 23% reduce 0%
```

```
14/07/19 15:06:23 INFO mapreduce.Job: map 24% reduce 0%
```

```
14/07/19 15:06:25 INFO mapreduce.Job: map 25% reduce 0%
```

```
14/07/19 15:06:27 INFO mapreduce.Job: map 26% reduce 0%
```

```
14/07/19 15:06:29 INFO mapreduce.Job: map 27% reduce 0%
```

```
14/07/19 15:06:32 INFO mapreduce.Job: map 28% reduce 0%
```

```
14/07/19 15:06:33 INFO mapreduce.Job: map 29% reduce 0%
```

[illegible]

```

14/07/19 15:08:34 INFO mapreduce.Job: map 93% reduce 0%
14/07/19 15:08:36 INFO mapreduce.Job: map 95% reduce 0%
14/07/19 15:08:39 INFO mapreduce.Job: map 96% reduce 0%
14/07/19 15:08:41 INFO mapreduce.Job: map 97% reduce 0%
14/07/19 15:08:43 INFO mapreduce.Job: map 97% reduce 3%
14/07/19 15:08:44 INFO mapreduce.Job: map 97% reduce 7%
14/07/19 15:08:45 INFO mapreduce.Job: map 97% reduce 11%
14/07/19 15:08:46 INFO mapreduce.Job: map 98% reduce 13%
14/07/19 15:08:47 INFO mapreduce.Job: map 98% reduce 19%
14/07/19 15:08:48 INFO mapreduce.Job: map 98% reduce 21%
14/07/19 15:08:49 INFO mapreduce.Job: map 98% reduce 25%
14/07/19 15:08:50 INFO mapreduce.Job: map 98% reduce 29%
14/07/19 15:08:51 INFO mapreduce.Job: map 98% reduce 32%
14/07/19 15:09:04 INFO mapreduce.Job: map 99% reduce 32%
14/07/19 15:09:12 INFO mapreduce.Job: map 99% reduce 33%
14/07/19 15:09:14 INFO mapreduce.Job: map 100% reduce 33%
14/07/19 15:09:18 INFO mapreduce.Job: map 100% reduce 35%
14/07/19 15:09:20 INFO mapreduce.Job: map 100% reduce 56%
14/07/19 15:09:20 INFO mapreduce.Job: map 100% reduce 90%
14/07/19 15:09:21 INFO mapreduce.Job: map 100% reduce 100%
14/07/19 15:09:22 INFO mapreduce.Job: Job job_1405742103144_0005 completed successfully
14/07/19 15:09:22 INFO mapreduce.Job: Counters: 50

```

File System Counters

```

FILE: Number of bytes read=5204822
FILE: Number of bytes written=33532715
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=26491025013
HDFS: Number of bytes written=46093116
HDFS: Number of read operations=41843
HDFS: Number of large read operations=0
HDFS: Number of write operations=96

```

Job Counters

```

Launched map tasks=197
Launched reduce tasks=48
Data-local map tasks=196
Rack-local map tasks=1
Total time spent by all maps in occupied slots (ms)=19400430
Total time spent by all reduces in occupied slots (ms)=1996797
Total time spent by all map tasks (ms)=19400430
Total time spent by all reduce tasks (ms)=1996797
Total vcore-seconds taken by all map tasks=19400430
Total vcore-seconds taken by all reduce tasks=1996797
Total megabyte-seconds taken by all map tasks=19866040320
Total megabyte-seconds taken by all reduce tasks=2044720128

```

Map-Reduce Framework

```

Map input records=41305
Map output records=41305
Map output bytes=24303808
Map output materialized bytes=5473915
Input split bytes=5160127
Combine input records=0
Combine output records=0
Reduce input groups=248
Reduce shuffle bytes=5473915
Reduce input records=41305
Reduce output records=248
Spilled Records=82610
Shuffled Maps =9456

```

```
Failed Shuffles=0
Merged Map outputs=9456
GC time elapsed (ms)=192374
CPU time spent (ms)=17760830
Physical memory (bytes) snapshot=122276225024
Virtual memory (bytes) snapshot=392579117056
Total committed heap usage (bytes)=123562622976
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=0
File Output Format Counters
  Bytes Written=46093116
```

Apêndice A



Ministério da Educação
Universidade Tecnológica Federal do Paraná
Pró-Reitoria de Graduação e Educação Profissional
Pró-Reitoria de Pesquisa e Pós-Graduação
Sistema de Bibliotecas

DECLARAÇÃO DE AUTORIA

Autor: Marco Aurélio Barbosa Fagnani Gomes Lotz

CPF: 080.541.029-50 Código de matrícula: 1019171

Telefone: (41) 9968-7424 e-mail: marco@marcolotz.com

Curso: Engenharia Industrial Elétrica: Ênfase em Eletrônica e Telecomunicações

Orientador: Gustavo Benvenuti Borba

Co-orientador: Charlie Antoni Miquelin

Data da defesa: 29/08/2014

Título/subtítulo: Processamento Distribuído Extensível Utilizando um Cluster e MapReduce com Aplicação na Detecção de Nódulos Pulmonares para Grandes Volumes de Tomografias

Tipo de produção intelectual: (X) TCC () TCCE () Dissertação () Tese

Declaro(amos), para os devidos fins, que o presente trabalho é de minha/nossa autoria e que estou(amos) ciente(s):

- dos Artigos 297 a 299 do Código Penal, Decreto-Lei nº 2.848 de 7 de dezembro de 1940;
- da Lei nº 9.610, de 19 de fevereiro de 1998, sobre os Direitos Autorais,
- do Regulamento Disciplinar do Corpo Discente da UTFPR; e
- que plágio consiste na reprodução de obra alheia e submissão da mesma como trabalho próprio ou na inclusão, em trabalho próprio, de idéias, textos, tabelas ou ilustrações (quadros, figuras, gráficos, fotografias, retratos, lâminas, desenhos, organogramas, fluxogramas, plantas, mapas e outros) transcritos de obras de terceiros sem a devida e correta citação da referência.

Curitiba, 03 de setembro de 2014

Marco Aurélio B. F. G. Lotz
Nome: Marco Aurélio Barbosa Fagnani Gomes Lotz

Apêndice B



Ministério da Educação
Universidade Tecnológica Federal do Paraná
Pró-Reitoria de Graduação e Educação Profissional
Pró-Reitoria de Pesquisa e Pós-Graduação
Sistema de Bibliotecas

Termo de Autorização para Publicação de Trabalhos de Conclusão de Curso de Graduação e Especialização, Dissertações e Teses no Portal de Informação e nos Catálogos Eletrônicos do Sistema De Bibliotecas da UTFPR

Na qualidade de titular dos direitos de autor da publicação, autorizo a UTFPR a veicular, através do Portal de Informação (PIA) e dos Catálogos das Bibliotecas desta Instituição, sem ressarcimento dos direitos autorais, de acordo com a Lei nº 9.610/98, o texto da obra abaixo citada, observando as condições de disponibilização no item 4, para fins de leitura, impressão e/ou *download*, visando a divulgação da produção científica brasileira.

1. Tipo de produção intelectual: (X) TCC¹ () TCCE² () Dissertação () Tese

2. Identificação da obra:

Autor: Marco Aurélio Barbosa Fagnani Gomes Lotz e-mail: marco@marcolotz.com

RG: 7.260.022-3 CPF: 080.541.029-50 Telefone: (41) 9968-7424

Curso/Programa de Pós-graduação: Engenharia Industrial Elétrica: Ênfase em Eletrônica e Telecomunicações

Orientador: Gustavo Benvenutti Borba

Co-orientador: Charlie Antoni Miquelin

Data da defesa: 29/08/2014

Título/subtítulo (português): Processamento Distribuído Extensível Utilizando um Cluster e MapReduce com Aplicação na Detecção de Nódulos Pulmonares para Grandes Volumes de Tomografias

Título/subtítulo em outro idioma: Extensible Distributed Processing Using a Cluster and MapReduce with Applications on Lung Nodules Detection for Large Sets of Tomographies

Área de conhecimento do CNPq: Processamento Digital de Imagens, Computação Distribuída

Palavras-chave: Hadoop, detecção de nódulos pulmonares, Big Data, computação distribuída, processamento de grandes volumes de dados, processamento de imagens médicas, computação de alta performance, MapReduce.

Palavras-chave em outro idioma: Hadoop, lung nodules detection, Big Data, distributed computing, large datasets computing, medical images processing, High Performance Computing, MapReduce.

3. Agência(s) de fomento (quando existir): _____

4. Inf. de disponibilização do documento: Restrição para publicação: () Total⁴ () Parcial (X) Não Restringir

Em caso de restrição total, especifique o porquê da restrição: _____

Em caso de restrição parcial, especifique capítulo(s) restrito(s): _____

Curitiba, 03 de setembro de 2014

Orientador: Gustavo Benvenutti Borba

Nome: Marco Aurélio Barbosa Fagnani Gomes Lotz

⁴ A restrição parcial ou total para publicação com informações de empresas será mantida pelo período especificado no Termo de Autorização para Divulgação de Informações de Empresas. A restrição total para publicação de trabalhos que forem base para a geração de patente ou registro será mantida até que seja feito o protocolo do registro ou depósito de PI junto ao INPI pela Agência de Inovação da UTFPR. A íntegra do resumo e os metadados ficarão sempre disponibilizados.